

Computational Biology, Biostatistics, and AI/ML for Precision Medicine

A Graduate Textbook and Professional Reference for
Multi-Omics Integration, Machine Learning, and
Biostatistics

Eskezeia Yihunie Dessie, PhD

August 2026

Contents

Copyright	1
Dedication	2
Preface	3
1 Biostatistics and Statistical Modeling	5
1.1 Bayesian vs. Frequentist Methods	5
1.2 Survival Analysis	7
1.2.1 Kaplan-Meier Estimator	7
1.2.2 Cox Proportional Hazards Model	7
1.2.3 Code (R — survival)	7
1.3 Longitudinal / Mixed-Effects Models (Laird & Ware, 1982; Pinheiro & Bates, 2000)	7
1.3.1 Example Result — Mixed-Effects Model Output	8
1.3.2 Code (R — lme4)	9
1.4 Logistic / Linear Regression	9
1.4.1 Code (R)	9
1.5 High-Dimensional Modeling (LASSO, Tibshirani, 1996; Elastic Net, Zou & Hastie, 2005)	9
1.5.1 Code (R — glmnet)	10
1.6 Hypothesis Testing	10
1.7 Diagnostic Test Evaluation	10
1.8 Assay Performance Modeling	10
1.9 Experimental Design & Power Analysis	10
1.9.1 Code (R — pwr)	10
1.10 Multiple Testing Correction (Benjamini & Hochberg, 1995)	11
1.10.1 Choosing a Correction: It Depends on Whether the Analysis Is Hypothesis-Driven or Discovery-Driven	11
1.10.2 Career Application	11
1.10.3 Summary Table — Module 3	11
1.11 References	12
2 Computational Biology and Bioinformatics	17
2.1 Statistical Genetics & Genomics	17
2.1.1 Conceptual Foundation	17
2.1.2 GWAS Statistical Model	17
2.1.3 Real Data Example	18
2.1.4 Workflow (GWAS Pipeline)	18
2.1.5 Code	18
2.1.6 Pitfalls & Best Practices	18

2.1.7	Interpretation Guidelines	19
2.2	Genomic Data Processing	19
2.2.1	Conceptual Foundation	19
2.2.2	Workflow	19
2.2.3	Example Result — VCF Summary Statistics (bcftools stats, Simulated Trio)	20
2.2.4	Code	21
2.2.5	Pitfalls	21
2.3	Multi-Omics Integration	21
2.3.1	Conceptual Foundation	21
2.3.2	Statistical Framework — eQTM (expression Quantitative Trait Methylation)	21
2.3.3	Multi-Block Integration Methods	22
2.3.4	Real Data Example	22
2.3.5	Workflow	22
2.3.6	Code (R — MOFA+)	22
2.3.7	Pitfalls	23
2.3.8	Practical Integration Strategy: Discovery vs. Confirmation .	23
2.3.9	Common Technical Pitfalls in Cross-Platform Integration . .	23
2.3.10	Missing Data Across Omics Platforms — A Decision Frame- work	24
2.3.11	Career Application	24
2.4	RNA-seq Differential Expression & Batch Effects	24
2.4.1	Conceptual Foundation	24
2.4.2	Pipeline (Raw Reads → DE Results)	25
2.4.3	DESeq2 vs. edgeR — Practical Differences	25
2.4.4	Batch Effects in RNA-seq	25
2.4.5	Code (R — DESeq2 with batch covariate)	25
2.4.6	Pitfalls	26
2.5	DNA Methylation QC & Cell-Type Heterogeneity	26
2.5.1	850K/EPIC Array QC Pipeline	26
2.5.2	Cell-Type Heterogeneity — Why It Matters	26
2.5.3	cis vs. trans eQTM/mQTL — Why Significance Thresholds Differ	27
2.5.4	Code (R — cell-type deconvolution)	27
2.5.5	Pitfalls	27
2.6	Biomarker Discovery	27
2.6.1	Conceptual Foundation	27
2.6.2	Key Metrics	27
2.6.3	Workflow	28
2.6.4	Code (R — ROC/AUC)	28
2.6.5	Pitfalls	28
2.6.6	From Statistically Significant to Clinically Actionable	28
2.6.7	The Validation Ladder (Discovery → Clinical Deployment) .	28
2.6.8	Career Application	29
2.7	Infectious Disease Genomics	29

2.7.1	Conceptual Foundation	29
2.7.2	Key Methods	29
2.7.3	Workflow	29
2.7.4	Pitfalls	30
2.8	Variant Calling & QC	30
2.8.1	Tools Summary Table	30
2.8.2	Code	30
2.8.3	Pitfalls	31
2.9	Network & Pathway Analysis	31
2.9.1	Conceptual Foundation	31
2.9.2	Methods	31
2.9.3	Tools	31
2.9.4	Code (R)	31
2.9.5	Pitfalls	31
2.10	Mendelian Randomization (Davey Smith & Ebrahim, 2003)	32
2.10.1	Conceptual Foundation	32
2.10.2	Core Assumptions	32
2.10.3	Formula (Two-Sample MR, Inverse-Variance Weighted) . . .	32
2.10.4	Code (R — TwoSampleMR)	32
2.10.5	Pitfalls	32
2.11	Meta-Analysis	33
2.11.1	Conceptual Foundation	33
2.11.2	Fixed vs. Random Effects	33
2.11.3	Heterogeneity	33
2.11.4	Code (R — metafor)	33
2.11.5	Pitfalls	33
2.12	CpG / Gene / Variant Prioritization	33
2.12.1	Conceptual Foundation	33
2.12.2	Prioritization Framework	34
2.12.3	Pitfalls	34
2.13	References	34

3 Survival Analysis: Classical, Machine-Learning, and Multimodal Methods

3.1	Foundations of Time-to-Event Data	36
3.1.1	Core Definitions	36
3.1.2	Censoring Types	36
3.1.3	Assumption Common to Nearly All Classical Survival Models	37
3.2	Kaplan-Meier Estimator	37
3.2.1	Example Result — Log-Rank Test Output	38
3.2.2	Code (R)	39
3.3	Cox Proportional Hazards Model	39
3.3.1	Example Result — Cox Model Output Table	40
3.3.2	Code (R)	41
3.4	Parametric Survival Models	41
3.4.1	Example Result — Model Selection by AIC	44
3.4.2	Code (R)	44

3.5	Accelerated Failure Time (AFT) Models	44
3.5.1	Example Result — AFT (Weibull) Output	45
3.5.2	Code (R)	45
3.6	Competing Risks (Fine-Gray Model)	46
3.6.1	Example Result — Fine-Gray Model Output	47
3.6.2	Code (R)	48
3.7	Recurrent Event Models (Andersen-Gill, PWP, WLW)	48
3.7.1	Example Result — Recurrent Exacerbations (Andersen-Gill Model)	49
3.7.2	Code (R)	49
3.8	Frailty Models (Random-Effects Survival Models)	50
3.8.1	Code (R)	51
3.9	Joint Models (Longitudinal + Survival)	51
3.9.1	Example Result — Joint Model Association Parameter	52
3.9.2	Code (R)	52
3.10	Bayesian Survival Models	52
3.10.1	Code (R)	53
3.11	Machine-Learning Survival Models	54
3.11.1	Example Result — Model Discrimination Comparison	55
3.11.2	Code (Python — scikit-survival)	55
3.12	Deep-Learning Survival Models	56
3.12.1	Example Result — DeepSurv vs. Cox Discrimination and Calibration	58
3.12.2	Code (Python — pycox)	58
3.13	Multimodal Survival Models (Image + Omics + EHR Fusion)	58
3.13.1	Example Result — Multimodal Survival Benchmark (Simulated Oncology Cohort)	59
3.13.2	Code (Python — conceptual multimodal DeepSurv)	60
3.14	Chapter Summary: Choosing a Survival Model	60
3.15	References	61
4	Artificial Intelligence and Machine Learning	64
4.1	Predictive Modeling — Foundations	64
4.1.1	Bias-Variance Tradeoff	64
4.1.2	Loss Functions	64
4.2	Ensemble Methods	64
4.2.1	Random Forest (Breiman, 2001)	64
4.2.2	Gradient Boosting (XGBoost, Chen & Guestrin, 2016; LightGBM, Ke et al., 2017)	64
4.2.3	Code (Python — XGBoost)	65
4.2.4	Code (R — LightGBM style via lightgbm pkg)	65
4.2.5	Choosing Between Ensembles and Deep Learning — A Practical Decision Rule	65
4.2.6	Preventing Overfitting on High-Dimensional Genomic Data (p » n) — A Layered Checklist	65
4.2.7	Career Application	66
4.3	Deep Learning	66

4.3.1	CNN (for imaging/sequence motifs)	66
4.3.2	RNN/LSTM (Hochreiter & Schmidhuber, 1997) (for sequential/longitudinal data)	66
4.3.3	Code (Python — PyTorch CNN skeleton)	66
4.3.4	Pitfalls	67
4.4	Explainable AI (SHAP, LIME)	67
4.4.1	SHAP (Shapley values; Lundberg & Lee, 2017)	68
4.4.2	Example Result — Concordance Between SHAP and Known Clinical Risk Factors	68
4.4.3	Code (Python)	69
4.4.4	Pitfalls	69
4.5	Large Language Models (LLMs) for Biological Data	70
4.5.1	Applications	70
4.5.2	Pitfalls	70
4.6	Causal Inference & Mediation Modeling	71
4.6.1	Mediation Model	71
4.6.2	Causal Frameworks (relevant to your CIMLA/TRACE work)	71
4.6.3	Code (R)	71
4.6.4	Pitfalls	71
4.6.5	Sensitivity Analysis for Causal Claims — The Key Assumption Reviewers Probe	71
4.6.6	Summary Table — Sensitivity Diagnostics for Causal Claims	72
4.6.7	Career Application	73
4.7	Model Evaluation & Validation	73
4.7.1	Cross-Validation	73
4.7.2	Summary Table — Metrics by Task	73
4.8	Graph Neural Networks	74
4.8.1	Conceptual Foundation	74
4.8.2	Applications	74
4.9	scRNA-seq Analysis	74
4.9.1	Workflow (Seurat/Scanpy)	74
4.9.2	Example Result — Cluster Marker Gene Table	75
4.9.3	Code (R — Seurat)	76
4.9.4	Code (Python — Scanpy)	76
4.9.5	Pitfalls	78
4.10	References	78
5	Multimodal Data Integration and Medical Imaging	82
5.1	Image Preprocessing Pipeline (Raw → Analysis-Ready)	82
5.1.1	Conceptual Foundation	82
5.1.2	Workflow (Raw → Final)	82
5.1.3	Code (Python — basic MRI preprocessing with SimpleITK/nibabel)	82
5.1.4	Code (Python — PyRadiomics (van Griethuysen et al., 2017) feature extraction)	83
5.1.5	Pitfalls	83
5.2	Multimodal Fusion: Imaging + Omics + EHR	83

5.2.1	Conceptual Foundation	83
5.2.2	Pipeline (Pseudocode)	83
5.2.3	Formula — Contrastive Alignment Loss (used in multimodal foundation models, e.g., CLIP-style)	84
5.2.4	State-of-the-Art / Emerging Methods	84
5.2.5	Example	84
5.2.6	Code (Python — simple late-fusion skeleton)	85
5.2.7	Pitfalls	85
5.2.8	Summary Table — Fusion Strategy Selection	85
5.3	References	86
6	Foundation Models for Computational Biology and Medicine	88
6.1	Shared Mathematical Foundations	88
6.2	GPT-Style Language Models (LLMs)	89
6.3	Vision Transformers (ViT)	91
6.4	CLIP (Contrastive Language-Image Pretraining)	92
6.5	Segment Anything Model (SAM)	93
6.6	Protein Language Models (ESM-2, ESMFold, UniRep)	95
6.7	Genomics Foundation Models (DNABERT, Geneformer)	99
6.8	Single-Cell Foundation Models (scGPT)	101
6.9	Multimodal Foundation Models (Perceiver IO, Flamingo, GPT-4o)	102
6.10	Graph Foundation Models (GNN-FM)	104
6.11	Chapter Summary: Foundation Model Selection Guide	106
6.12	Cross-Cutting Assumptions, Advantages, and Limitations	107
6.13	References	107
7	Comprehensive Model Reference Guide	110
7.1	Simple Linear/Logistic Regression (Baseline for Comparison)	110
7.2	Mixed-Effects Models for Longitudinal Data	111
7.3	Machine Learning / Deep Learning as Alternatives to Mixed-Effects Models	112
7.3.1	Comparison Table — Mixed-Effects vs. ML/DL for Longitudinal Data	115
7.4	Frequentist vs. Bayesian Inference	115
7.4.1	Example Result — Same Data, Two Framings	116
7.5	Is Bayesian Inference Valid Without Strong Prior Evidence?	117
7.5.1	Example Result — Posterior Sensitivity Analysis	119
7.6	Image Preprocessing Pipeline (Raw → QC → Normalization → Segmentation → Feature Extraction → Modeling)	120
7.6.1	Summary Table — Imaging Pipeline Stages	122
7.6.2	Example Result — Extracted Radiomic Features (Simulated)	122
7.7	Multimodal Modeling (Imaging + Omics + EHR)	123
7.7.1	Example Result — Unimodal vs. Fused Model Benchmark (Simulated)	125
7.8	Ensemble Models (XGBoost, LightGBM, Random Forest)	126
7.8.1	Example Result — Ensemble Comparison on Simulated Omics Data (n=500, p=200)	128

7.9	Deep Learning (CNN, RNN, LSTM, Transformers)	129
7.9.1	Comparison Table — DL Architecture Selection	131
7.9.2	Example Result — Architecture Choice on a Chest X-ray Classification Task (Simulated)	132
7.10	Explainable AI (SHAP, LIME)	132
7.11	scRNA-seq Workflows (Seurat, Scanpy)	134
7.12	Variant Calling Pipelines (GATK, bcftools, SAMtools)	136
7.13	A Quick Guide to This Book’s Mathematical Notation	138
7.14	Model Selection by Sample Size: Small-n vs. Large-n Regimes	139
7.14.1	The Core Idea: Effective Parameters vs. Available Data	139
7.14.2	Why Decision Trees, Random Forests, and XGBoost Work Well for Small Samples	140
7.14.3	Why Deep Learning Requires Large Samples	141
7.14.4	When Bayesian Models Are Preferred for Small Samples	141
7.14.5	When Frequentist Models Need Large Samples	142
7.14.6	When Mixed-Effects Models Outperform Simple Regression for Repeated Measures	142
7.14.7	Summary Table — Model Selection by Sample Size	142
7.15	Master Comparison Table — Choosing the Right Model	143
7.16	Best-Practice Guidelines for Model Selection	144
8	Programming Languages and Computational Tools	146
8.1	Summary Table	146
8.1.1	Example: PRS Calculation (PRSice-2)	146
8.1.2	Example: eQTL Mapping (MatrixEQTL, R)	147
8.1.3	Reproducibility Best Practices	147
8.2	References	147
9	Cross-Chapter Method Selection Guide	151
10	Glossary of Key Terms	152
Index		156
Index		156

Copyright

Computational Biology, Biostatistics, and AI/ML for Precision Medicine

Copyright © 2026 Eskezeia Yihunie Dessie. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

This book is provided for educational purposes. While every effort has been made to ensure the accuracy of the technical, statistical, and biomedical content presented, the author makes no warranties, express or implied, regarding the completeness or accuracy of the material. Code examples, statistical results, and simulated datasets throughout this book are provided for illustrative and pedagogical purposes and should be independently validated before use in research, clinical, or regulatory contexts. Mentions of specific software, tools, or commercial products do not constitute an endorsement.

Draft notice: This book is a work in progress and has not been finalized. Content, structure, examples, and references are subject to ongoing revision, correction, and expansion. Please do not cite, distribute, or rely on this version as a completed or authoritative reference.

First edition (draft), 2026.

Dedication

To everyone teaching themselves the next part of the field they need to know.

Preface

This book grew out of a simple observation: the people who work at the intersection of computational biology, biostatistics, and artificial intelligence rarely learn these subjects from a single source. A biostatistician trained in survival analysis and mixed-effects models often has to teach herself deep learning on the job; a bioinformatician comfortable with variant-calling pipelines may never have formally studied Bayesian inference; a machine learning engineer moving into healthcare often has not encountered the specific assumptions – proportional hazards, sequential ignorability, non-informative censoring – that make biomedical data analysis different from a general data science problem. This book is an attempt to bring these threads together in one place.

Each chapter follows a consistent pedagogical structure: a conceptual foundation stated in plain language, the mathematical formalism stated precisely, a real biological or clinical data example, a step-by-step workflow, the current state-of-the-art tools, working R and/or Python code, common pitfalls, and worked practice problems. Figures and simulated example results accompany the major methods throughout, each with an accompanying interpretation paragraph explaining what the result means and how to read it – the goal is not just to present a formula, but to show what it looks like in practice and how a careful analyst reasons about it.

The book is organized to move from foundations to application, with each chapter building on ideas introduced in the ones before it. It begins with core biostatistics (regression, hypothesis testing, mixed-effects models, Bayesian and frequentist inference), before turning to computational biology and bioinformatics, where those statistical foundations are applied to genomic, multi-omics, and variant-calling data. A complete chapter on survival analysis follows, extending the statistical foundations to time-to-event outcomes across classical, machine-learning, and deep-learning methods. The book then turns to artificial intelligence and machine learning more broadly, multimodal data integration across imaging, omics, and electronic health records, and the modern foundation models reshaping genomics, proteomics, and clinical prediction. Each technical chapter closes with a set of sample questions in the style of an applied technical discussion, translating the chapter's content into the kind of reasoning valued in applied practice. A comprehensive model reference guide compares every major method covered side by side, and the book closes with a practical reference chapter on the programming languages and computational tools used throughout.

This book does not assume its reader is choosing between statistics and machine learning, or between academic rigor and industry pragmatism. It assumes both are necessary, and that the strongest analysts are fluent in when to reach for a linear mixed-effects model with a defensible p-value and when to reach for a deep survival model whose main virtue is predictive accuracy at scale – and, just as importantly, know how to tell the difference.

1 Biostatistics and Statistical Modeling

1.1 Bayesian vs. Frequentist Methods

Bayes' theorem:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Frequentist inference relies on long-run sampling properties (p-values, confidence intervals); Bayesian inference yields posterior probability distributions and credible intervals, incorporating prior knowledge — increasingly used in fine-mapping (SuSiE) and adaptive clinical trial design.

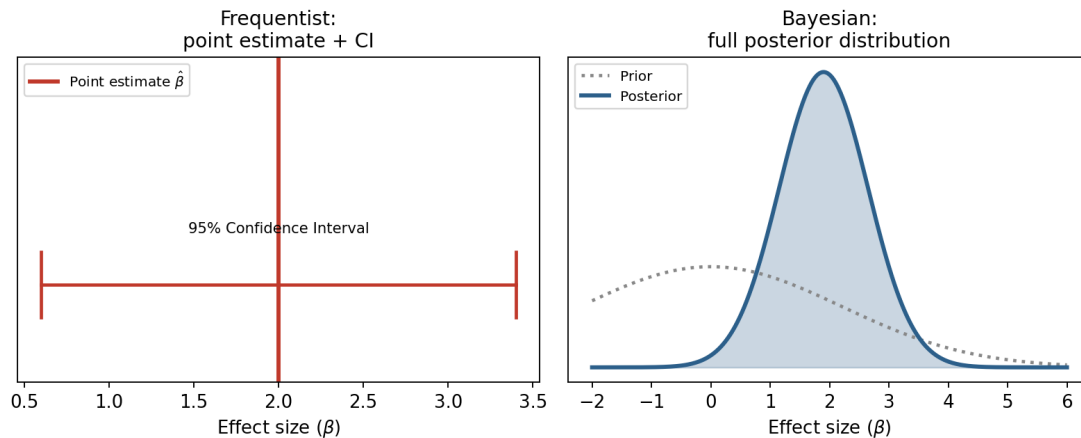


Figure 1.1: **Frequentist versus Bayesian representations of the same estimated effect.** *Left:* the frequentist output is a single point estimate ($\hat{\beta}$) with a 95% confidence interval, computed from the sampling distribution of the estimator. *Right:* the Bayesian output is a full posterior probability distribution over the effect size, obtained by updating a prior with the likelihood of the observed data.

Interpretation: Both panels describe a very similar underlying effect (centered near 2.0), and in a well-powered study with a weak prior, the two approaches will often lead to numerically similar conclusions. The difference is in what the output represents and how it can be used: the frequentist CI only has a valid interpretation under hypothetical repeated sampling (“95% of such intervals, constructed this way, would contain the true value”), while the Bayesian

posterior is a direct probability statement about the parameter itself given the observed data and prior (“there’s a 95% probability the true value lies in this range”). This distinction is why Bayesian output is often preferred for direct decision-making (e.g., “what’s the probability this drug’s effect exceeds a minimum clinically important difference?”), a question the frequentist CI cannot directly answer.

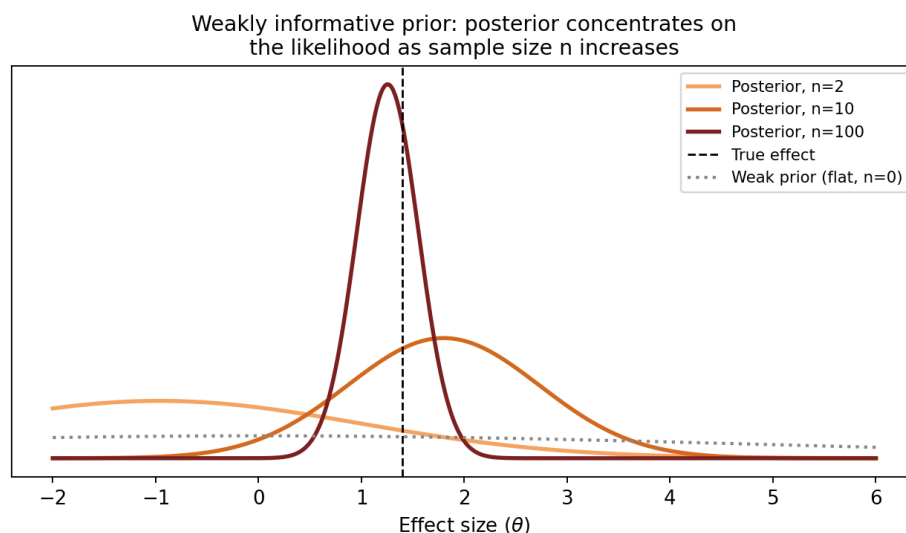


Figure 1.2: **A weakly informative prior is dominated by the likelihood as sample size grows.** The dotted grey curve shows a deliberately vague (weak) prior centered at zero. The three colored curves show the resulting posterior distribution for the same underlying effect at increasing sample sizes ($n=2, 10, 100$), holding the prior fixed.

Interpretation: At $n=2$ (light orange), the posterior is still fairly wide and pulled somewhat toward the prior’s center — there simply isn’t enough data yet to overwhelm the prior. By $n=10$ (darker orange) the posterior has narrowed and shifted closer to the true effect (dashed vertical line). By $n=100$ (dark red) the posterior has become tight and is centered almost exactly on the true value, essentially indistinguishable from what a frequentist maximum-likelihood estimate would report. This is the Bernstein-von Mises phenomenon described in Chapter 7: a weak prior does not prevent valid inference — it simply means the data, once there is enough of it, does virtually all of the work. This directly answers the recurring professional question of whether Bayesian inference is valid without strong prior evidence: yes, provided the prior is honestly weak (not silently informative) and the sample size is reported alongside the posterior so a reader can judge how much of the conclusion is prior-driven versus data-driven.

1.2 Survival Analysis

1.2.1 Kaplan-Meier Estimator

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right)$$

1.2.2 Cox Proportional Hazards Model

$$h(t|X) = h_0(t) \exp(\beta_1 X_1 + \dots + \beta_p X_p)$$

Proportional hazards assumption: hazard ratio $\exp(\beta)$ constant over time (test via Schoenfeld residuals).

1.2.3 Code (R — survival)

```
library(survival)
fit_km <- survfit(Surv(time, status) ~ group, data = df)
plot(fit_km, col = c("blue", "red"))
cox_fit <- coxph(Surv(time, status) ~ age + sex + biomarker, data = df)
cox.zph(cox_fit) # test PH assumption
```

1.3 Longitudinal / Mixed-Effects Models (Laird & Ware, 1982; Pinheiro & Bates, 2000)

$$y_{ij} = X_{ij}\beta + Z_{ij}b_i + \epsilon_{ij}, \quad b_i \sim N(0, \Sigma), \quad \epsilon_{ij} \sim N(0, \sigma^2)$$

Fixed effects (β) apply population-wide; random effects (b_i) capture subject-specific deviation.

Reading the figure: In the left panel, the single red regression line is forced through all 60 points regardless of which patient they came from — a patient's five repeated visits are treated exactly like five independent patients. In the right panel, the green line still summarizes the population-average trend, but the model additionally recognizes that each patient's five points are clustered together and allows each one to deviate from that average trend with their own intercept and slope (dashed lines).

Interpretation: The pooled OLS fit in the left panel and the fixed-effect line in the right panel can look deceptively similar on average — but the pooled model's standard errors are too small, because it effectively (and incorrectly) treats 60 correlated observations as 60 independent ones. This is the pseudoreplication problem introduced in the career-development chapter: with real

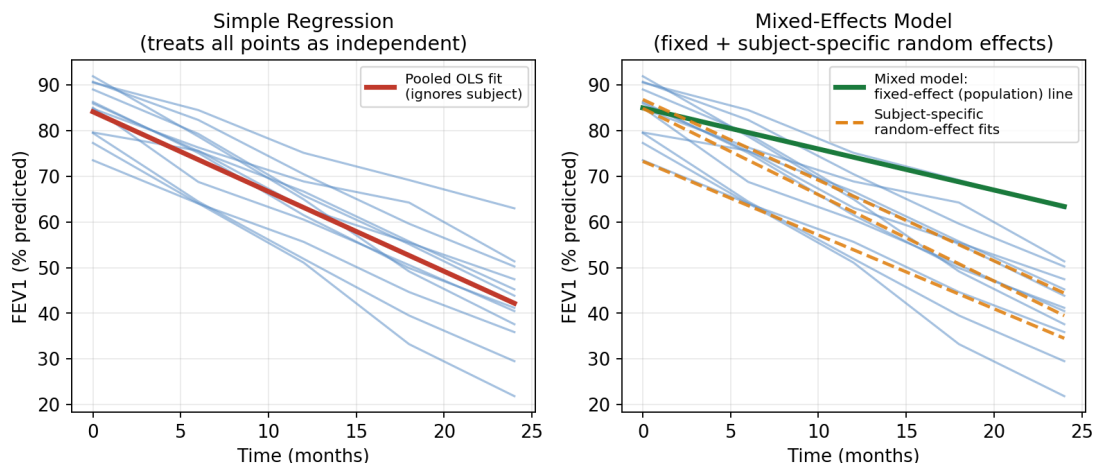


Figure 1.3: **Mixed-effects models versus simple regression for longitudinal data.** Simulated FEV1 trajectories for 12 patients measured at five clinic visits over 24 months. *Left:* a simple (pooled) regression line fit by ordinary least squares, treating all 60 observations as independent. *Right:* a mixed-effects model, showing the fixed-effect (population-average) line in green together with three example subject-specific fitted trajectories (dashed orange), each incorporating that patient’s own random intercept and slope.

within-subject correlation, the simple regression will report artificially narrow confidence intervals and inflated statistical significance for the treatment effect. The mixed model’s random-slope terms also reveal genuine between-patient heterogeneity in disease trajectory (some patients decline faster than others) — information a simple regression discards entirely.

1.3.1 Example Result — Mixed-Effects Model Output

Simulated output from `lmer(FEV1 ~ time * treatment + age + (1 + time | subject_id))` on a trial with 120 patients (60 per arm), matching the figure above:

Term	Estimate	SE	95% CI	p-value
Intercept	84.7	1.8	(81.2, 88.2)	<0.001
Time (months)	-0.91	0.06	(-1.03, -0.79)	<0.001
Treatment (biologic)	1.4	2.1	(-2.7, 5.5)	0.50
Time × Treatment	0.38	0.09	(0.20, 0.56)	<0.001
Random intercept SD (τ_0)	6.1	—	—	—
Random slope SD (τ_1)	0.47	—	—	—
Residual SD (σ)	2.3	—	—	—

How this is obtained: Fixed effects are estimated by restricted maximum

likelihood (REML); the random-effects variance components (τ_0 , τ_1) are estimated jointly, and the time-by-treatment interaction tests whether the *rate of decline* differs between arms — the clinically relevant question in a disease-modifying trial, rather than a treatment main effect alone.

Interpretation: The non-significant main effect of treatment ($p = 0.50$) but highly significant time×treatment interaction ($p < 0.001$) tells a coherent clinical story: the two arms start at a similar level, but the biologic-treated arm declines 0.38 percentage points/month *more slowly* than standard care. Reporting only a treatment main effect (as a naive cross-sectional analysis might) would have missed this entirely — the effect only becomes visible when the trajectory itself is modeled.

1.3.2 Code (R — lme4)

```
library(lme4)
model <- lmer(FEV1 ~ time * treatment + age + (1 + time | subject_id), data =
  ↪ longdata)
summary(model)
```

1.4 Logistic / Linear Regression

$$\text{logit}(P(Y = 1)) = \log \frac{P}{1 - P} = \beta_0 + \beta_1 X_1 + \dots$$

1.4.1 Code (R)

```
model <- glm(disease ~ age + sex + exposure, family = binomial, data = df)
exp(coef(model)) # odds ratios
```

1.5 High-Dimensional Modeling (LASSO, Tibshirani, 1996; Elastic Net, Zou & Hastie, 2005)

$$\hat{\beta}_{LASSO} = \arg \min_{\beta} \left\{ \sum_i (y_i - X_i \beta)^2 + \lambda \sum_j |\beta_j| \right\}$$

$$\hat{\beta}_{Ridge} = \arg \min_{\beta} \left\{ \sum_i (y_i - X_i \beta)^2 + \lambda \sum_j \beta_j^2 \right\}$$

Elastic Net combines both: $\lambda [\alpha \sum |\beta_j| + (1 - \alpha) \sum \beta_j^2]$

1.5.1 Code (R — glmnet)

```
library(glmnet)
cvfit <- cv.glmnet(as.matrix(X), y, alpha = 0.5, family = "binomial") # elastic
↪ net
coef(cvfit, s = "lambda.min")
```

1.6 Hypothesis Testing

Standard framework: null hypothesis H_0 , test statistic, p-value = $P(\text{data as extreme} | H_0)$.
Type I error (α), Type II error (β), Power = $1 - \beta$.

1.7 Diagnostic Test Evaluation

$$\text{PPV} = \frac{\text{Sens} \times \text{Prev}}{\text{Sens} \times \text{Prev} + (1 - \text{Spec})(1 - \text{Prev})}$$

Likelihood ratios: $LR^+ = \frac{\text{Sens}}{1 - \text{Spec}}$, $LR^- = \frac{1 - \text{Sens}}{\text{Spec}}$

1.8 Assay Performance Modeling

Precision (CV%), accuracy (bias vs. reference standard), limit of detection (LOD), limit of quantification (LOQ), and reproducibility (inter-/intra-assay CV) — critical for biomarker/companion diagnostic development (relevant to FMI/diagnostics contexts).

1.9 Experimental Design & Power Analysis

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\delta^2} \quad (\text{two-sample means})$$

1.9.1 Code (R — pwr)

```
library(pwr)
pwr.t.test(d = 0.5, sig.level = 0.05, power = 0.8, type = "two.sample")
```

1.10 Multiple Testing Correction (Benjamini & Hochberg, 1995)

- **Bonferroni:** $\alpha_{adj} = \alpha/m$
- **Benjamini-Hochberg (FDR):** rank p-values $p_{(1)} \leq \dots \leq p_{(m)}$; find largest k where $p_{(k)} \leq \frac{k}{m}\alpha$.

1.10.1 Choosing a Correction: It Depends on Whether the Analysis Is Hypothesis-Driven or Discovery-Driven

For a **small, pre-specified set of candidate genes/regions/CpGs** (a hypothesis-driven, confirmatory analysis), a more conservative but still interpretable correction like **Bonferroni** is appropriate, since m is small and the family-wise error rate remains a meaningful, achievable standard. For **genome-wide or epigenome-wide discovery**, where the number of tests can be in the hundreds of thousands, **FDR control (Benjamini-Hochberg)** is preferred, since Bonferroni would be prohibitively conservative at that scale and would bury real signal. Critically, the threshold and correction method should be decided **before** looking at results, not adjusted after the fact to reach significance — pre-registering the analysis plan before viewing outcome data is the practical safeguard against this.

1.10.2 Career Application

Q: How do you handle multiple testing correction across different analysis contexts? *Model answer:* It depends on whether the analysis is hypothesis-driven or discovery-driven. For a small, pre-specified set of candidate genes or regions, a conservative but interpretable correction like Bonferroni is appropriate. For genome-wide or epigenome-wide discovery, where the number of tests can reach the hundreds of thousands, false discovery rate control (Benjamini-Hochberg) is the right standard, since Bonferroni would be prohibitively conservative and would bury real signal at that scale. The threshold itself should be decided before looking at results, not adjusted afterward to reach significance.

1.10.3 Summary Table — Module 3

Method	Use Case	Key Assumption
Cox PH	Time-to-event	Proportional hazards
Mixed-effects	Repeated measures	Correct random-effects structure

Method	Use Case	Key Assumption
LASSO/EN FDR (BH)	High-dim, $p \gg n$ Many simultaneous tests	Sparsity Tests independent or positively dependent
Bonferroni	Small, pre-specified test set	Controls family-wise error rate; overly conservative at large scale

1.11 References

Laird, N. M., & Ware, J. H. (1982). Random-effects models for longitudinal data. *Biometrics*, 38(4), 963-974.

Pinheiro, J. C., & Bates, D. M. (2000). *Mixed-effects models in S and S-PLUS*. Springer.

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1), 1-48.

Liang, K. Y., & Zeger, S. L. (1986). Longitudinal data analysis using generalized linear models. *Biometrika*, 73(1), 13-22.

Efron, B. (1986). Why isn't everyone a Bayesian? *The American Statistician*, 40(1), 1-11.

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian data analysis* (3rd ed.). CRC Press.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288.

Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2), 301-320.

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289-300.

VanderWeele, T. J. (2015). *Explanation in causal inference: Methods for mediation and interaction*. Oxford University Press. ## Sample Questions

Q: How would you approach analyzing longitudinal biomarker data (e.g., repeated lung function measurements over 2 years)? *Model answer:* I'd first plot individual trajectories (spaghetti plots) to check for nonlinearity and outliers, then fit a linear mixed-effects model with a random intercept

(and possibly random slope) per subject to account for within-subject correlation, include time as both fixed and random effect, test for a treatment-by-time interaction as the primary effect of interest, and check the covariance structure (unstructured vs. compound symmetry vs. autoregressive AR(1)) using AIC/BIC. I'd also assess missing-data mechanism (MCAR/MAR/MNAR) since mixed models assume MAR by default under maximum likelihood estimation — if MNAR is suspected (e.g., sicker patients drop out), I'd consider joint models or pattern-mixture models.

Q: Why are mixed-effects models preferred over simple linear regression for repeated-measures/longitudinal data? *Conceptual explanation:* Simple linear regression assumes all observations are **independent**. When you have repeated measurements on the same subject (e.g., 5 clinic visits per patient), those observations are correlated — a patient with high FEV1 at visit 1 will tend to have high FEV1 at visit 2 as well. Ignoring this correlation: 1. **Underestimates standard errors** (pretends you have more independent information than you do — pseudoreplication), inflating false-positive rates. 2. **Cannot separate within-subject from between-subject effects** — e.g., does a drug work because it changes trajectories within a person, or because sicker people were assigned to different arms? 3. Mixed-effects models solve this by explicitly modeling a subject-specific random intercept/slope $b_i \sim N(0, \Sigma)$, which absorbs the correlation structure and correctly partitions variance into within- vs. between-subject components (as in Module 3.3's formula).

Q: When would you NOT need a mixed-effects model, even with repeated measures? *Model answer:* If you only care about a single summary per subject (e.g., area-under-the-curve or a single endpoint per subject, "one row per patient"), a simple regression on the summarized outcome can be valid and simpler (a two-stage approach). Mixed models are essential specifically when you want to model the trajectory itself, handle unbalanced/missing visit schedules, or estimate subject-specific random slopes.

When Can ML/Deep Learning Replace Mixed-Effects Models?

Q: Can machine learning or deep learning replace mixed-effects models for longitudinal data? When and why (or why not)? *Conceptual explanation:* ML/DL methods (e.g., **recurrent neural networks, LSTMs, Gaussian Process regression, transformer-based time-series models**) can model longitudinal data and, in some cases, capture more complex non-linear trajectories than a parametric mixed model. However: - **When ML/DL is a good fit:** large sample size (hundreds to thousands of subjects with many repeated measures), primary goal is *prediction accuracy* rather than *interpretable effect estimates* (e.g., predicting next-visit risk score rather than estimating a treatment coefficient), and/or the trajectory shape is highly

nonlinear/non-parametric (deep learning, GPs, or **mixed-effects random forests / MERF**, and **GLMM-boosting** hybrids that combine tree ensembles with a random-effects term, can help here). - **When mixed-effects models remain preferred:** small-to-moderate sample sizes (typical clinical trials, $n < 1000$), when statistical inference (p-values, confidence intervals, hypothesis testing about a specific fixed effect like treatment) is the primary goal — most DL methods do not natively provide calibrated inferential uncertainty — and when regulatory submission requires a pre-specified, interpretable model (FDA/EMA strongly favor mixed models/GEE for repeated-measures endpoints). - **Hybrid approaches:** Increasingly, teams use “**deep mixed models**” (e.g., neural network with a random-effects layer, or LSTM followed by a mixed-effects layer on residuals) to combine flexible representation learning with proper handling of within-subject correlation — this is an active area and a good answer to signal awareness of current SOTA.

Bayesian vs. Frequentist Inference

Q: Compare Bayesian and frequentist approaches to statistical inference. What are the strengths, weaknesses, and assumptions of each?

Aspect	Frequentist	Bayesian
Core idea	Parameters are fixed, unknown constants; probability = long-run frequency	Parameters are random variables with a probability distribution reflecting uncertainty
Output	Point estimate + p-value + confidence interval	Full posterior distribution + credible interval
Prior information	Not used	Explicitly incorporated via prior distribution
Interpretation	“95% of such intervals would contain the true value in repeated sampling”	“There is a 95% probability the true value lies in this interval, given the data and prior”
Strength	No need to specify a prior; well-established for regulatory/clinical trial contexts (Type I error control)	Naturally handles small samples, sequential/adaptive designs, and incorporates prior knowledge (e.g., previous trial data)

Aspect	Frequentist	Bayesian
Weakness	p-values/CIs are often misinterpreted; doesn't formally use prior evidence	Sensitive to prior choice; computationally intensive (MCMC/variational inference); regulatory acceptance more variable

Bayesian posterior formula (recap from Module 3.1):

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \propto \text{Likelihood} \times \text{Prior}$$

Q: Is Bayesian inference valid without strong prior evidence? *Model answer:* Yes — this is a common misconception. When little prior information exists, you use a **weakly informative** or **non-informative (flat/vague) prior** (e.g., $N(0, 100^2)$) for a regression coefficient, or a uniform prior over a plausible range). With enough data, the **likelihood dominates the posterior** regardless of a diffuse prior, and Bayesian point estimates typically converge to results very similar to frequentist MLE. Bayesian methods remain valid and useful in this setting because they still provide (a) full uncertainty quantification via the posterior, (b) natural handling of small samples or sparse events (e.g., rare adverse events, rare-variant genetic association), and (c) a coherent framework for sequential updating as new data arrives (e.g., adaptive clinical trial designs, Bayesian optimal interim monitoring). The prior is a modeling choice, not a requirement for “strong prior evidence” — sensitivity analysis across a few reasonable priors is standard practice to demonstrate robustness.

Rapid-Fire Conceptual Review

- **Q: Why not just use ANOVA instead of a mixed model for repeated measures?** A: Repeated-measures ANOVA requires complete, balanced data (same number/timing of visits per subject) and a restrictive compound-symmetry covariance assumption; mixed models handle unbalanced/missing visits and flexible covariance structures.
- **Q: What's the difference between a fixed effect and a random effect?** A: Fixed effects are the specific levels/values of interest for inference (e.g., treatment arm); random effects are a sample from a broader population of subjects/clusters, modeled via their variance rather than individual estimates, allowing generalization beyond the observed subjects.
- **Q: Why does LASSO produce sparse solutions but Ridge doesn't?**

A: The L1 penalty's geometry (diamond-shaped constraint region) has corners on the axes, so the optimum often lands exactly at zero for some coefficients; the L2 penalty (circular constraint) shrinks coefficients smoothly toward zero without ever hitting it exactly.

- **Q: What's the difference between GEE (Liang & Zeger, 1986) and mixed-effects models?** A: GEE (Generalized Estimating Equations) models the population-averaged (marginal) effect and treats within-subject correlation as a nuisance (robust "sandwich" SEs); mixed-effects models estimate subject-specific effects and explicitly model the random-effects distribution. In practice: lean toward **mixed-effects models** when the question calls for subject-specific inference and you're willing to model the specific between-subject variation structure (e.g., random intercepts and slopes per patient); lean toward **GEE** when the question is about the population-average effect and you're less confident about correctly specifying the random-effects structure, since GEE remains valid ("robust") to misspecification of the working correlation structure as long as the mean model itself is correctly specified — mixed models, by contrast, can give biased inference if the random-effects structure is wrong.

2 Computational Biology and Bioinformatics

2.1 Statistical Genetics & Genomics

2.1.1 Conceptual Foundation

Statistical genetics quantifies the relationship between genetic variation (SNPs, indels, CNVs) and phenotypes. Core concepts: allele frequency, Hardy-Weinberg equilibrium (HWE), linkage disequilibrium (LD), heritability, and genetic architecture (polygenic vs. monogenic).

Hardy-Weinberg Equilibrium:

$$p^2 + 2pq + q^2 = 1$$

where p and q are allele frequencies. Deviation (via χ^2 test) signals genotyping error, selection, or population stratification.

Linkage Disequilibrium (D' and r^2):

$$D = p_{AB} - p_A p_B, \quad r^2 = \frac{D^2}{p_A(1-p_A)p_B(1-p_B)}$$

Narrow-sense heritability:

$$h^2 = \frac{V_A}{V_P} = \frac{V_A}{V_A + V_D + V_E}$$

2.1.2 GWAS Statistical Model

$$y_i = \beta_0 + \beta_1 g_i + \sum_k \gamma_k PC_{ik} + \epsilon_i$$

where $g_i \in \{0, 1, 2\}$ is additive genotype dosage, PC_{ik} are ancestry principal components (population stratification control), and significance threshold is genome-wide $p < 5 \times 10^{-8}$.

2.1.3 Real Data Example

- **UK Biobank** (~500K individuals, genotyped + imaged + EHR-linked) — GWAS of asthma, lung function (FEV1/FVC).
- **CAAPA** (Consortium on Asthma among African-ancestry Populations) — admixture-aware GWAS.
- **GEO/dbGaP** — raw genotype/phenotype deposits for replication.

2.1.4 Workflow (GWAS Pipeline)

Raw genotypes (PLINK .bed/.bim/.fam)

- > QC: call rate >98%, HWE $p > 1e-6$, MAF >1%, sex check
- > Imputation (Michigan/TOPMed Imputation Server, reference panel)
- > Post-imputation QC: INFO score >0.3, MAF filter
- > Population stratification: PCA (PLINK --pca or EIGENSOFT)
- > Association testing: PLINK2 --glm / REGENIE / SAIGE (mixed models for related samples)
- > Genomic control (λ_{GC}), Manhattan & QQ plots
- > Fine-mapping: SuSiE, FINEMAP
- > Functional annotation: ANNOVAR, VEP

2.1.5 Code

R (association test using PLINK output):

```
library(data.table)
gwas <- fread("assoc_results.assoc.linear")
gwas$logP <- -log10(gwas$P)
lambda_gc <- median(qchisq(1 - gwas$P, df = 1)) / qchisq(0.5, df = 1)

library(qqman)
manhattan(gwas, chr = "CHR", bp = "BP", p = "P", snp = "SNP",
           suggestiveline = -log10(1e-5), genomewideline = -log10(5e-8))
qq(gwas$P)
```

Bash (PLINK2 GWAS):

```
plink2 --bfile study_qc --pheno pheno.txt --covar covar.txt \
      --glm hide-covar --out gwas_results
```

2.1.6 Pitfalls & Best Practices

- Population stratification inflates false positives — always adjust with PCs or use mixed models (SAIGE, REGENIE, BOLT-LMM) for related/admixed samples.

- Winner’s curse inflates effect sizes at discovery; replicate in an independent cohort.
- MAF filtering: rare variants need burden/SKAT tests, not single-variant GLM.

2.1.7 Interpretation Guidelines

Report effect size (beta/OR), 95% CI, and replication p-value — not just discovery p-value. Always state the reference/effect allele.

2.2 Genomic Data Processing

2.2.1 Conceptual Foundation

Raw sequencing reads (FASTQ) must be quality-controlled, aligned to a reference genome, and converted into analysis-ready formats (BAM, VCF).

2.2.2 Workflow

FASTQ (raw reads)

- > FastQC / MultiQC (quality assessment)
- > Trimming (Trimmomatic, fastp) - adapter & low-quality base removal
- > Alignment (BWA-MEM for DNA; STAR (Dobin et al., 2013)/HISAT2 (Kim et al., 2019) for RNA)
- > Sorting/indexing (SAMtools sort/index)
- > Duplicate marking (Picard MarkDuplicates)
- > Base quality score recalibration (GATK BQSR)
- > Variant calling (GATK HaplotypeCaller / bcftools mpileup)
- > Joint genotyping (GATK GenomicsDBImport + GenotypeGVCFs)
- > Variant filtering (GATK VQSR or hard filters)
- > Annotation (VEP, ANNOVAR, SnpEff)

Interpretation: The pipeline is a chain of error-correction steps, each targeting a different stage of the sequencing process where noise enters. Skipping duplicate marking, for example, means PCR-duplicated reads are counted as independent evidence for a variant, artificially inflating confidence in false-positive calls — precisely the kind of error the final filtered VCF is meant to have already screened out. This is why the Pitfalls note below treats reference-build mismatches and unmarked duplicates as the leading causes of downstream errors: both occur early in the pipeline (left side of the figure) and silently corrupt every step that follows.

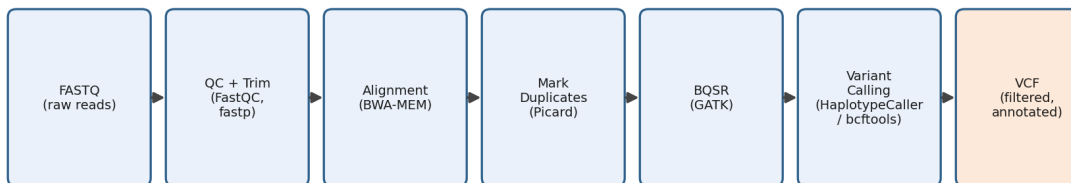


Figure 2.1: **The variant calling pipeline, raw reads to filtered VCF.** Each stage removes a specific source of error before variant calls are trusted: adapter contamination and low-quality bases (QC/Trim), misalignment noise (Alignment), PCR duplication bias (Mark Duplicates), and systematic base-quality miscalibration (BQSR), culminating in a filtered, annotated call set (highlighted).

2.2.3 Example Result — VCF Summary Statistics (bcftools stats, Simulated Trio)

Metric	Child	Mother	Father
Total variants (PASS)	4,812,340	4,795,110	4,808,220
SNVs	4,312,005	4,298,440	4,304,900
Indels	500,335	496,670	503,320
Ts/Tv ratio	2.04	2.03	2.05
Mendelian error rate	0.08%	—	—

How this is obtained: `bcftools stats` summarizes the joint-genotyped, filtered VCF for each trio member; the Mendelian error rate compares the child’s genotype at each site against what is possible given the two parents’ genotypes.

Interpretation: A transition/transversion (Ts/Tv) ratio near 2.0-2.1 for whole-genome data is a standard quality signal — a value far from this range (e.g., close to 0.5, the value expected under random sequencing error) would suggest a high false-positive rate somewhere upstream in the pipeline. A Mendelian error rate this low (0.08%) indicates the trio’s genotype calls are highly internally consistent, which is reassuring evidence against sample mislabeling or a serious upstream QC failure; a rate an order of magnitude higher would prompt a check for sample swaps before any family-based analysis (e.g., *de novo* variant calling) proceeds.

2.2.4 Code

Alignment

```
bwa mem -t 8 ref.fa sample_R1.fastq.gz sample_R2.fastq.gz | \
  samtools sort -@8 -o sample.sorted.bam
samtools index sample.sorted.bam
```

Variant calling

```
gatk HaplotypeCaller -R ref.fa -I sample.sorted.bam -O sample.g.vcf.gz -ERC GVCF
gatk GenotypeGVCFs -R ref.fa -V sample.g.vcf.gz -O sample.vcf.gz
```

QC with bcftools

```
bcftools stats sample.vcf.gz > sample.stats.txt
bcftools filter -e 'QUAL<30 || DP<10' sample.vcf.gz -O z -o sample.filtered.vcf.gz
```

2.2.5 Pitfalls

- Reference genome build mismatch (GRCh37 vs. GRCh38) is the #1 source of downstream errors — always confirm build before merging cohorts.
- Duplicate reads inflate coverage/variant confidence if not marked before calling.
- Batch effects between sequencing runs — track lane/flowcell as covariates.

2.3 Multi-Omics Integration

2.3.1 Conceptual Foundation

Multi-omics integrates DNA methylation (epigenome), RNA-seq (transcriptome), metabolomics, and microbiome data to build a systems-level model of disease. Two broad strategies: **early integration** (concatenate features before modeling) vs. **late integration** (model each omic separately, combine outputs) vs. **intermediate/network-based integration** (e.g., correlation networks, factor analysis).

2.3.2 Statistical Framework — eQTM (expression Quantitative Trait Methylation)

$$\text{Expression}_{ij} = \beta_0 + \beta_1 \cdot M_{ij} + \sum \gamma_k C_{ik} + \epsilon_{ij}$$

where M_{ij} is methylation beta-value/M-value at CpG j for sample i , and C_{ik} are covariates (age, sex, cell-type proportions, batch, PCs).

2.3.3 Multi-Block Integration Methods

- **MOFA / MOFA+** (Argelaguet et al., 2020) (Multi-Omics Factor Analysis) — Bayesian latent factor model.
- **DIABLO (mixOmics)** — supervised, discriminant multi-block PLS.
- **WGCNA** (Langfelder & Horvath, 2008) — co-expression/co-methylation network modules via topological overlap.
- **xMWAS** — network-based multi-omics correlation mapping.
- **SNF (Similarity Network Fusion)** — patient similarity network integration.

2.3.4 Real Data Example

- **TCGA** — matched RNA-seq, methylation (450K/EPIC arrays), miRNA, CNV, clinical for >30 cancer types.
- **CAAPA/GALA II** — asthma multi-omics (genotype + methylation + RNA-seq).
- **Human Microbiome Project (HMP)** — 16S/shotgun metagenomics.

2.3.5 Workflow

Layer 1: DNA methylation (850K array, minfi/ChAMP -> normalized beta values)
Layer 2: RNA-seq (STAR -> featureCounts -> DESeq2/limma-voom normalized counts)
Layer 3: Metabolomics (LC-MS/GC-MS -> XCMS -> normalized peak intensities)
Layer 4: Microbiome (16S/shotgun -> QIIME2/DADA2 -> relative abundance)
-> Batch correction (ComBat) per layer
-> Feature selection per layer (variance filter, univariate association)
-> Integration (MOFA+, DIABLO, WGCNA, or xMWAS)
-> Identify shared latent factors / modules
-> Link to clinical outcome (regression on factor scores)
-> Validate with independent cohort / permutation testing

2.3.6 Code (R — MOFA+)

```
library(MOFA2)
mofa_data <- list(methylation = meth_mat, rnaseq = rna_mat, metabolomics =
  ↪ metab_mat)
MOFAobject <- create_mofa(mofa_data)
model_opts <- get_default_model_options(MOFAobject)
model_opts$num_factors <- 10
MOFAobject <- prepare_mofa(MOFAobject, model_options = model_opts)
MOFAobject <- run_mofa(MOFAobject)
plot_variance_explained(MOFAobject)
```

2.3.7 Pitfalls

- Different omics layers have different noise structures and scales — normalize/standardize each block independently before integration.
- Sample size mismatch across layers requires careful missing-data handling (don't naively drop incomplete cases if it biases toward a subgroup).
- Correlation $\neq$ causation: a shared latent factor across omics layers is a hypothesis-generating signal, not a mechanistic proof — pair with causal frameworks (Module 1.8, 2.6).

2.3.8 Practical Integration Strategy: Discovery vs. Confirmation

Before choosing an integration method, decide explicitly whether the analysis is **discovery** (broad, hypothesis-generating signature search) or **confirmation** (testing a specific, pre-specified hypothesis) — this single decision determines the appropriate multiple-testing correction and validation strategy downstream, and should be made before looking at results, not adjusted afterward to reach significance. - **Hypothesis-driven / confirmatory**: test direct pairwise associations (e.g., eQTM analysis linking a specific CpG to a specific gene) with a nominal or Bonferroni-style threshold appropriate to the small, pre-specified search space. - **Discovery / signature search**: use network-based integration (WGCNA, xMWAS) or factor models (MOFA+, DIABLO) to find modules/factors spanning omics layers, with FDR control appropriate to the much larger effective search space, followed by mandatory independent-cohort replication before any biological claim.

2.3.9 Common Technical Pitfalls in Cross-Platform Integration

The most common pitfall is **naively concatenating features from different platforms without accounting for scale** — a metabolomics intensity value and a methylation beta-value live on completely different numeric ranges, so without proper normalization, a model implicitly weights one layer more heavily just because of its numeric scale, not its biological signal. A second common pitfall is **ignoring platform-specific batch effects before integration**, which can create spurious cross-omic correlations that look like biology but are actually shared technical artifacts (e.g., all three layers processed on the same day for a subset of samples). Both are addressed by normalizing/batch-correcting each layer independently *before* integration, then using integration methods that explicitly model layer-specific variance (MOFA+'s per-view noise model, for instance) rather than assuming a shared scale across layers.

2.3.10 Missing Data Across Omics Platforms — A Decision Framework

Handling depends on the **mechanism and extent** of missingness, not a single blanket rule: - **Missing completely at random (MCAR), limited extent**: use multiple imputation appropriate to the data type (e.g., k-NN or PCA-based imputation for continuous omics features) rather than dropping samples — dropping incomplete cases across three or four omics layers compounds quickly and can leave very little usable data. - **Structural missingness** (e.g., a sample simply wasn't run on a given platform, not a random gap): keep that layer's analysis restricted to the samples that actually have it, and report the different effective sample size per layer explicitly, rather than forcing a single, much smaller complete-case dataset across all layers — silently restricting to complete cases can bias the analysis toward whichever subgroup happened to have full multi-omics coverage.

2.3.11 Career Application

Q: What are common pitfalls when combining multi-omic layers with different scales and distributions, and how do you handle missing data across platforms? *Model answer:* The most common pitfall is naive concatenation without scale normalization, which implicitly lets whichever layer has the largest numeric range dominate the model regardless of biological signal — the fix is normalizing and batch-correcting each layer independently before any integration step. For missing data, the right approach depends on mechanism: MCAR gaps of limited extent get multiple imputation rather than case deletion, while structural missingness (a platform simply wasn't run for some samples) is better handled by restricting that layer's analysis to samples that have it and reporting the different effective n per layer, rather than collapsing to a much smaller complete-case set that may not represent the full cohort.

2.4 RNA-seq Differential Expression & Batch Effects

2.4.1 Conceptual Foundation

Differential expression (DE) analysis identifies genes whose expression differs between conditions (e.g., disease vs. healthy) from RNA-seq count data, using models built for the discrete, over-dispersed nature of sequencing counts rather than ordinary linear regression on raw counts.

2.4.2 Pipeline (Raw Reads → DE Results)

FASTQ → FastQC (QC) → adapter/quality trimming

- > Alignment: STAR (splice-aware) or pseudo-alignment: Salmon/kallisto
- > Count matrix (genes x samples): featureCounts, tximport (for Salmon)
- > Normalization + DE testing: DESeq2 (Love, Huber, & Anders, 2014) or edgeR (Robinson, McCarthy, & Smyth, 2010)

of-ratios normalization,

- negative-binomial GLM, dispersion shrinkage)
- > Multiple testing correction: Benjamini-Hochberg FDR
- > Downstream: pathway enrichment (Module 1.7), co-expression network (WGCNA)

2.4.3 DESeq2 vs. edgeR — Practical Differences

Both use similar median-of-ratios-style normalization to account for library size and composition, but they differ in how they model dispersion: **DESeq2** uses a shrinkage estimator that pulls per-gene dispersion estimates toward a fitted mean-dispersion trend, stabilizing estimates for genes with low counts, while **edgeR** offers multiple dispersion-estimation modes (common, trended, tagwise). In practice the two tools usually agree closely on well-powered datasets; running both as a robustness check is reasonable when a specific result is borderline significant, rather than relying on a single tool's output for an important finding.

2.4.4 Batch Effects in RNA-seq

First, check whether batch is confounded with the biological variable of interest — if cases and controls were sequenced in entirely separate batches, no statistical correction fully repairs that; it must be flagged as a design limitation, not corrected away. **Where batch and biology are not fully confounded**, use **ComBat-seq** (designed specifically for count data, unlike the original ComBat which assumes Gaussian data) or include batch as a covariate directly in the DESeq2/edgeR model formula (ComBat-seq: Zhang, Parmigiani, & Johnson, 2020) ($\sim$ batch + condition) rather than pre-adjusting the raw counts, which preserves the count-based negative-binomial statistical framework these tools rely on.

2.4.5 Code (R — DESeq2 with batch covariate)

```
library(DESeq2)
dds <- DESeqDataSetFromMatrix(countData = counts, colData = coldata,
                              design = ~ batch + condition)
dds <- DESeq(dds)
```

```
res <- results(dds, contrast = c("condition", "case", "control"))
res <- res[order(res$padj), ]
```

2.4.6 Pitfalls

- Applying DESeq2/edgeR statistical machinery to already-batch-adjusted counts (e.g., ComBat-corrected) breaks the negative-binomial count assumption these tools rely on — prefer including batch as a covariate in the design formula instead.
- Confounded batch and biology cannot be fixed by any downstream statistical correction — this must be addressed at study-design stage.

2.5 DNA Methylation QC & Cell-Type Heterogeneity

2.5.1 850K/EPIC Array QC Pipeline

Raw IDAT files

- > Detection p-value filtering (remove low-confidence probes)
- > Remove known cross-reactive probes and probes overlapping common SNPs
- > Batch/plate effect check and correction (ComBat)
- > Predicted-sex vs. reported-sex check (sample-swap QC)
- > Cell-type composition estimation (if bulk tissue)
- > Normalization (e.g., functional normalization, minfi/ChAMP)
- > Beta-value / M-value matrix ready for eQTM/EWAS testing

2.5.2 Cell-Type Heterogeneity — Why It Matters

Bulk tissue methylation reflects a mixture of cell types, and cell-type proportions themselves are often correlated with the phenotype of interest (e.g., immune cell composition differing between asthma cases and controls). Left unaddressed, this creates confounding: an association that appears to be a genuine methylation effect may actually be driven by shifting cell-type proportions between groups, not a true per-cell-type regulatory change. The standard fix is **reference-based deconvolution** (e.g., Houseman's method, `minfi::estimateCellCounts()`, or EpiDISH) to estimate cell-type proportions from the methylation data itself using a reference panel, then including those estimated proportions as covariates in the association model — rather than assuming bulk methylation directly reflects a single, homogeneous cell population.

2.5.3 cis vs. trans eQTM/mQTL — Why Significance Thresholds Differ

Cis analysis tests methylation-expression (or SNP-methylation) pairs that are physically local, typically within the same gene region or a defined window (e.g., ± 1 Mb); the search space and multiple-testing burden are small, so a nominal threshold (e.g., $p < 0.05$, sometimes with a modest FDR correction within the cis window) is often used. **Trans** analysis tests genome-wide pairs — an enormous search space (potentially billions of CpG-gene combinations) requiring stringent FDR correction (Module 3.10). The threshold difference reflects **search-space size, a statistical consequence of multiple testing** — not a claim that trans effects are biologically less important than cis effects.

2.5.4 Code (R — cell-type deconvolution)

```
library(minfi); library(FlowSorted.Blood.EPIC)
cellCounts <- estimateCellCounts2(rgSet, compositeCellType = "Blood",
                                processMethod = "preprocessNoob",
                                referencePlatform =
                                ↪ "IlluminaHumanMethylationEPIC")
# Include cellCounts columns as covariates in the eQTM/EWAS model
```

2.5.5 Pitfalls

- Skipping the predicted-sex-vs-reported-sex QC check can let mislabeled/swapped samples silently corrupt downstream association results — treat this check as non-negotiable regardless of deadline pressure.
- Failing to adjust for cell-type composition in bulk-tissue EWAS is one of the most common sources of spurious methylation associations in the literature.

2.6 Biomarker Discovery

2.6.1 Conceptual Foundation

A biomarker is a measurable indicator of a biological state (diagnostic, prognostic, predictive, or pharmacodynamic). Discovery involves screening (univariate), refinement (multivariate/penalized regression), and validation (independent cohort, ROC/AUC).

2.6.2 Key Metrics

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP}$$

$$\text{AUC} = P(\text{score}_{\text{case}} > \text{score}_{\text{control}})$$

2.6.3 Workflow

```
Discovery cohort -> univariate filter (t-test/limma, FDR<0.05)
-> multivariate model (LASSO/Elastic Net/Random Forest)
-> internal validation (cross-validation, bootstrap)
-> external validation cohort (independent AUC, calibration plot)
-> clinical utility assessment (decision curve analysis)
```

2.6.4 Code (R — ROC/AUC)

```
library(pROC)
roc_obj <- roc(response = clinical$disease, predictor = clinical$biomarker_score)
auc(roc_obj); ci.auc(roc_obj)
plot(roc_obj, print.auc = TRUE)
```

2.6.5 Pitfalls

- Overfitting in small discovery cohorts ($p \gg n$) — always validate externally.
- Optimism bias: AUC reported on training data is inflated; use nested CV.

2.6.6 From Statistically Significant to Clinically Actionable

Statistical significance only means an association is unlikely to be due to chance at a given sample size — it says nothing about whether the effect size is large enough to change a clinical decision, whether the biomarker is measurable practically and quickly in a real clinical workflow, or whether there's a defined action tied to the result. An **actionable** biomarker requires all three: (1) a clinically meaningful effect size (not just a significant p-value), (2) practical measurability within clinical turnaround times, and (3) a specific downstream decision it informs (e.g., initiating or de-escalating a specific treatment) — rather than only reporting a statistical association in isolation.

2.6.7 The Validation Ladder (Discovery → Clinical Deployment)

```
Internal cross-validation (same cohort)
-> External validation (independent cohort, ideally different site/demographics)
-> Discrimination check (AUC) AND calibration check (a model can rank well
    but still be poorly calibrated for absolute risk – check separately)
-> Clinical utility (decision curve analysis: does using the biomarker improve
    net clinical benefit over treat-all/treat-none at realistic decision thresholds?)
-> Prospective clinical validation study (not just retrospective cohort validation)
```

-> Clinical deployment / decision-support integration

A model validated only via internal cross-validation (even repeated/nested CV) has *not* climbed this ladder — external, independent-cohort validation is the step that most credibly rules out overfitting, and is the first thing a skeptical technical panel will ask about for any headline discrimination metric (e.g., a high AUC).

2.6.8 Career Application

Q: Your model reports a high AUC (e.g., 0.91) in a modest-sized cohort. How do you defend against overfitting, and what would make you skeptical of your own number? *Model answer:* Any AUC above ~0.9 in a small genomic cohort deserves default skepticism unless backed by external validation — what earns trust is when the number comes from a model *locked before validation* (no refitting) and evaluated on a genuinely independent cohort, ideally with different demographics or collection sites. Calibration should be checked alongside discrimination, since a model can rank patients correctly (good AUC) while still being poorly calibrated for absolute risk. The absence of that kind of out-of-sample check — cross-validation alone — would be the specific reason to distrust a similarly high number in future work.

2.7 Infectious Disease Genomics

2.7.1 Conceptual Foundation

Applies phylogenetics and variant calling to pathogen genomes for outbreak tracking, resistance prediction, and transmission inference.

2.7.2 Key Methods

- **Phylogenetic tree construction** (maximum likelihood: IQ-TREE, RAxML; Bayesian: BEAST).
- **Molecular clock / effective reproduction number (R_t) estimation.**
- **Genomic surveillance pipelines:** Nextstrain, Pangolin (SARS-CoV-2 lineages).

2.7.3 Workflow

Pathogen sequencing reads -> reference alignment (minimap2) -> consensus calling
 -> variant calling (relative to reference strain)
 -> multiple sequence alignment (MAFFT)
 -> phylogenetic tree (IQ-TREE/RAxML)

- > lineage assignment (Pangolin/Nextclade)
- > transmission cluster inference

2.7.4 Pitfalls

- Sequencing depth/coverage gaps create false lineage calls — require minimum depth thresholds (e.g., 10-20x).
- Recombination violates simple bifurcating tree assumptions (common in bacteria) — use recombination-aware tools (ClonalFrameML).

2.8 Variant Calling & QC

2.8.1 Tools Summary Table

Tool	Purpose	Typical Use
GATK	Germline/somatic variant calling (HaplotypeCaller, Mutect2)	Gold-standard, best practices pipeline
bcftools (Danecek et al., 2021)	Fast variant calling & manipulation	Large-cohort calling, filtering
SAMtools (Li et al., 2009)	BAM/CRAM manipulation, indexing, stats	Alignment QC
BEDTools	Genomic interval operations (intersect, merge)	Annotation overlap, region filtering

2.8.2 Code

Interval intersection with BEDTools

```
bedtools intersect -a variants.vcf -b gene_regions.bed -header >  
↪ variants_in_genes.vcf
```

QC metrics

```
samtools flagstat sample.bam  
bcftools stats sample.vcf.gz | grep "^SN"
```

2.8.3 Pitfalls

- Strand bias and low mapping quality reads inflate false variant calls — apply GATK hard filters (QD, FS, MQ, ReadPosRankSum) or VQSR.
- Multi-allelic sites must be normalized/split (`bcftools norm -m -both`) before downstream annotation.

2.9 Network & Pathway Analysis

2.9.1 Conceptual Foundation

Maps genes/proteins/metabolites onto known biological networks (protein-protein interaction, KEGG/Reactome pathways) to interpret omics results at a systems level.

2.9.2 Methods

- **Over-representation analysis (ORA)**: hypergeometric/Fisher's exact test for pathway enrichment.
- **Gene Set Enrichment Analysis (GSEA)**: rank-based enrichment (Kolmogorov-Smirnov statistic).

$$p = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}} \quad (\text{hypergeometric test})$$

2.9.3 Tools

- **Cytoscape** — network visualization; plugins: ClueGO, StringApp.
- **STRING database** (Szklarczyk et al., 2023) — PPI networks.
- **clusterProfiler (R/Bioconductor)** — automated ORA/GSEA.

2.9.4 Code (R)

```
library(clusterProfiler)
ego <- enrichGO(gene = sig_genes, OrgDb = org.Hs.eg.db, ont = "BP", pAdjustMethod
↪ = "BH")
dotplot(ego)
```

2.9.5 Pitfalls

- Gene-set redundancy inflates apparent significance — always use FDR-adjusted p-values, and prefer semantic-similarity-reduced results.

- Background gene set choice matters — use the full array/panel background, not the whole genome, if the platform doesn't measure the whole genome.

2.10 Mendelian Randomization (Davey Smith & Ebrahim, 2003)

2.10.1 Conceptual Foundation

Uses genetic variants as instrumental variables (IVs) to infer causal effects of an exposure on an outcome, leveraging Mendel's law of random allele assortment (analogous to natural randomization).

2.10.2 Core Assumptions

1. **Relevance:** IV strongly associated with exposure.
2. **Independence:** IV independent of confounders.
3. **Exclusion restriction:** IV affects outcome only through the exposure.

2.10.3 Formula (Two-Sample MR, Inverse-Variance Weighted)

$$\hat{\beta}_{IVW} = \frac{\sum_j w_j \hat{\beta}_{Yj} / \hat{\beta}_{Xj}}{\sum_j w_j}, \quad w_j = \left(\frac{\hat{\beta}_{Xj}}{se(\hat{\beta}_{Yj})} \right)^2$$

2.10.4 Code (R — TwoSampleMR)

```
library(TwoSampleMR)
exposure_dat <- extract_instruments(outcomes = "ieu-a-2")
outcome_dat <- extract_outcome_data(snps = exposure_dat$SNP, outcomes =
  ↪ "ieu-a-7")
dat <- harmonise_data(exposure_dat, outcome_dat)
res <- mr(dat, method_list = c("mr_ivw", "mr_egger_regression",
  ↪ "mr_weighted_median"))
mr_pleiotropy_test(dat) # tests exclusion restriction violation
```

2.10.5 Pitfalls

- Pleiotropy (IV affects outcome via other pathways) violates exclusion restriction — test with MR-Egger intercept.
- Weak instrument bias — check F-statistic > 10.

2.11 Meta-Analysis

2.11.1 Conceptual Foundation

Combines effect estimates across independent studies to increase power and generalizability.

2.11.2 Fixed vs. Random Effects

$$\hat{\beta}_{FE} = \frac{\sum_i w_i \hat{\beta}_i}{\sum_i w_i}, \quad w_i = \frac{1}{se_i^2}$$

Random effects (DerSimonian-Laird) add between-study variance τ^2 :

$$w_i^* = \frac{1}{se_i^2 + \tau^2}$$

2.11.3 Heterogeneity

$$I^2 = \max\left(0, \frac{Q - (k - 1)}{Q}\right) \times 100\%$$

2.11.4 Code (R — metafor)

```
library(metafor)
res <- rma(yi = effect_size, sei = se, data = studies, method = "REML")
forest(res)
```

2.11.5 Pitfalls

- High I^2 (>75%) signals substantial heterogeneity — investigate via meta-regression or subgroup analysis rather than pooling blindly.
- Publication bias — assess with funnel plot and Egger's test.

2.12 CpG / Gene / Variant Prioritization

2.12.1 Conceptual Foundation

Ranks candidate CpGs, genes, or variants using convergent evidence (statistical significance + functional annotation + independent validation) rather than p-value alone.

2.12.2 Prioritization Framework

Candidate list (from EWAS/GWAS/DE analysis)

- > Statistical strength (effect size, FDR)
- > Functional annotation (ENCODE, Roadmap Epigenomics chromatin state)
- > Cross-omics convergence (e.g., CpG maps to eQTM gene, gene maps to GWAS locus)
- > Independent replication cohort
- > Biological plausibility (pathway membership, prior literature)
- > Final ranked candidate list

2.12.3 Pitfalls

- Relying solely on p-value rank ignores biological plausibility and can prioritize statistical noise — always triangulate with independent evidence layers (multi-omics convergence, as in your traceAsthma/TRACE framework).

2.13 References

Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457-481.

McKenna, A., Hanna, M., Banks, E., Sivachenko, A., Cibulskis, K., Kernyt-sky, A., Garimella, K., Altshuler, D., Gabriel, S., Daly, M., & DePristo, M. A. (2010). The Genome Analysis Toolkit: A MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Research*, 20(9), 1297-1303.

Van der Auwera, G. A., & O'Connor, B. D. (2020). *Genomics in the cloud: Using Docker, GATK, and WDL in Terra*. O'Reilly Media.

Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079.

Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008.

Dobin, A., Davis, C. A., Schlesinger, F., Drenkow, J., Zaleski, C., Jha, S., Batut, P., Chaisson, M., & Gingeras, T. R. (2013). STAR: Ultrafast universal RNA-seq aligner. *Bioinformatics*, 29(1), 15-21.

Kim, D., Paggi, J. M., Park, C., Bennett, C., & Salzberg, S. L. (2019). Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Na-*

ture Biotechnology, 37(8), 907-915.

Langmead, B., & Salzberg, S. L. (2012). Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 9(4), 357-359.

Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12), 550.

Robinson, M. D., McCarthy, D. J., & Smyth, G. K. (2010). edgeR: A Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics*, 26(1), 139-140.

Zhang, Y., Parmigiani, G., & Johnson, W. E. (2020). ComBat-seq: Batch effect adjustment for RNA-seq count data. *NAR Genomics and Bioinformatics*, 2(3), lqaa078.

Langfelder, P., & Horvath, S. (2008). WGCNA: An R package for weighted correlation network analysis. *BMC Bioinformatics*, 9, 559.

Argelaguet, R., Arnol, D., Bredikhin, D., Deloro, Y., Velten, B., Marioni, J. C., & Stegle, O. (2020). MOFA+: A statistical framework for comprehensive integration of multi-modal single-cell data. *Genome Biology*, 21, 111.

Davey Smith, G., & Ebrahim, S. (2003). 'Mendelian randomization': Can genetic epidemiology contribute to understanding environmental determinants of disease? *International Journal of Epidemiology*, 32(1), 1-22.

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289-300.

Weinstein, J. N., Collisson, E. A., Mills, G. B., Shaw, K. R., Ozenberger, B. A., Ellrott, K., Shmulevich, I., Sander, C., Stuart, J. M., & Cancer Genome Atlas Research Network. (2013). The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10), 1113-1120.

GTEx Consortium. (2020). The GTEx Consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509), 1318-1330.

Bycroft, C., Freeman, C., Petkova, D., Band, G., Elliott, L. T., Sharp, K., Molyer, A., Vukcevic, D., Delaneau, O., O'Connell, J., & Marchini, J. (2018). The UK Biobank resource with deep phenotyping and genomic data. *Nature*, 562(7726), 203-209.

3 Survival Analysis: Classical, Machine-Learning, and Multimodal Methods

This chapter provides a complete, self-contained treatment of time-to-event (survival) analysis, spanning classical estimators through modern machine-learning, deep-learning, and multimodal survival models. Each model follows a consistent template: description, mathematical notation, assumptions, advantages, limitations, data requirements, interpretation guidance, example results, and code.

3.1 Foundations of Time-to-Event Data

3.1.1 Core Definitions

Survival analysis models the time T until an event of interest (death, relapse, exacerbation, device failure). Because not every subject experiences the event during the observation window, the data are **censored**: for censored subjects, only a lower bound on T is known.

Survival function: $S(t) = P(T > t)$ — the probability of remaining event-free beyond time t .

Hazard function: $h(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}$ — the instantaneous event rate among those still at risk at time t .

Relationship: $S(t) = \exp\left(-\int_0^t h(u) du\right) = \exp(-H(t))$, where $H(t)$ is the cumulative hazard.

3.1.2 Censoring Types

- **Right censoring** (most common): the subject's event time is only known to exceed the last observed follow-up time (e.g., still event-free at study end, or lost to follow-up).
- **Left censoring:** the event is known to have occurred before a certain time, but the exact time is unknown.

- **Interval censoring:** the event is known to have occurred between two observation times (e.g., between two scheduled clinic visits).
- **Competing risks:** a subject can experience one of several mutually exclusive event types (e.g., relapse vs. death without relapse), where the occurrence of one event precludes observing the other.

3.1.3 Assumption Common to Nearly All Classical Survival Models

Non-informative (independent) censoring: the reason a subject is censored must be unrelated to their underlying, unobserved risk of the event. Violations (e.g., sicker patients dropping out) bias survival estimates upward and are difficult to detect from the data alone — this parallels the untestable sequential-ignorability assumption in mediation analysis (Chapter 4).

3.2 Kaplan-Meier Estimator

Model Description: A non-parametric estimator of the survival function (Kaplan & Meier, 1958), requiring no assumption about the shape of the hazard over time. It is almost always the first descriptive step in any survival analysis, typically compared visually across groups before any regression modeling.

Mathematical Notation:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right)$$

where t_i are the distinct observed event times, d_i is the number of events at t_i , and n_i is the number at risk just before t_i .

Assumptions: Non-informative censoring; event times are observed precisely (or the data are treated as if they are); no covariate adjustment (a purely descriptive, unconditional estimate).

Advantages: No distributional assumptions; simple, universally interpretable step-function output; standard first step in any regulatory submission involving a time-to-event endpoint.

Limitations: Cannot adjust for covariates or confounders directly (comparisons across groups require a separate test, e.g., log-rank); provides no basis for extrapolation beyond the observed follow-up period; large-sample standard errors (Greenwood's formula) can be unstable in the curve's right tail where few subjects remain at risk.

Data Requirements: Time-to-event or time-to-censoring for each subject, plus an event indicator (1 = event, 0 = censored); no covariates strictly required,

though group membership (e.g., treatment arm) is typically used to stratify the plot.

Interpretation Guidance: Read the curve’s height at any time t as the estimated proportion still event-free at that time; a steeper drop indicates a period of higher event rate; compare curves across groups visually and formally via the log-rank test (χ^2 test comparing observed vs. expected events under the null of equal survival).

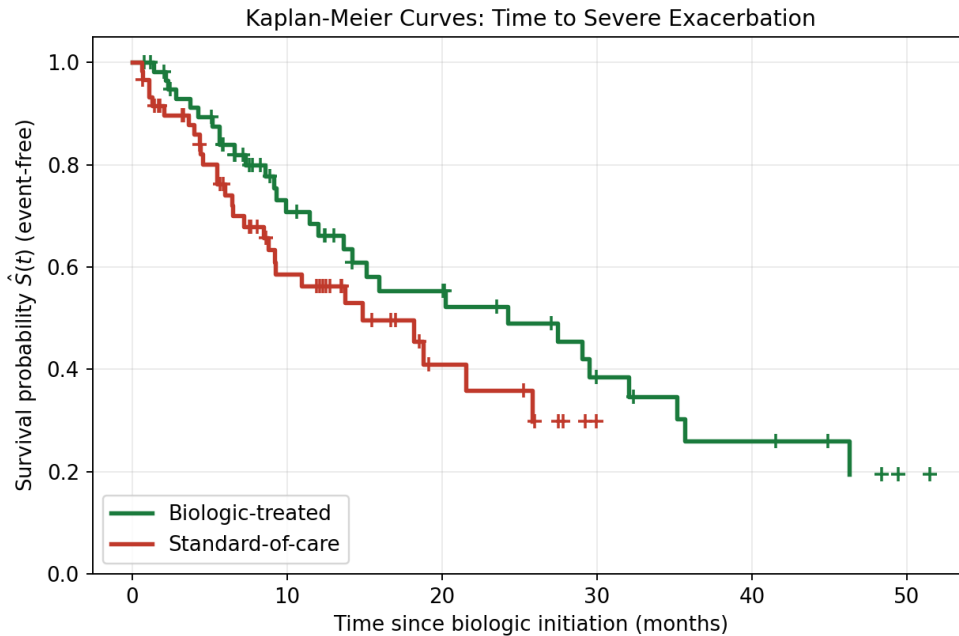


Figure 3.1: **Kaplan-Meier curves comparing two treatment groups.** Step-function survival curves for biologic-treated versus standard-of-care patients, with tick marks (+) indicating censored observations.

Interpretation: The biologic-treated curve (green) remains consistently above the standard-of-care curve (red) throughout follow-up, indicating a lower cumulative probability of severe exacerbation at every time point shown. The censoring tick marks show where patients exited the risk set without an event (e.g., study ended before they had an exacerbation) — these patients still contribute information up to their last observed time but do not count as failures. A log-rank test comparing these two curves would formally test whether the visual separation is statistically distinguishable from chance.

3.2.1 Example Result — Log-Rank Test Output

Comparison	Events (Biologic)	Events (SOC)	Log-rank χ^2	p-value
Biologic vs. Standard of Care	14 / 60	24 / 60	5.82	0.016

Interpretation: With fewer observed events in the biologic arm (14 vs. 24) despite equal group sizes, and a log-rank p-value below 0.05, there is statistical evidence that the two survival curves differ — consistent with what the KM plot shows visually. Note that the log-rank test, like the KM estimator itself, does not adjust for baseline covariate imbalances between arms; a Cox model (next section) is needed for that.

3.2.2 Code (R)

```
library(survival)
fit_km <- survfit(Surv(time, status) ~ treatment, data = df)
plot(fit_km, col = c("firebrick", "forestgreen"), mark.time = TRUE)
survdif(Surv(time, status) ~ treatment, data = df) # log-rank test

from lifelines import KaplanMeierFitter
kmf = KaplanMeierFitter()
kmf.fit(durations=df["time"], event_observed=df["status"])
kmf.plot_survival_function()
```

3.3 Cox Proportional Hazards Model

Model Description: A semi-parametric regression model relating covariates to the hazard function (Cox, 1972) without specifying the baseline hazard's shape, making it the default workhorse for covariate-adjusted survival analysis in clinical research.

Mathematical Notation:

$$h(t \mid X) = h_0(t) \exp(\beta_1 X_1 + \cdots + \beta_p X_p)$$

Estimated via partial likelihood (baseline hazard $h_0(t)$ cancels out):

$$L(\beta) = \prod_{i: \delta_i = 1} \frac{\exp(\beta^\top X_i)}{\sum_{j \in R(t_i)} \exp(\beta^\top X_j)}$$

where $R(t_i)$ is the risk set at event time t_i and δ_i is the event indicator.

Assumptions: Proportional hazards (PH) — the hazard ratio between any two covariate levels is constant over time; log-linearity of continuous covariates on the log-hazard scale (or explicit modeling of nonlinearity via splines); non-informative censoring; independent observations (violated by repeated events per subject — see Recurrent Event Models below).

Advantages: No need to specify the baseline hazard’s functional form; hazard ratios are widely understood and reported; extensive tooling, diagnostics, and regulatory precedent; naturally extends to stratified and time-varying-covariate versions.

Limitations: Proportional hazards can be violated in practice (e.g., a surgical treatment’s benefit may fade or grow over time); provides only *relative* risk (hazard ratios), not absolute survival probabilities, without an additional baseline hazard estimate; sensitive to unmeasured confounding, like any observational regression.

Data Requirements: Time-to-event, event indicator, and a covariate matrix; sample size guidance is often expressed in **events** (not total n) — a common rule of thumb is at least 10 events per estimated covariate to avoid unstable estimates.

Interpretation Guidance: $\exp(\hat{\beta})$ is the hazard ratio (HR): $HR > 1$ indicates increased hazard (worse prognosis) per unit increase in the covariate; $HR < 1$ indicates a protective effect. Always check the proportional-hazards assumption via Schoenfeld residuals (`cox.zph()`) before trusting a single, time-constant hazard ratio.

Interpretation: Covariates whose confidence interval crosses $HR=1$ (age, blood eosinophils in this example) are not statistically distinguishable from no effect at the 5% level, while biologic therapy (HR well below 1) and baseline FEV1 < 50% (HR well above 1) show intervals entirely on one side of the line — the log scale is used because hazard ratios are inherently multiplicative, so equal visual distances represent equal *relative* rather than absolute changes in risk. Biologic therapy’s HR of 0.52 means treated patients experience roughly half the hazard of exacerbation at any given time point compared to untreated patients, holding the other covariates fixed.

3.3.1 Example Result — Cox Model Output Table

Covariate	HR	95% CI	p-value
Biologic therapy (vs. standard care)	0.52	(0.33, 0.81)	0.004
Age (per 10 years)	1.18	(0.98, 1.42)	0.08
Baseline FEV1 < 50%	2.05	(1.21, 3.48)	0.007

Covariate	HR	95% CI	p-value
Blood eosinophils (per 100 cells/ μ L)	1.09	(0.97, 1.23)	0.15
Prior exacerbation (past year)	1.74	(1.12, 2.71)	0.014

Schoenfeld residual test for PH assumption: global $p = 0.31$ (no evidence of PH violation).

Interpretation: The non-significant global Schoenfeld test supports treating all five hazard ratios as constant over the follow-up period, so the single HR per covariate is an adequate summary. Had this test been significant for a specific covariate (e.g., biologic therapy), the appropriate next step would be a time-varying-coefficient model or stratification by follow-up period, since a single hazard ratio would then be misleading (e.g., averaging over an early strong effect and a later fading one).

3.3.2 Code (R)

```
library(survival)
cox_fit <- coxph(Surv(time, status) ~ treatment + age + fev1_low + eos +
  ↪ prior_exac, data = df)
summary(cox_fit)
cox.zph(cox_fit) # test proportional hazards assumption

from lifelines import CoxPHFitter
cph = CoxPHFitter()
cph.fit(df, duration_col="time", event_col="status")
cph.print_summary()
cph.check_assumptions(df, p_value_threshold=0.05)
```

3.4 Parametric Survival Models

Model Description: Fully parametric models that specify an explicit distributional form for the survival/hazard function (Exponential, Weibull, Log-normal, Log-logistic, Gompertz), enabling direct estimation of absolute survival probabilities and extrapolation beyond the observed follow-up — a common requirement in health-economic modeling.

Mathematical Notation: - **Exponential:** constant hazard, $h(t) = \lambda$, $S(t) = \exp(-\lambda t)$. - **Weibull:** $h(t) = \lambda \gamma t^{\gamma-1}$, $S(t) = \exp(-\lambda t^\gamma)$ — monotone increasing ($\gamma > 1$) or decreasing ($\gamma < 1$) hazard. - **Gompertz:** $h(t) = \lambda \exp(\gamma t)$ — exponentially increasing hazard, common for age-related mortality. - **Log-**

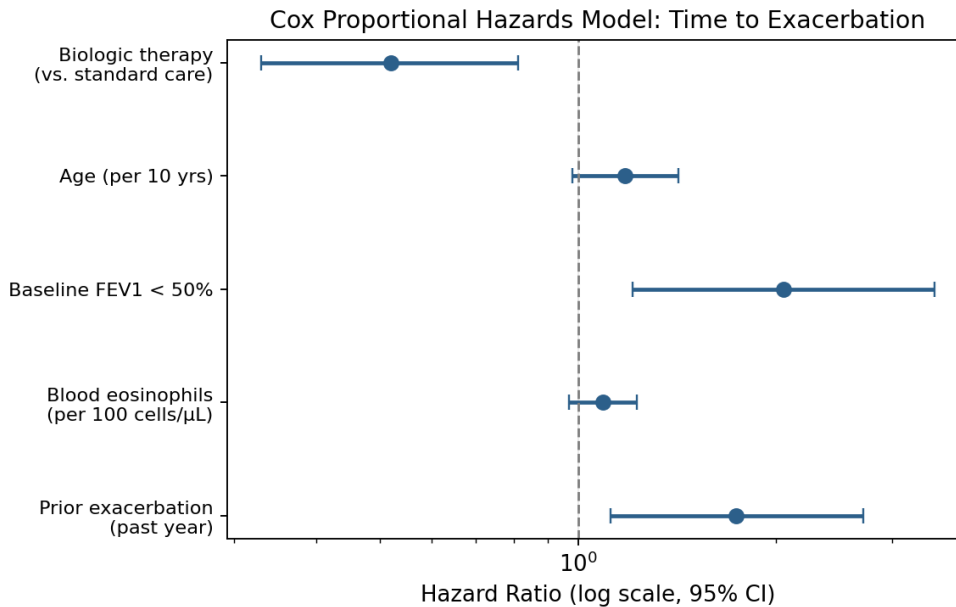


Figure 3.2: **Cox model forest plot.** Hazard ratios (points) and 95% confidence intervals (error bars) for five covariates in a model of time to severe asthma exacerbation, plotted on a log scale. The vertical dashed line at HR=1 marks no effect.

logistic: hazard $h(t) = \frac{\lambda \gamma t^{\gamma-1}}{1 + \lambda t^\gamma}$ — can rise then fall, useful for post-surgical risk that peaks and later declines. - **Log-normal:** $\log T \sim N(\mu, \sigma^2)$ — similarly non-monotone hazard shape.

Assumptions: The chosen distributional family correctly describes the true hazard shape (a strong, checkable-but-often-violated assumption); non-informative censoring; (for AFT-style parameterizations, see below) covariate effects act multiplicatively on survival time rather than additively on the log-hazard.

Advantages: Direct estimates of absolute survival/hazard at any time point, including beyond the observed data (extrapolation) — essential for cost-effectiveness/health-economic models submitted to payers; often more statistically efficient (narrower CIs) than Cox when the distributional assumption holds; smooth, fully specified hazard simplifies simulation.

Limitations: Model misspecification (wrong distributional family) biases every downstream estimate, especially extrapolated tails; more assumptions than the semi-parametric Cox model; choosing among five-plus candidate distributions adds a model-selection step (via AIC/BIC or visual hazard-shape diagnostics) not needed for Cox.

Data Requirements: Same as Cox (time, event indicator, covariates); ideally enough events across the follow-up period to visually or statistically distinguish between candidate hazard shapes (e.g., via a log cumulative hazard plot).

Interpretation Guidance: Compare candidate distributions via AIC/BIC and via a plot of the fitted hazard function against a non-parametric (e.g., smoothed Nelson-Aalen) hazard estimate; report both hazard ratios (for the Weibull PH parameterization) and, where relevant, time ratios (for the AFT parameterization, below).

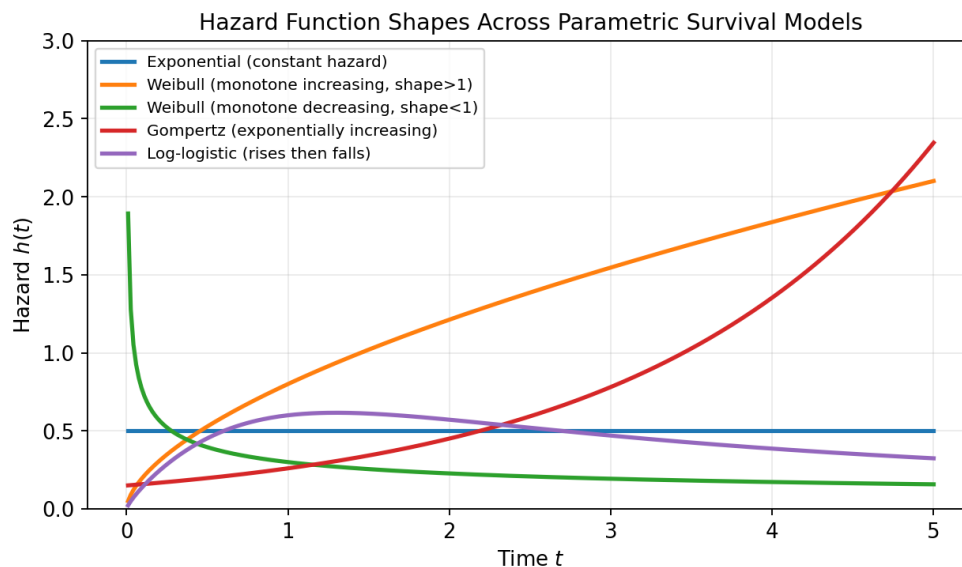


Figure 3.3: **Hazard function shapes across five parametric survival distributions.** Exponential’s flat, constant hazard contrasts with Weibull’s monotone increasing/decreasing shapes, Gompertz’s exponential increase, and log-logistic’s rise-then-fall pattern.

Interpretation: Choosing among these families is fundamentally a question about the *shape* of risk over time for the specific clinical context. A constant (Exponential) hazard implies no “aging” or accumulating risk — appropriate for, e.g., a sudden random failure mode with no wear-out effect. A monotonically increasing Weibull or Gompertz hazard suits a progressive disease process or age-related mortality. A log-logistic or log-normal hazard, which rises and then falls, is often the right choice for post-operative or post-treatment risk that peaks in an early window and then declines as survivors stabilize. Choosing the wrong shape doesn’t just bias the fitted curve visually — it specifically distorts any extrapolated tail used for long-term cost-effectiveness projections, which is exactly where these models are most consequential.

3.4.1 Example Result — Model Selection by AIC

Distribution	Log-likelihood	AIC	Rank
Exponential	-412.6	827.2	4
Weibull	-401.3	806.6	1
Log-normal	-403.9	811.8	2
Log-logistic	-404.5	813.0	3
Gompertz	-407.1	818.2	5

Interpretation: The Weibull model achieves the lowest AIC, indicating the best balance of fit and parsimony among the five candidates for this dataset, and would typically be selected for the primary extrapolation used in a health-economic model — though a visual hazard-shape check against the data (not shown) should always accompany a purely AIC-based choice, since AIC differences of only a few points (as between Weibull and log-normal here) do not represent decisive evidence for one shape over another.

3.4.2 Code (R)

```
library(flexsurv)
fit_weibull <- flexsurvreg(Surv(time, status) ~ treatment + age, data = df, dist
  ↪ = "weibull")
fit_lognormal <- flexsurvreg(Surv(time, status) ~ treatment + age, data = df,
  ↪ dist = "lognormal")
AIC(fit_weibull, fit_lognormal)
```

3.5 Accelerated Failure Time (AFT) Models

Model Description: An alternative parameterization in which covariates act multiplicatively on the survival *time* itself, rather than on the hazard — a covariate can be interpreted as accelerating or decelerating the pace toward the event, which is often more intuitive for patients and clinicians than a hazard ratio.

Mathematical Notation:

$$\log T = \mu + \beta^\top X + \sigma\epsilon$$

where ϵ follows a specified distribution (e.g., standard normal for log-normal AFT, standard extreme-value for Weibull AFT — the Weibull is the one distribution that admits both a PH and an AFT parameterization). The **time ratio**

(TR) $\exp(\beta)$ describes how a one-unit covariate change multiplies the expected survival time.

Assumptions: Correctly specified error distribution for ϵ ; the covariate effect is genuinely multiplicative on time (constant time ratio across the follow-up period) — the AFT analogue of the PH assumption.

Advantages: Direct, intuitive interpretation (“this treatment extends expected survival time by a factor of $1.4\times$ ”); remains valid under certain PH violations that would bias a Cox model; useful in regulatory contexts (e.g., oncology) where median-survival-extension framing is standard.

Limitations: Requires a correctly specified error distribution, similarly to other parametric models; less universally used/understood in general biostatistics practice compared to the Cox HR framing; software support is less uniformly available than for Cox.

Data Requirements: Same as Cox/parametric models.

Interpretation Guidance: $TR > 1$ means the covariate *extends* survival time (protective); $TR < 1$ means it *shortens* survival time (harmful) — the inverse directionality from a hazard ratio’s interpretation is a common source of confusion and should always be stated explicitly in a results table.

3.5.1 Example Result — AFT (Weibull) Output

Covariate	Time Ratio (TR)	95% CI	Interpretation
Biologic therapy	1.62	(1.15, 2.28)	Extends expected time-to-exacerbation by 62%
Baseline FEV1 < 50%	0.58	(0.39, 0.86)	Shortens expected time-to-exacerbation by 42%

3.5.2 Code (R)

```
library(survival)
aft_fit <- survreg(Surv(time, status) ~ treatment + fev1_low, data = df, dist =
  ↪ "weibull")
summary(aft_fit)
exp(coef(aft_fit)) # time ratios
```

3.6 Competing Risks (Fine-Gray Model)

Model Description: Models the **cumulative incidence function (CIF)** (Fine & Gray, 1999) of one event type in the presence of other, mutually exclusive event types — for example, relapse versus death without relapse, where death precludes ever observing a subsequent relapse.

Mathematical Notation: Cumulative incidence for cause k :

$$CIF_k(t) = \int_0^t S(u^-) h_k(u) du$$

where $h_k(u)$ is the cause-specific hazard for event type k and $S(u^-)$ is the overall event-free survival just before u (accounting for *all* competing causes). The **Fine-Gray subdistribution hazard model** regresses covariates directly on the subdistribution hazard, giving hazard ratios that map directly onto CIF comparisons.

Assumptions: Non-informative censoring (as before); correct specification of which events compete with which; the Fine-Gray model’s subdistribution hazard has a less intuitive risk-set definition than a standard cause-specific hazard (subjects who experience a competing event remain artificially “at risk” in the subdistribution risk set) — an assumption/convention that is frequently misunderstood even by experienced analysts.

Advantages: Produces CIFs that sum correctly across competing event types (unlike naively treating competing events as censoring); Fine-Gray hazard ratios map directly onto the clinically relevant cumulative incidence scale.

Limitations: Naively applying standard KM/Cox methodology to one event type while treating competing events as “censored” **overestimates** the risk of that event type, since it implicitly assumes competing-event subjects would have gone on to experience the event of interest had they not died/dropped out first — a common and serious analytical error; Fine-Gray coefficients do not have as clean a cause-specific hazard interpretation as a standard Cox model.

Data Requirements: Event type indicator (not just event/censoring) distinguishing which of several competing events occurred, in addition to time and covariates.

Interpretation Guidance: Always report CIFs (not naive 1-KM curves) for any endpoint with a plausible competing risk; a Fine-Gray hazard ratio > 1 means the covariate increases the subdistribution hazard, translating to a higher cumulative incidence of that specific event type over time, accounting

for the competing event.

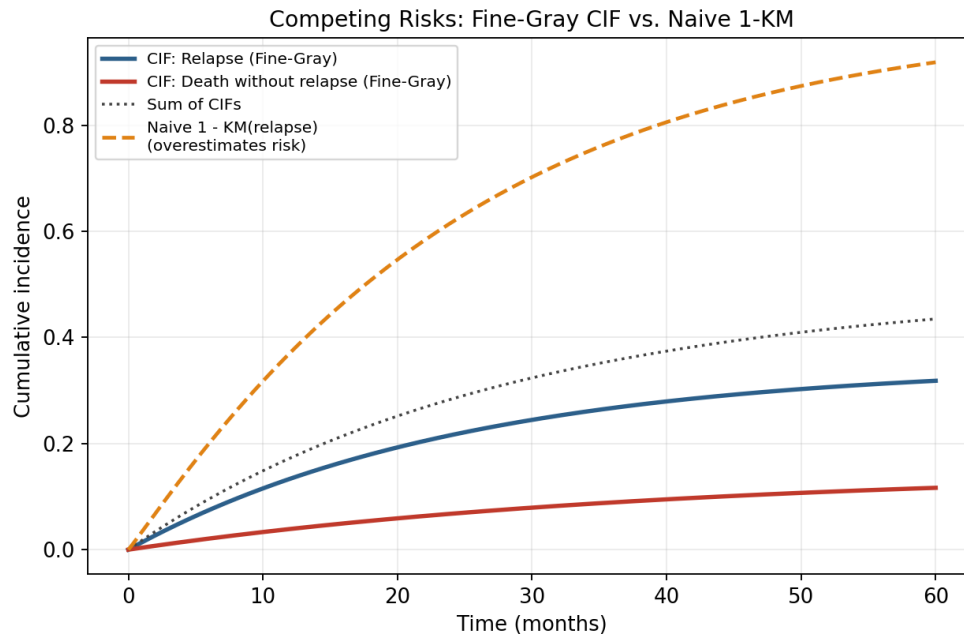


Figure 3.4: **Cumulative incidence functions under competing risks.** Fine-Gray CIFs for relapse (blue) and death without relapse (red) sum to the total event probability (dotted grey); the naive “1 minus Kaplan-Meier” curve for relapse (orange dashed), which incorrectly treats death as simple censoring, visibly overestimates the true relapse risk.

Interpretation: The gap between the naive 1-KM curve (orange) and the correct Fine-Gray CIF for relapse (blue) is the direct visual consequence of the estimation error described above: 1-KM implicitly assumes that patients who died without relapsing would eventually have relapsed had they lived long enough, inflating the apparent relapse risk. The correct approach — computing the CIF for each competing event separately, so that they sum to the total cumulative event probability — is the only way to get a risk estimate that is both internally consistent and clinically accurate when more than one type of event is possible.

3.6.1 Example Result — Fine-Gray Model Output

Covariate	Subdistribution HR (Relapse)	95% CI	p-value
Biologic therapy	0.61	(0.41, 0.91)	0.015
Age (per 10 years)	1.05	(0.89, 1.24)	0.55

Interpretation: Biologic therapy’s subdistribution HR of 0.61 for relapse

means treated patients have a lower cumulative incidence of relapse over time, properly accounting for the competing risk of death without relapse — a naive analysis treating death as censoring could have produced a materially different (and less trustworthy) estimate of this same treatment effect.

3.6.2 Code (R)

```
library(cmprsk)
fg_fit <- crr(ftime = df$time, fstatus = df$event_type, cov1 = df[,
  ↪ c("treatment", "age")])
summary(fg_fit)
```

3.7 Recurrent Event Models (Andersen-Gill, PWP, WLW)

Model Description: Extensions of the Cox model for outcomes that can recur multiple times per subject (e.g., repeated hospitalizations, repeated exacerbations), rather than a single terminal event.

Mathematical Notation: All three approaches extend the Cox partial likelihood to multiple event times per subject, differing in how they define risk intervals and stratify: - **Andersen-Gill (AG)** (Andersen & Gill, 1982): treats each subject’s follow-up as a sequence of (start, stop] intervals, all contributing to a single pooled risk set, assuming events are independent within a subject after conditioning on covariates (including a time-varying event-count covariate if desired). - **Prentice-Williams-Peterson (PWP)** (Prentice, Williams, & Peterson, 1981): stratifies the risk set by event number (a subject isn’t at risk for their 2nd event until their 1st has occurred), in either a total-time or gap-time formulation. - **Wei-Lin-Weissfeld (WLW)** (Wei, Lin, & Weissfeld, 1989): treats each event number as a separate, marginal Cox model (all subjects at risk for “event slot k” regardless of whether they had events 1 through k-1), analyzed jointly with robust standard errors.

Assumptions: All three require non-informative censoring; AG additionally assumes the risk of subsequent events is well captured by the pooled model without explicit stratification on event order (can be relaxed by adding an event-count covariate); PWP explicitly assumes risk depends on event order (appropriate when, e.g., a 3rd hospitalization carries different risk than a 1st, holding covariates fixed); WLW’s marginal formulation can be harder to justify clinically since it treats “the risk of a subject’s 3rd event” as defined even for subjects who never had a 1st or 2nd.

Advantages: Makes full use of all recurrent event information per subject rather than only analyzing time-to-first-event (which discards data and can bias

effect estimates for chronic, recurring conditions); robust (“sandwich”) standard errors account for within-subject correlation across repeated events.

Limitations: Choice among AG/PWP/WLW is not always obvious and can materially change effect estimates; interpretation is more complex than a single-event Cox model; robust variance estimation is essential and easy to omit by mistake.

Data Requirements: Multiple (start, stop, event) intervals per subject in “counting process” format, with a subject ID to link intervals belonging to the same person.

Interpretation Guidance: An AG hazard ratio describes the effect on the instantaneous rate of *any* (re)event, pooling across event number; a PWP hazard ratio is conditional on having already had a given number of prior events, which can reveal whether treatment effects change as disease recurrence accumulates.

3.7.1 Example Result — Recurrent Exacerbations (Andersen-Gill Model)

Covariate	HR (any recurrence)	95% CI	p-value
Biologic therapy	0.55	(0.42, 0.72)	<0.001
Prior exacerbation count	1.21	(1.09, 1.34)	<0.001

Interpretation: Modeling all recurrent exacerbations (rather than only the first) both increases statistical power (more events contribute information) and directly answers the clinically relevant question of whether a treatment reduces the *ongoing* burden of exacerbations, not just delays a single first event — the significant HR for prior exacerbation count also quantifies a “risk begets risk” pattern common in chronic respiratory disease.

3.7.2 Code (R)

```
library(survival)
# Counting-process format: (start, stop, event) per row, subject_id links rows
ag_fit <- coxph(Surv(start, stop, event) ~ treatment + prior_count +
  ↪ cluster(subject_id), data = df_long)
summary(ag_fit)
```

3.8 Frailty Models (Random-Effects Survival Models)

Model Description: The survival-analysis analogue of a mixed-effects model — introduces a subject- or cluster-level random effect (the “frailty”) into the hazard function, to account for unobserved heterogeneity in baseline risk shared within clusters (e.g., patients within a clinical site, or repeated events within a patient).

Mathematical Notation:

$$h_{ij}(t) = h_0(t) u_i \exp(\beta^\top X_{ij})$$

where u_i is the frailty term for cluster/subject i , typically assumed $u_i \sim \text{Gamma}(\theta, \theta)$ (mean 1, variance $1/\theta$) or log-normal.

Assumptions: The frailty distribution (Gamma or log-normal) is correctly specified; frailties are independent of covariates and of the censoring mechanism; conditional on the frailty, events within a cluster are independent (the same conditional-independence logic underlying mixed-effects models in Chapter 1).

Advantages: Explicitly accounts for within-cluster correlation (multi-center trials, recurrent events, litter/family-clustered data) that a standard Cox model ignores; the estimated frailty variance itself quantifies how much unexplained heterogeneity in baseline risk exists across clusters, which can be informative in its own right (e.g., revealing substantial unmeasured site-level variation in a multi-center trial).

Limitations: Choice of frailty distribution can affect results, and is harder to check than choosing a covariate functional form; adds a variance-component estimation problem similar to mixed models, with corresponding convergence/identifiability challenges in small samples; interpretation of the frailty variance itself requires care (it’s on a multiplicative hazard scale, less intuitive than a mixed model’s additive random intercept).

Data Requirements: A cluster/subject identifier linking correlated observations, in addition to standard survival data.

Interpretation Guidance: A hazard ratio from a frailty model is interpreted as **conditional on the frailty** (i.e., for two subjects with the *same* frailty value) — this is subtly different from, and typically larger in magnitude than, the population-averaged hazard ratio a marginal (GEE-style) survival model would report, mirroring the same subject-specific-vs-population-average distinction between mixed models and GEE in Chapter 1.

3.8.1 Code (R)

```
library(survival)
frailty_fit <- coxph(Surv(time, status) ~ treatment + age + frailty(site,
  ↪ distribution="gamma"), data = df)
summary(frailty_fit)
```

3.9 Joint Models (Longitudinal + Survival)

Model Description: Simultaneously models a longitudinal biomarker trajectory (via a mixed-effects submodel) and a time-to-event outcome (via a survival submodel), linking the two through a shared or correlated random-effects structure — directly relevant to using a biomarker like FEV1 or a rising tumor marker to dynamically predict event risk.

Mathematical Notation: Longitudinal submodel: $m_i(t) = X_i(t)\beta + Z_i(t)b_i$ (as in Chapter 1's mixed-effects model) Survival submodel: $h_i(t) = h_0(t) \exp(\gamma^\top W_i + \alpha m_i(t))$ where α quantifies the association between the *current, true, underlying* value of the longitudinal trajectory $m_i(t)$ (not the noisy observed value) and the hazard at the same time t .

Assumptions: Correct specification of both submodels (the longitudinal mixed-effects model's assumptions from Chapter 1, plus the survival submodel's PH-style assumption); the shared random effects b_i fully capture the association between the two processes; non-informative dropout is explicitly relaxed and replaced by the assumption that dropout/event timing is captured through the shared random effects (this is precisely why joint models exist: to correctly handle the common real-world case where biomarker measurement stops because the event occurred).

Advantages: Properly accounts for the fact that longitudinal measurement and survival/dropout are usually not independent (a patient who is about to have an event often has a rapidly worsening biomarker, and their biomarker series stops being measured after the event) — using the observed biomarker trajectory naively as a time-varying covariate in a standard Cox model, ignoring its measurement error and informative dropout, biases the estimated association; enables dynamic, updated individual risk prediction as new biomarker measurements accrue.

Limitations: Computationally intensive (joint likelihood over both submodels, typically via MCMC or specialized EM algorithms); requires larger sample sizes and more repeated measurements per subject than either submodel alone; more complex to specify, fit, and communicate to a non-statistical audience than a two-stage naive approach.

Data Requirements: Longitudinal biomarker measurements at multiple time points per subject (as in Chapter 1’s mixed model) plus a time-to-event outcome with censoring indicator.

Interpretation Guidance: The association parameter α answers “how much does the hazard change per unit increase in the *true underlying* biomarker trajectory at the same time,” properly separating measurement noise from the genuine biomarker-risk relationship — a subtly different (and more defensible) quantity than the coefficient from a naive time-varying-covariate Cox model.

3.9.1 Example Result — Joint Model Association Parameter

Parameter	Estimate	95% CI	Interpretation
α (FEV1 trajectory → exacerbation hazard)	-0.042	(-0.061, -0.023)	Each 1-point drop in true underlying FEV1 % increases hazard by ~4.3%

Interpretation: Because this estimate comes from the joint model’s shared-random-effects structure rather than plugging noisy, observed FEV1 measurements directly into a Cox model, it is not distorted by measurement error or by the informative stopping of FEV1 measurement that occurs right around the time of an exacerbation — both of which would bias a naive time-varying Cox coefficient toward the null.

3.9.2 Code (R)

```
library(JMbayes2)
lme_fit <- lme(fev1 ~ time, random = ~ time | subject_id, data = long_data)
cox_fit <- coxph(Surv(time, status) ~ treatment, data = surv_data, x = TRUE)
joint_fit <- jm(cox_fit, lme_fit, time_var = "time")
summary(joint_fit)
```

3.10 Bayesian Survival Models

Model Description: Any of the survival models above (parametric, Cox, frailty, joint) reformulated in a Bayesian framework, replacing point estimates and confidence intervals with full posterior distributions over survival parameters — particularly valuable for small-sample or rare-event survival analyses (e.g., a

rare pediatric disease) and for formal borrowing of information across related strata via hierarchical priors.

Mathematical Notation: For a parametric (e.g., Weibull) Bayesian survival model:

$$P(\lambda, \gamma, \beta \mid D) \propto \left[\prod_{i:\delta_i=1} h(t_i) \right] \left[\prod_i S(t_i) \right] \times P(\lambda)P(\gamma)P(\beta)$$

combining the survival likelihood (hazard contribution for events, survival contribution for all subjects) with priors on the distributional and regression parameters.

Assumptions: Same distributional/PH-style assumptions as the corresponding frequentist model, plus a chosen prior distribution for each parameter (weakly informative priors are standard absent strong external evidence, exactly as discussed in Chapter 1’s Bayesian-without-priors section).

Advantages: Valid posterior inference even with very few events (a common situation in rare pediatric or orphan-disease trials); naturally supports hierarchical models that partially pool hazard/survival parameters across sites, subgroups, or related studies; directly answers probabilistic questions a frequentist CI cannot (e.g., “what is the probability this drug’s hazard ratio is below 0.7?”).

Limitations: Computationally intensive (MCMC); requires the same prior-sensitivity reporting discussed in Chapter 1 to reassure reviewers the conclusion isn’t prior-driven; less uniform regulatory precedent than a standard frequentist Cox model for a confirmatory primary endpoint, though Bayesian survival analysis is increasingly accepted for adaptive and rare-disease trial designs.

Data Requirements: Same as the corresponding frequentist survival model.

Interpretation Guidance: Report the posterior median/mean hazard ratio alongside a 95% credible interval, and where relevant, a direct posterior probability statement (e.g., $P(HR < 1 \mid D)$) — the latter is often the most clinically useful summary and has no frequentist equivalent.

3.10.1 Code (R)

```
library(rstanarm)
bayes_surv <- stan_surv(Surv(time, status) ~ treatment + age, data = df,
                       basehaz = "weibull", chains = 4, iter = 2000)
```

```
print(bayes_surv)
```

3.11 Machine-Learning Survival Models

Model Description: Tree-ensemble methods adapted for censored time-to-event outcomes, relaxing the Cox model’s proportional-hazards and log-linearity assumptions in favor of flexible, nonparametric risk functions. Includes **Random Survival Forests (RSF)** (Ishwaran, Kogalur, Blackstone, & Lauer, 2008), **XGBoost-Survival** (Cox or AFT objective), **LightGBM-Survival**, and **Survival-SVM**.

Mathematical Notation: RSF grows survival trees using a log-rank splitting rule at each node (maximizing between-daughter-node survival difference) and aggregates via an ensemble cumulative hazard estimate:

$$\hat{H}(t | x) = \frac{1}{B} \sum_{b=1}^B \hat{H}_b(t | x)$$

XGBoost-Survival replaces the standard squared-error/log-loss objective with a **Cox partial-likelihood loss** or an **AFT loss** (accommodating censored observations directly in the gradient/Hessian computation used for tree boosting).

Assumptions: No proportional-hazards assumption required (a key advantage); still assumes non-informative censoring; tree-based methods assume enough events to reliably estimate log-rank splits at each node (very low event counts can produce unstable trees, as with any tree ensemble under limited effective sample size).

Advantages: Capture nonlinear covariate effects and high-order interactions automatically, without manually specifying splines or interaction terms; RSF’s variable importance and XGBoost-Survival’s SHAP compatibility (Chapter 4) provide interpretable feature rankings despite the flexible functional form; often outperform Cox when the true risk relationship is substantially nonlinear or interactive.

Limitations: Lose the clean, single-number hazard-ratio summary that makes Cox output easy to report and communicate; still require careful validation (nested CV) to avoid overfitting, especially in the $p \gg n$ omics regime; less consistent regulatory precedent than Cox for confirmatory endpoints; risk-score outputs require calibration checks (Chapter 7) just as with any ML risk model.

Data Requirements: Same censored time-to-event data as Cox, but can additionally exploit high-dimensional covariates (omics, imaging-derived features)

without the same “10 events per parameter” constraint, given adequate regularization and validation.

Interpretation Guidance: Evaluate discrimination via **Harrell’s C-index** (the survival analogue of AUC — the probability that, for a random pair of subjects, the one who experienced the event first also had the higher predicted risk) rather than AUC directly; always accompany the C-index with a calibration plot (predicted vs. observed survival probability at a fixed horizon).

3.11.1 Example Result — Model Discrimination Comparison

Model	C-index	95% CI
Cox PH (5 covariates)	0.71	(0.66, 0.76)
Random Survival Forest	0.75	(0.70, 0.80)
XGBoost-Survival (Cox objective)	0.77	(0.72, 0.82)

Interpretation: Both ML survival methods modestly outperform the standard Cox model’s discrimination, suggesting meaningful nonlinear or interaction structure in the covariate-risk relationship that Cox’s log-linear form misses — a plausible signal that further justifies the added modeling complexity, provided the improvement is validated on a held-out cohort rather than reported only from training-fold performance.

3.11.2 Code (Python — scikit-survival)

```
from sksurv.ensemble import RandomSurvivalForest
from sksurv.metrics import concordance_index_censored

rsf = RandomSurvivalForest(n_estimators=300, min_samples_leaf=10,
    ↪ random_state=42)
rsf.fit(X_train, y_train_structured)  # y: structured array (event, time)
risk_scores = rsf.predict(X_test)
c_index = concordance_index_censored(y_test["event"], y_test["time"],
    ↪ risk_scores)

import xgboost as xgb
dtrain = xgb.DMatrix(X_train, label=time_train)
dtrain.set_float_info('label_lower_bound', time_train)
dtrain.set_float_info('label_upper_bound', np.where(event_train==1, time_train,
    ↪ np.inf))
params = {"objective": "survival:aft", "eval_metric": "aft-nloglik"}
model = xgb.train(params, dtrain, num_boost_round=300)
```

3.12 Deep-Learning Survival Models

Model Description: Neural-network-based survival models that replace the Cox model’s linear risk score, or the parametric model’s fixed distributional shape, with a flexible neural function — including **DeepSurv** (a neural Cox model; Katzman et al., 2018), **DeepHit** (Lee, Zame, Yoon, & van der Schaar, 2018; a discrete-time competing-risks model with a fully flexible survival distribution), **Nnet-survival** (Gensheimer & Narasimhan, 2019; a discrete-time hazard model trained with standard binary cross-entropy), **Transformer-based survival models** (attending over longitudinal/multimodal input sequences), and **GNN-survival models** (propagating risk information over a graph, e.g., a patient-similarity network or a molecular interaction graph).

Mathematical Notation: DeepSurv replaces the Cox linear predictor with a neural network $f_\theta(X)$:

$$h(t | X) = h_0(t) \exp(f_\theta(X))$$

trained by maximizing the same Cox partial likelihood as before, but with f_θ a multi-layer network rather than a linear combination. DeepHit instead directly models the discrete-time probability mass function of the first event via a softmax output over time bins, jointly optimizing a likelihood term and a ranking-based concordance loss:

$$\mathcal{L} = \mathcal{L}_{likelihood} + \sigma \mathcal{L}_{rank}$$

Assumptions: DeepSurv retains the proportional-hazards assumption on the *learned* risk score $f_\theta(X)$ (a nonlinear function of covariates, but still PH once that transformation is applied); DeepHit and Nnet-survival relax the PH assumption entirely by directly modeling the full discrete-time survival distribution; all deep survival models assume sufficient training data relative to model capacity, and correct patient-level (not observation-level) train/test splitting.

Advantages: Can model highly nonlinear, high-order-interaction risk functions and, for DeepHit/Nnet-survival, arbitrary (non-proportional) hazard shapes; naturally extend to high-dimensional or multimodal inputs (raw images, omics, structured EHR sequences) as covariates via an appropriate encoder (Chapter 4); transformer- and GNN-survival variants can incorporate long-range temporal dependencies or relational/network structure directly.

Limitations: Data-hungry, as with any deep model (Chapter 4); “black box” — requires the same post-hoc interpretability tools (SHAP, integrated gradients, attention maps) discussed in Chapter 4 to explain individual risk predictions; less regulatory precedent than Cox for a confirmatory primary endpoint; cali-

bration is not guaranteed by default and must be explicitly checked and often recalibrated (e.g., via isotonic regression on the predicted survival probabilities).

Data Requirements: Same censored survival data as classical methods, but deep survival models realize their advantage primarily with large sample sizes and/or rich, high-dimensional covariates (images, omics, long EHR sequences) where a neural encoder can extract useful structure a linear Cox model cannot.

Interpretation Guidance: Use the C-index and calibration plots as with ML survival models; for DeepHit, additionally examine the predicted cumulative incidence curves directly (since the model outputs a full discrete-time distribution rather than a single hazard ratio), and use the requisite interpretability tools (Chapter 4) to explain individual-level predictions to a clinical audience.

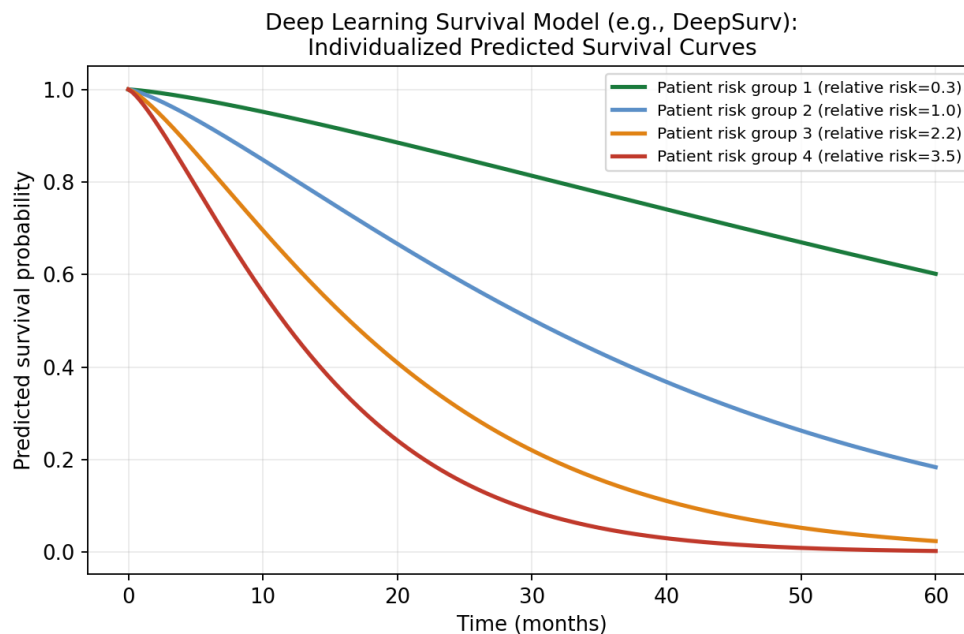


Figure 3.5: **Individualized predicted survival curves from a deep learning survival model (e.g., DeepSurv).** Four patients with different neural-network-derived relative risk scores show visibly different predicted survival trajectories, illustrating the model’s ability to produce fully individualized risk curves rather than a small number of hazard-ratio-defined group averages.

Interpretation: Unlike a Cox model, which effectively produces one baseline survival curve shifted by a single multiplicative hazard ratio per patient, a flexible deep survival model can, in principle, produce curves with genuinely different *shapes* across patients (not just different heights) — reflecting more complex, interaction-driven risk profiles. This flexibility is valuable when the underlying biology suggests patients may have qualitatively different risk tra-

jectories (e.g., an early-peaking risk for one subgroup versus a steadily accumulating risk for another) that a single shared baseline hazard, merely rescaled, cannot represent.

3.12.1 Example Result — DeepSurv vs. Cox Discrimination and Calibration

Model	C-index (test set)	Calibration slope (ideal = 1.0)
Cox PH	0.71	0.97
DeepSurv	0.79	0.85

Interpretation: DeepSurv’s higher C-index indicates better discrimination (ranking patients by risk), but its calibration slope further from 1.0 indicates its raw predicted probabilities are somewhat overconfident (too extreme) relative to observed outcomes — a common pattern for flexible models and a reminder that discrimination and calibration are two distinct properties that must both be reported; a recalibration step (e.g., Platt scaling or isotonic regression on the validation set) is standard practice before deploying a deep survival model’s absolute risk estimates clinically.

3.12.2 Code (Python — pycox)

```
from pycox.models import CoxPH
import torch.nn as nn

net = nn.LazyLinear(32)
out_features=1, batch_norm=True, dropout=0.1)

model = CoxPH(net, nn.LazyLinear(32))
model.fit(X_train, (time_train, event_train), epochs=200, batch_size=256)
surv = model.predict_surv_df(X_test)
```

3.13 Multimodal Survival Models (Image + Omics + EHR Fusion)

Model Description: Survival models that combine imaging, omics, and EHR inputs — each passed through a modality-specific encoder as in Chapter 5’s multimodal fusion architecture — feeding into a shared risk score used in a Cox, DeepSurv, or DeepHit-style survival loss.

Mathematical Notation: Extending the fusion architecture from Chapter 5:

$$h(t \mid \text{img, omics, ehr}) = h_0(t) \exp(f_\theta(\text{Fusion}(z_{\text{img}}, z_{\text{omics}}, z_{\text{ehr}})))$$

where $z_{\text{img}}, z_{\text{omics}}, z_{\text{ehr}}$ are modality-specific embeddings and the fusion layer's output feeds a survival-specific loss (Cox partial likelihood or a DeepHit-style discrete-time loss).

Assumptions: All modalities are available (or appropriately imputed/handled for missingness, as discussed in Chapter 5) within a clinically meaningful time window relative to the survival outcome; the combined model still assumes non-informative censoring; the same patient-level train/test splitting integrity required of any multimodal model (Chapter 5) applies with particular force here, since leaking a single patient's imaging or omics data across the train/test boundary can trivially and invisibly inflate a survival model's apparent C-index.

Advantages: Can integrate complementary prognostic information across modalities (e.g., tumor imaging phenotype, molecular subtype from omics, and comorbidity burden from EHR) into a single risk score, often outperforming any single modality; foundation-model encoders (Chapter 5) reduce the feature-engineering burden per modality even in the survival setting.

Limitations: Requires the rarer and more expensive multi-modal-linked cohorts described in Chapter 5; interpretability is compounded (both “why this modality mattered” and “why this specific risk score” require explanation); calibration and discrimination must be validated per modality-combination to justify the added complexity over a simpler EHR-only or omics-only survival model.

Data Requirements: Time-to-event outcome and censoring indicator, plus imaging, omics, and structured EHR data for each patient (or a documented, principled strategy for missing modalities).

Interpretation Guidance: Benchmark the fused survival model's C-index and calibration against each unimodal survival model (imaging-only, omics-only, EHR-only) exactly as in Chapter 5's unimodal-vs-fused benchmarking table, and require a statistically meaningful C-index improvement (e.g., via a bootstrap comparison) before adopting the added complexity of multimodal fusion for a survival endpoint specifically.

3.13.1 Example Result — Multimodal Survival Benchmark (Simulated Oncology Cohort)

Model	C-index	95% CI
EHR only (Cox)	0.68	(0.62, 0.74)

Model	C-index	95% CI
Imaging (radiomics) only	0.66	(0.60, 0.72)
Omics (molecular subtype) only	0.70	(0.64, 0.76)
Multimodal fusion (EHR + imaging + omics, DeepSurv)	0.79	(0.74, 0.84)

Interpretation: As in Chapter 5’s general multimodal benchmarking example, no single modality alone approaches the fused model’s discrimination, and the magnitude of improvement (C-index 0.79 vs. the best unimodal 0.70) is large enough to justify the substantially greater data-collection and modeling complexity multimodal fusion requires — the kind of quantitative justification that should always accompany a decision to pursue multimodal survival modeling rather than a simpler unimodal alternative.

3.13.2 Code (Python — conceptual multimodal DeepSurv)

```
import torch.nn as nn
from pycox.models import CoxPH

class MultimodalSurvivalNet(nn.Module):
    def __init__(self, img_dim, omics_dim, ehr_dim, hidden=64):
        super().__init__()
        self.img_enc = nn.Sequential(nn.Linear(img_dim, hidden), nn.ReLU())
        self.omics_enc = nn.Sequential(nn.Linear(omics_dim, hidden), nn.ReLU())
        self.ehr_enc = nn.Sequential(nn.Linear(ehr_dim, hidden), nn.ReLU())
        self.risk_head = nn.Linear(hidden*3, 1)
    def forward(self, img, omics, ehr):
        fused = torch.cat([self.img_enc(img), self.omics_enc(omics),
        ↪ self.ehr_enc(ehr)], dim=1)
        return self.risk_head(fused) # log-risk score, fed into Cox partial
        ↪ likelihood
```

3.14 Chapter Summary: Choosing a Survival Model

Scenario	Recommended Model
Simple, descriptive group comparison	Kaplan-Meier + log-rank test
Covariate-adjusted hazard ratio, standard trial	Cox proportional hazards
Need absolute survival estimates / extrapolation (health economics)	Parametric model (Weibull/log-normal, selected via AIC)
Intuitive “time extended by X%” communication	AFT model
Multiple possible event types (e.g., relapse vs. death)	Fine-Gray competing risks
Repeated/recurring events per subject	Andersen-Gill, PWP, or WLW
Multi-site or clustered/repeated data with unexplained heterogeneity	Frailty model
Longitudinal biomarker informing event risk dynamically	Joint model
Very small sample / rare disease / need probability statements	Bayesian survival model
High-dimensional tabular covariates, nonlinear risk suspected	RSF / XGBoost-Survival / LightGBM-Survival
Very large sample, raw image/sequence/long-EHR input	Deep learning survival model (DeepSurv/DeepHit/Nnet-survival)
Multiple linked data modalities (image + omics + EHR)	Multimodal fusion survival model

3.15 References

Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457-481.

Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187-220.

Klein, J. P., & Moeschberger, M. L. (2003). *Survival analysis: Techniques for censored and truncated data* (2nd ed.). Springer.

Kalbfleisch, J. D., & Prentice, R. L. (2002). *The statistical analysis of failure time data* (2nd ed.). Wiley.

Fine, J. P., & Gray, R. J. (1999). A proportional hazards model for the subdistribution of a competing risk. *Journal of the American Statistical Association*, 94(446), 496-509.

Andersen, P. K., & Gill, R. D. (1982). Cox's regression model for counting processes: A large sample study. *Annals of Statistics*, 10(4), 1100-1120.

Prentice, R. L., Williams, B. J., & Peterson, A. V. (1981). On the regression analysis of multivariate failure time data. *Biometrika*, 68(2), 373-379.

Wei, L. J., Lin, D. Y., & Weissfeld, L. (1989). Regression analysis of multivariate incomplete failure time data by modeling marginal distributions. *Journal of the American Statistical Association*, 84(408), 1065-1073.

Vaupel, J. W., Manton, K. G., & Stallard, E. (1979). The impact of heterogeneity in individual frailty on the dynamics of mortality. *Demography*, 16(3), 439-454.

Hougaard, P. (2000). *Analysis of multivariate survival data*. Springer.

Tsiatis, A. A., & Davidian, M. (2004). Joint modeling of longitudinal and time-to-event data: An overview. *Statistica Sinica*, 14(3), 809-834.

Rizopoulos, D. (2012). *Joint models for longitudinal and time-to-event data: With applications in R*. CRC Press.

Ibrahim, J. G., Chen, M. H., & Sinha, D. (2001). *Bayesian survival analysis*. Springer.

Ishwaran, H., Kogalur, U. B., Blackstone, E. H., & Lauer, M. S. (2008). Random survival forests. *Annals of Applied Statistics*, 2(3), 841-860.

Katzman, J. L., Shaham, U., Cloninger, A., Bates, J., Jiang, T., & Kluger, Y. (2018). DeepSurv: Personalized treatment recommender system using a Cox proportional hazards deep neural network. *BMC Medical Research Methodology*, 18, 24.

Lee, C., Zame, W. R., Yoon, J., & van der Schaar, M. (2018). DeepHit: A deep learning approach to survival analysis with competing risks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).

Gensheimer, M. F., & Narasimhan, B. (2019). A scalable discrete-time survival model for neural networks. *PeerJ*, 7, e6257.

Harrell, F. E., Lee, K. L., & Mark, D. B. (1996). Multivariable prognostic models: Issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors. *Statistics in Medicine*, 15(4), 361-387. ## Sample Questions

- **Q: What is the proportional hazards assumption and how do you test it?** A: It assumes the hazard ratio between groups is constant over time; test via Schoenfeld residuals (`cox.zph()` in R) — a significant trend indicates a time-varying effect, which may require a time-interaction term

or stratification.

4 Artificial Intelligence and Machine Learning

4.1 Predictive Modeling — Foundations

4.1.1 Bias-Variance Tradeoff

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

4.1.2 Loss Functions

- Regression (MSE): $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- Classification (log-loss / cross-entropy): $L = -\frac{1}{n} \sum [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$

4.2 Ensemble Methods

4.2.1 Random Forest (Breiman, 2001)

Aggregates B bootstrap-trained decision trees:

$$\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

4.2.2 Gradient Boosting (XGBoost, Chen & Guestrin, 2016; LightGBM, Ke et al., 2017)

Sequentially fits residuals:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x), \quad h_m = \arg \min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i))$$

XGBoost adds regularization to the objective:

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

4.2.3 Code (Python — XGBoost)

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)
model = xgb.XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8)
model.fit(X_train, y_train)
```

4.2.4 Code (R — LightGBM style via lightgbm pkg)

```
library(lightgbm)
dtrain <- lgb.Dataset(as.matrix(X_train), label = y_train)
params <- list(objective = "binary", metric = "auc", learning_rate = 0.05,
    ↪ max_depth = 4)
model <- lgb.train(params, dtrain, nrounds = 300)
```

4.2.5 Choosing Between Ensembles and Deep Learning — A Practical Decision Rule

For most genomic/omics biomarker work, sample sizes are in the **hundreds, not tens of thousands** — this strongly favors tree-based ensembles (XGBoost, Random Forest, LightGBM) over deep learning, since neural networks generally need substantially more data to avoid overfitting and to actually outperform simpler models. A practical workflow: start with an ensemble comparison — test XGBoost, Random Forest, and LightGBM against each other under identical cross-validation folds before committing to one — and reserve a neural network for cases where the sample size is much larger, or where the data has a structure (sequence, image, or dense longitudinal data) that deep architectures are specifically suited to exploit (Module 8.9).

4.2.6 Preventing Overfitting on High-Dimensional Genomic Data (p » n) — A Layered Checklist

1. **Dimensionality reduction / feature selection before modeling** — e.g., use co-expression modules (WGCNA) rather than tens of thousands of individual genes as raw features.
2. **Regularization** — LASSO or Elastic Net (Module 3.5) shrinks less-informative coefficients toward zero.
3. **Nested cross-validation** — feature selection and hyperparameter tuning happen *inside* the training folds only, never leaking information from the held-out fold (a single train-test split is not sufficient at small n).

4. **External validation on an independent cohort** whenever possible — internal cross-validation alone, no matter how carefully nested, cannot substitute for a genuinely held-out cohort evaluated on a model that was locked before validation.

4.2.7 Career Application

Q: How do you choose between XGBoost, Random Forest, and a neural network for a biomarker prediction task? *Model answer:* With typical omics sample sizes in the hundreds rather than tens of thousands, tree-based ensembles are usually the right starting point — neural networks need far more data to avoid overfitting and to beat simpler models at that scale. A practical approach is to run an ensemble comparison (XGBoost, Random Forest, LightGBM) under identical cross-validation folds first, and only reach for a neural network if the sample size is much larger or the data structure (sequence/image) specifically favors it.

4.3 Deep Learning

4.3.1 CNN (for imaging/sequence motifs)

Convolution operation: $(f * g)(t) = \sum_{\tau} f(\tau)g(t - \tau)$

4.3.2 RNN/LSTM (Hochreiter & Schmidhuber, 1997) (for sequential/longitudinal data)

LSTM gating equations:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

4.3.3 Code (Python — PyTorch CNN skeleton)

```
import torch.nn as nn
class GenomicCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(4, 32, kernel_size=8) # 4 = one-hot DNA bases
        self.pool = nn.MaxPool1d(4)
        self.fc = nn.Linear(32 * ((1000-8+1)//4), 1)
    def forward(self, x):
```

```
x = self.pool(torch.relu(self.conv1(x)))
x = x.flatten(1)
return torch.sigmoid(self.fc(x))
```

4.3.4 Pitfalls

- Deep learning requires large sample sizes; with omics data (n often <1000), simpler models (elastic net, RF) often outperform DL — reserve DL for large-scale imaging/sequence data or pretrained transfer-learning contexts.

4.4 Explainable AI (SHAP, LIME)

Deep learning models are inherently black boxes: a trained network is, at its core, millions of learned weights with no built-in mechanism for stating *why* it produced a given output. We make these models explainable after the fact by applying post-hoc interpretability tools — **SHAP**, **LIME**, **Integrated Gradients**, **Grad-CAM**, **attention maps**, **surrogate models**, and **TCAV** — each of which reveals, in a different way, which features, pixels, genes, or clinical variables drove a specific prediction. This matters for three concrete reasons: it lets scientists validate that the model is picking up genuine biological signal rather than a spurious shortcut; it supports regulatory compliance, since agencies increasingly expect a documented rationale for a model’s clinical output; and it builds the trust clinicians and collaborators need before acting on a model’s recommendation.

- **SHAP** — Shapley-value-based attribution (below); best for tabular/tree-based models, gives globally consistent, additive feature attributions.
- **LIME** (Ribeiro, Singh, & Guestrin, 2016) — fits a local linear surrogate around a single prediction; model-agnostic and fast, but less theoretically grounded and can be unstable across repeated runs.
- **Integrated Gradients** (Sundararajan, Taly, & Yan, 2017) — for differentiable models (neural networks): integrates the gradient of the output with respect to the input along a path from a baseline (e.g., a blank image or all-zero vector) to the actual input, attributing importance to each input feature/pixel.
- **Grad-CAM** (Selvaraju et al., 2017) — for CNNs specifically: uses the gradients flowing into the final convolutional layer to produce a coarse heatmap highlighting which regions of an image most influenced the prediction (e.g., which part of a chest X-ray drove a pneumonia classification).
- **Attention maps** — for transformer-based models: the learned attention weights themselves can be visualized to show which tokens/positions

(words, genomic positions, time steps) the model weighted most heavily when producing its output.

- **Surrogate models** — train a simple, fully interpretable model (a shallow decision tree or sparse linear model) to approximate the black-box model’s predictions, trading some fidelity for a globally readable summary of its behavior.
- **TCAV (Testing with Concept Activation Vectors)** — tests whether a human-defined high-level concept (e.g., “presence of ground-glass opacity,” or “high GC-content region”) is linearly represented in a network’s internal activations, and how much that concept influences the prediction — useful when the explanation needed is a clinical concept, not just a raw pixel or feature.

4.4.1 SHAP (Shapley values; Lundberg & Lee, 2017)

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

(the average marginal contribution of feature i across all feature coalitions)

Interpretation: the average marginal contribution of feature i across all possible feature coalitions.

Interpretation: This is what SHAP looks like applied to one individual, rather than averaged across a whole cohort. The base rate (0.32) represents the average predicted risk across all patients before considering anything specific about this one; each subsequent bar shows how much a specific feature value moves this particular patient’s prediction away from that average. Here, high blood eosinophils and two prior exacerbations both push the risk substantially upward, a low CpG methylation value contributes a smaller additional push upward, while this patient’s age contributes slightly negatively (i.e., protective relative to the average patient). This is precisely the individual-level explanation a clinician needs — not “the model is 80% confident this patient will have an exacerbation,” but “these three specific, clinically checkable factors are what’s driving that number for this patient.”

4.4.2 Example Result — Concordance Between SHAP and Known Clinical Risk Factors

Feature	Mean	SHAP value
Prior exacerbations (past year)	0.29	Yes — well-established

Feature	Mean	SHAP value
Blood eosinophil count	0.24	Yes — well-established (T2-high endotype marker)
Baseline FEV1 % predicted	0.19	Yes — well-established
CpG cg0142 methylation	0.11	No — candidate biomarker under investigation
Age	0.06	Partial — weak, inconsistent literature support

How this is obtained: Mean absolute SHAP value is computed across all patients in the validation cohort for each feature, then features are ranked and cross-checked against the existing clinical literature on asthma exacerbation risk factors.

Interpretation: The fact that the top three model-driven features by SHAP importance are all independently well-established clinical risk factors is a form of face-validity check: it suggests the model has learned genuine, clinically recognizable signal rather than an artifact of the training data. The fourth feature (a candidate methylation biomarker) is the novel, hypothesis-generating result — its comparatively lower but still meaningful SHAP importance, alongside its absence from prior clinical literature, is exactly the pattern that would justify further validation (independent replication, mediation analysis) before treating it as a confirmed biomarker rather than a promising signal the model surfaced.

4.4.3 Code (Python)

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

4.4.4 Pitfalls

- SHAP values reflect model behavior, not ground-truth causality — high SHAP importance $\neq$ causal driver (pair with MR/mediation, Module 2.6).
- Correlated features split SHAP importance among them, diluting apparent effect of each.

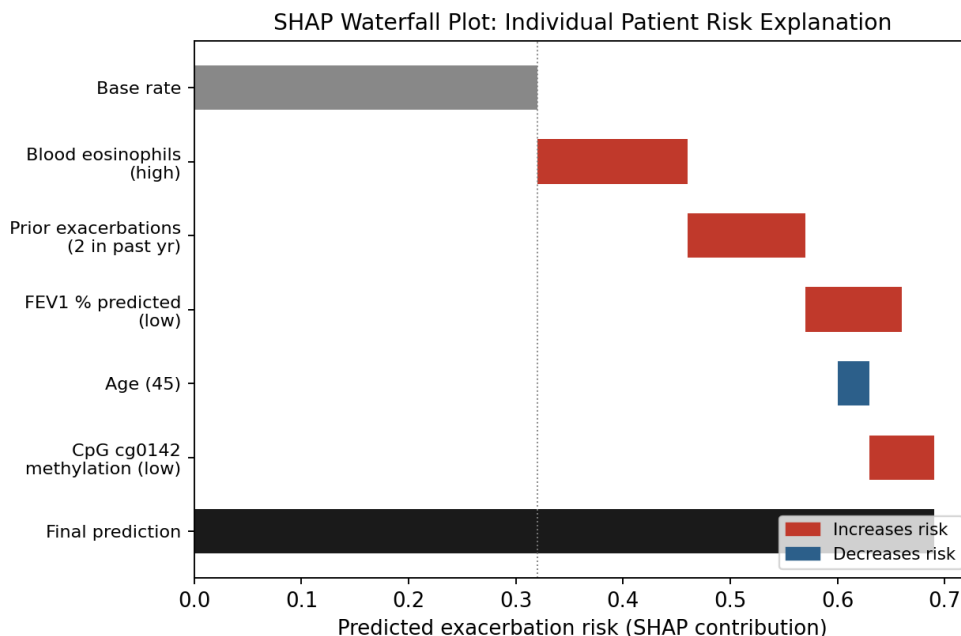


Figure 4.1: **SHAP waterfall plot for a single patient’s exacerbation-risk prediction.** Starting from the model’s base rate (grey bar), each subsequent bar shows how one feature pushes the predicted risk up (red) or down (blue), ending at the patient’s final predicted risk score.

4.5 Large Language Models (LLMs) for Biological Data

4.5.1 Applications

- Literature mining/summarization (PubMed abstract synthesis).
- Protein language models (ESM, ProtBERT) for structure/function prediction.
- Clinical note NLP (named entity recognition for phenotyping from EHR).

4.5.2 Pitfalls

- Hallucination risk in biomedical fact generation — always require citation-grounded outputs and human expert verification before clinical use.
- Data leakage: verify LLM training cutoff doesn’t overlap with your “held-out” benchmark.

4.6 Causal Inference & Mediation Modeling

4.6.1 Mediation Model

$$M = aX + \epsilon_1, \quad Y = c'X + bM + \epsilon_2$$

Indirect (mediated) effect: ab ; direct effect: c' ; total effect: $c = c' + ab$.

4.6.2 Causal Frameworks (relevant to your CIMLA/TRACE work)

- **Structural Causal Models (SCMs)** with do-calculus for interventions: $P(Y|do(X = x))$.
- **Mediation analysis (R: mediation package)** for genotype -> methylation -> expression -> phenotype chains.

4.6.3 Code (R)

```
library(mediation)
med_fit <- lm(methylation ~ genotype, data = df)
out_fit <- lm(expression ~ genotype + methylation, data = df)
med_result <- mediate(med_fit, out_fit, treat = "genotype", mediator =
  ↪ "methylation",
                      boot = TRUE, sims = 1000)
summary(med_result)
```

4.6.4 Pitfalls

- Mediation analysis assumes no unmeasured confounding between mediator and outcome — a strong and often untestable assumption; sensitivity analysis (`medsens()`) is essential.

4.6.5 Sensitivity Analysis for Causal Claims — The Key Assumption Reviewers Probe

The assumption underlying both mediation analysis and most observational causal-effect estimation is **sequential ignorability**: no unmeasured confounding of the exposure-mediator, exposure-outcome, or mediator-outcome relationships. This assumption is *untestable* with the observed data alone — which is exactly why technical reviewers at Big Pharma/biotech routinely follow up a mediation or causal claim with “how would you know if that assumption were violated?” A strong answer names concrete diagnostics, not just the assumption itself:

1. E-value sensitivity analysis (VanderWeele & Ding, 2017) — quantifies how strong an unmeasured confounder would need to be (in terms of its association with both exposure and outcome) to fully explain away an observed effect, rather than requiring the confounder to be measured.

$$E\text{-value} = RR_{obs} + \sqrt{RR_{obs} \times (RR_{obs} - 1)}$$

(applies for $RR_{obs} \geq 1$; use $1/RR_{obs}$ first if $RR_{obs} < 1$)

For a continuous effect estimate, convert to an approximate risk ratio first (e.g., via $RR \approx \exp(0.91 \times \beta)$ for a standardized linear effect) before applying the formula. A **low E-value** (close to 1) means even a weak unmeasured confounder could explain the association away — a red flag; a **high E-value** means the finding is robust to all but an implausibly strong confounder.

```
library(EValue)
evaluates.RR(est = 1.8, lo = 1.3, hi = 2.5) # point estimate + 95% CI bounds
```

2. Negative control outcomes — test the same exposure/mediator against an outcome it *should not* plausibly affect (biologically unrelated). If the “effect” shows up there too, it suggests the pipeline is picking up residual confounding or technical artifact rather than the proposed mechanism, rather than genuine mediation.

3. Downgrading causal language proportionally to sensitivity results — rather than a binary “causal / not causal” claim, report the E-value and negative-control result alongside the effect estimate, and calibrate the strength of causal language (e.g., “consistent with,” “suggestive of,” vs. “demonstrates”) to what the sensitivity analysis actually supports.

4. Escalating to a genetic instrument (Mendelian Randomization, Module 1.8) when sequential ignorability is specifically doubted for a proposed mediator (e.g., a metabolite or protein level plausibly confounded with the outcome) — MR’s own assumptions (relevance, independence, exclusion restriction / no horizontal pleiotropy) must then be checked with equal rigor (MR-Egger intercept test, as in Module 1.8), so switching frameworks trades one set of assumptions for another rather than eliminating the need for sensitivity analysis altogether.

4.6.6 Summary Table — Sensitivity Diagnostics for Causal Claims

Diagnostic	What It Tests	Red Flag	Tool
E-value	Confounder strength needed to nullify the effect	E-value near 1.0	<code>evaluates.RR()</code>
Negative control outcome	Whether the pipeline detects false “effects”	Effect on an implausible outcome	Custom design
MR-Egger intercept	Horizontal pleiotropy in an MR instrument	Significant non-zero intercept	<code>mr_pleiotropy_test()</code>
<code>medsens()</code>	Sensitivity of indirect effect to residual confounding (ρ)	Effect nullified at small ρ	R <code>mediation</code> package

4.6.7 Career Application

Q: If the sequential ignorability assumption behind your mediation framework were violated, how would you actually know, and what would you do? *Model answer:* You wouldn’t know for certain — that’s the nature of an untestable assumption — but there are concrete warning signs: an implausibly low E-value (meaning even a weak unmeasured confounder could explain away the effect), or a negative-control outcome that unexpectedly shows the same “effect,” suggesting the pipeline is picking up something other than the proposed mechanism. If either appears, the appropriate response is to downgrade the causal language used to report the finding, and where feasible, pursue a genetic instrument via Mendelian Randomization to obtain an estimate less dependent on the confounding assumption.

4.7 Model Evaluation & Validation

4.7.1 Cross-Validation

k -fold CV error: $CV_{(k)} = \frac{1}{k} \sum_{i=1}^k \text{Error}_i$

4.7.2 Summary Table — Metrics by Task

Task	Key Metrics
Binary classification	AUC-ROC, AUC-PR, Sensitivity, Specificity, F1
Regression	RMSE, MAE, R^2
Survival	C-index, time-dependent AUC, Brier score
Calibration	Calibration slope/intercept, Hosmer-Lemeshow

4.8 Graph Neural Networks

4.8.1 Conceptual Foundation

Extend deep learning to graph-structured data (PPI networks, molecular graphs) via message passing:

$$h_v^{(k)} = \text{UPDATE} \left(h_v^{(k-1)}, \text{AGGREGATE} \left(\{h_u^{(k-1)} : u \in N(v)\} \right) \right)$$

4.8.2 Applications

Drug-target interaction prediction, gene regulatory network inference, protein structure (AlphaFold's evoformer uses attention over graph-like representations).

4.9 scRNA-seq Analysis

Interpretation: The pipeline diagram makes clear that clustering and UMAP visualization — often the first thing people look at — actually sit near the *end* of a long chain of preprocessing decisions, each of which can change the final picture substantially. A poorly chosen QC threshold (too lenient) lets damaged/dying cells with high mitochondrial content masquerade as a distinct population; skipping batch integration (highlighted) when combining multiple 10x runs risks creating clusters that reflect which day a sample was processed rather than genuine cell biology, as illustrated in the UMAP figure below.

4.9.1 Workflow (Seurat/Scanpy)

```
Raw counts (10x Genomics) -> QC (min genes/cell, %mito filter)
    -> Normalization (log-normalize / SCTransform)
    -> Highly variable gene selection
```

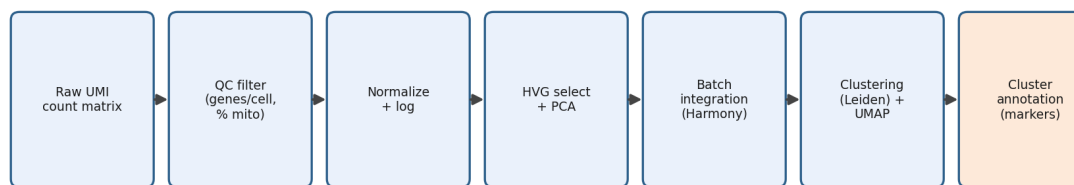


Figure 4.2: **The scRNA-seq analysis pipeline, raw counts to annotated clusters.** Each stage progressively cleans, reduces, and structures the data until biologically meaningful cell populations emerge; batch integration (highlighted) is essential whenever more than one sample or sequencing run is combined.

- > Dimensionality reduction (PCA -> UMAP/t-SNE)
- > Clustering (Louvain/Leiden on shared-nearest-neighbor graph)
- > Cluster annotation (marker genes, reference mapping)
- > Differential expression between clusters/conditions
- > Trajectory inference (Monocle3, Slingshot) if applicable

Interpretation: Each colored group in this UMAP corresponds to a cluster identified by Leiden clustering on the shared-nearest-neighbor graph, subsequently labeled using canonical marker genes (e.g., *FOXJ1* for ciliated cells, *MUC5AC* for goblet cells, *KRT5* for basal cells). The fact that cells from what would be multiple different patient samples mix smoothly within each colored group — rather than each patient forming its own separate island — is the visual signature of successful batch integration; had Harmony not been applied, or had it been applied with poorly chosen parameters, patient identity rather than cell type would likely have been the dominant separating structure in the plot.

4.9.2 Example Result — Cluster Marker Gene Table

Cluster	Top Marker Genes	Assigned Cell Type	Cells (n)	% of Total
0	<i>FOXJ1</i> , <i>TUBB4B</i> , <i>DNAI1</i>	Ciliated	412	28%
1	<i>MUC5AC</i> , <i>MUC5B</i> , <i>SPDEF</i>	Goblet	298	20%

Cluster	Top Marker Genes	Assigned Cell Type	Cells (n)	% of Total
2	<i>KRT5</i> , <i>TP63</i> , <i>KRT14</i>	Basal	356	24%
3	<i>SCGB1A1</i> , <i>SCGB3A1</i>	Club cells	201	14%
4	<i>PTPRC</i> , <i>CD3E</i> , <i>CD68</i>	Immune (mixed)	210	14%

How this is obtained: For each Leiden cluster, `FindAllMarkers()` (Seurat) or `sc.tl.rank_genes_groups()` (Scanpy) identifies genes most differentially expressed in that cluster versus all others; the top 2-3 genes per cluster are then matched against known canonical markers to assign a cell-type label.

Interpretation: A goblet-cell proportion of 20% would be compared against a healthy reference range for pediatric airway epithelium; a proportion notably elevated relative to healthy controls is consistent with goblet cell hyperplasia and mucus hypersecretion, a recognized feature of type-2-high asthma. This is the step where scRNA-seq moves from a purely descriptive cell atlas to a disease-relevant finding — reporting compositional shifts between conditions, not just which cell types exist.

4.9.3 Code (R — Seurat)

```
library(Seurat) # Hao et al., 2021
obj <- CreateSeuratObject(counts = raw_counts, min.cells = 3, min.features = 200)
obj[["percent.mt"]] <- PercentageFeatureSet(obj, pattern = "^MT-")
obj <- subset(obj, subset = nFeature_RNA > 200 & percent.mt < 10)
obj <- NormalizeData(obj) |> FindVariableFeatures() |> ScaleData()
obj <- RunPCA(obj) |> RunUMAP(dims = 1:20) |> FindNeighbors(dims = 1:20) |>
  FindClusters()
```

4.9.4 Code (Python — Scanpy)

```
import scanpy as sc # Wolf, Angerer, & Theis, 2018
adata = sc.read_10x_mtx("filtered_feature_bc_matrix/")
sc.pp.filter_cells(adata, min_genes=200)
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=2000)
sc.tl.pca(adata); sc.pp.neighbors(adata); sc.tl.leiden(adata); sc.tl.umap(adata)
```

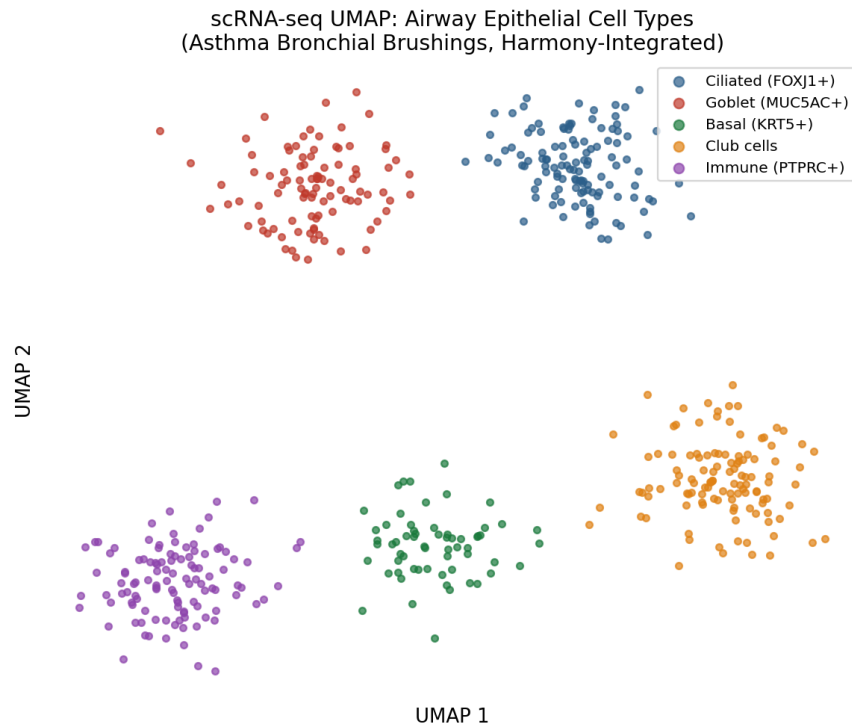


Figure 4.3: **Example UMAP embedding after Harmony batch integration.** Simulated single-cell transcriptomes from pediatric airway epithelial brushings, colored by annotated cell type. Clear separation between ciliated, goblet, basal, club, and immune cell populations indicates that clustering is being driven by cell identity rather than by batch/sample of origin.

4.9.5 Pitfalls

- Ambient RNA contamination and doublets inflate false “hybrid” cell types — use SoupX/DecontX and Scrublet/DoubletFinder.
- Batch effects across 10x runs require integration (Harmony, Seurat CCA, scVI).

4.10 References

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146-3154).

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998-6008).

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765-4774).

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135-1144).

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 618-626).

Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 3319-3328).

Imai, K., Keele, L., & Tingley, D. (2010). A general approach to causal mediation analysis. *Psychological Methods*, 15(4), 309-334.

VanderWeele, T. J., & Ding, P. (2017). Sensitivity analysis in observational research: Introducing the E-value. *Annals of Internal Medicine*, 167(4), 268-274.

Bowden, J., Davey Smith, G., & Burgess, S. (2015). Mendelian randomization with invalid instruments: Effect estimation and bias detection through Egger regression. *International Journal of Epidemiology*, 44(2), 512-525.

Wolf, F. A., Angerer, P., & Theis, F. J. (2018). SCANPY: Large-scale single-cell gene expression data analysis. *Genome Biology*, 19, 15.

Hao, Y., Hao, S., Andersen-Nissen, E., Mauck, W. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zager, M., Hoffman, P., Stoeckius, M., Papalexi, E., Mimitou, E. P., Jain, J., Srivastava, A., Stuart, T., Fleming, L. M., Yeung, B., ... Satija, R. (2021). Integrated analysis of multimodal single-cell data. *Cell*, 184(13), 3573-3587.

Korsunsky, I., Millard, N., Fan, J., Slowikowski, K., Zhang, F., Wei, K., Baglaenko, Y., Brenner, M., Loh, P. R., & Raychaudhuri, S. (2019). Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods*, 16(12), 1289-1296.

Lopez, R., Regier, J., Cole, M. B., Jordan, M. I., & Yosef, N. (2018). Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15(12), 1053-1058. ## Sample Questions

Q: How do SHAP and LIME differ, and when would you use each?
Model answer: Both are post-hoc, model-agnostic explanation methods, but they differ in theoretical grounding and mechanics. **SHAP** is grounded in cooperative game theory (Shapley values) and provides a mathematically consistent, additive attribution of each feature's contribution across all possible feature coalitions — it guarantees local accuracy (attributions sum to the prediction) and consistency (a feature's SHAP value can't decrease if its marginal contribution increases). **LIME** approximates the model locally with an interpretable surrogate (e.g., sparse linear model) fit on perturbed samples around the instance of interest — it's faster and more intuitive but less theoretically grounded and can be unstable (different runs can give different explanations depending on the perturbation sampling). In practice, I use **TreeExplainer (SHAP)** for tree ensembles (XGBoost/RF/LightGBM) because it's exact and fast, and I'd reach for LIME mainly for quick, model-agnostic sanity checks or when SHAP is computationally infeasible (e.g., very large deep models without a fast SHAP approximation).

Q: What’s a common pitfall when interpreting SHAP values in a biomarker discovery context? *Model answer:* High SHAP importance reflects the model’s reliance on a feature, not necessarily a causal biological driver — correlated features (e.g., co-methylated CpGs, genes in the same pathway) split importance among themselves, which can make a true causal driver look less important than it is. I always pair SHAP-based prioritization with orthogonal evidence (independent replication, pathway/functional annotation, or causal methods like Mendelian Randomization/mediation) before claiming biological significance.

Ensemble Methods (XGBoost, LightGBM, Random Forest)

Q: Compare Random Forest, XGBoost, and LightGBM. When would you choose one over the others?

Method	Core Mechanism	Strengths	When to Prefer
Random Forest	Bagging — independent trees averaged	Robust to overfitting, easy to tune, handles noisy features well	Baseline model, smaller/moderate datasets, when interpretability via feature importance suffices
XGBoost	Gradient boosting with regularized objective	High accuracy, handles missing values natively, strong regularization (L1/L2)	Tabular/omics data with moderate size, competitions, when squeezing out accuracy matters
LightGBM	Gradient boosting with histogram-based, leaf-wise growth	Very fast on large datasets, lower memory	Very large datasets (millions of rows), many categorical features

Model answer add-on: “For a typical omics biomarker dataset (n in the hundreds to low thousands, p in the thousands), I’d usually start with Elastic Net as an interpretable linear baseline, then try Random Forest/XGBoost for potential nonlinear interactions, always validating with nested cross-validation given the $p \gg n$ regime, and use SHAP for interpretability regardless of which ensemble wins.”

scRNA-seq (Seurat, Scanpy)

Q: Walk me through your scRNA-seq analysis pipeline and how you'd decide on the number of clusters. *Model answer* (mirrors Module 2.9): I'd describe QC (min genes/cell, %mitochondrial reads, doublet removal), normalization, HVG selection, PCA for dimensionality reduction, then graph-based clustering (Louvain/Leiden) on a shared-nearest-neighbor graph, followed by UMAP for visualization. For the number of clusters, I don't pre-specify k as in k -means; instead the Leiden/Louvain resolution parameter implicitly controls granularity, and I evaluate cluster stability (e.g., via bootstrapping or clustree) and biological validity (canonical marker gene expression per cluster) rather than choosing purely by an elbow/silhouette metric, since silhouette scores can be misleading on the highly non-Euclidean structure of single-cell graphs.

Q: How do you handle batch effects across multiple 10x runs/samples in scRNA-seq? *Model answer*: Use integration methods designed for single-cell data rather than simple ComBat — **Harmony** (Korsunsky et al., 2019; fast, iterative clustering-based correction), **Seurat's CCA/RPCA integration**, or **scVI** (Lopez, Regier, Cole, Jordan, & Yosef, 2018; a deep generative model, variational autoencoder, that models batch as a covariate in its generative process and is particularly strong for large atlases/cross-study integration). I'd always visualize UMAP before/after integration colored by batch to confirm mixing without erasing true biological (e.g., cell-type or disease) separation.

5 Multimodal Data Integration and Medical Imaging

5.1 Image Preprocessing Pipeline (Raw → Analysis-Ready)

5.1.1 Conceptual Foundation

Medical/biological images (histopathology whole-slide images, radiology CT/MRI, fluorescence microscopy) require a standardized pipeline before modeling, because raw acquisition introduces noise, intensity variation across scanners/sites, and irrelevant background.

5.1.2 Workflow (Raw → Final)

Raw image (DICOM/WSI/TIFF)

- > Format standardization (DICOM -> NIfTI via dcm2niix; WSI tiling via OpenSlide)
- > Denoising (Gaussian/median filter; for MRI: Non-Local Means, Rician noise correction)
- > Bias-field / intensity inhomogeneity correction (N4ITK, Tustison et al., 2010, for MRI)
- > Registration (rigid/affine/deformable; align to template e.g., MNI152 for brain MRI)
- > Normalization (z-score, min-max, or histogram matching across scanners)
 - > Segmentation (organ/tumor/nucleus/cell segmentation: U-Net, nnU-Net (Isensee et al., 2021), Cellpose, StarDist)
- > Feature extraction:
 - Handcrafted: radiomics (shape, texture - GLCM, wavelet features; via PyRadiomics)
 - Learned: CNN/ViT embeddings (transfer learning or foundation models)
- > Quality control (artifact detection, out-of-distribution/outlier flagging)
- > Final analysis-ready feature matrix or tensor for modeling

5.1.3 Code (Python — basic MRI preprocessing with SimpleITK/nibabel)

```
import SimpleITK as sitk
img = sitk.ReadImage("raw_mri.nii.gz")
img_denoised = sitk.CurvatureFlow(image1=img, timeStep=0.125,
    ↪ numberOfIterations=5)
corrector = sitk.N4BiasFieldCorrectionImageFilter()
img_corrected = corrector.Execute(img_denoised)
```

```
img_resampled = sitk.Resample(img_corrected, referenceImage_template)
```

5.1.4 Code (Python — PyRadiomics (van Griethuysen et al., 2017) feature extraction)

```
from radiomics import featureextractor
extractor = featureextractor.RadiomicsFeatureExtractor()
features = extractor.execute("image.nii.gz", "segmentation_mask.nii.gz")
```

5.1.5 Pitfalls

- Scanner/site batch effects in imaging are analogous to sequencing batch effects — apply ComBat-style harmonization (Johnson, Li, & Rabinovic, 2007) (e.g., **ComBat-GAM**, **Neuroharmonize**) across sites in multi-site studies (e.g., ADNI, UK Biobank Imaging).
- Segmentation errors propagate directly into all downstream radiomic/CNN features — always visually QC a random sample of segmentations.

5.2 Multimodal Fusion: Imaging + Omics + EHR

5.2.1 Conceptual Foundation

Multimodal fusion combines heterogeneous data types (imaging, genomics/omics, structured EHR, clinical notes) into a joint model. Three canonical fusion strategies:

- **Early fusion:** concatenate raw/engineered features from all modalities into one vector before a single model.
- **Late fusion:** train separate unimodal models, combine predictions (voting, stacking, weighted average).
- **Intermediate/joint fusion:** learn modality-specific embeddings (e.g., CNN for imaging, autoencoder for omics, transformer for EHR sequences), then fuse in a shared latent space (concatenation, cross-attention, or tensor fusion).

5.2.2 Pipeline (Pseudocode)

```
Imaging  -> CNN/ViT encoder      -> embedding_img  (dim d1)
Omics    -> Autoencoder/PCA     -> embedding_omics (dim d2)
EHR      -> Transformer/RNN     -> embedding_ehr   (dim d3)

-> Fusion layer:
    concat([embedding_img, embedding_omics, embedding_ehr])
    OR cross-attention(embedding_img, embedding_omics, embedding_ehr)
```

- > Joint dense layers -> Prediction head (classification/survival/regression)
- > Loss = task loss + optional modality-alignment/contrastive loss

5.2.3 Formula — Contrastive Alignment Loss (used in multimodal foundation models, e.g., CLIP-style)

$$\mathcal{L}_{contrastive} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_j \exp(\text{sim}(z_i, z_j)/\tau)}$$

where z_i, z_i^+ are paired embeddings from two modalities of the same patient, and τ is a temperature parameter.

5.2.4 State-of-the-Art / Emerging Methods

- **Foundation models for pathology:** UNI, CONCH, Virchow (self-supervised whole-slide image encoders).
- **Multimodal EHR + imaging:** fusion transformers combining structured labs/vitals with chest X-ray embeddings for mortality/readmission prediction.
- **Geneformer / scGPT:** transformer foundation models pretrained on millions of single-cell transcriptomes for transfer learning to new cell-type/disease tasks.
- **AlphaFold2/3, ESM2/ESM3:** protein structure/function foundation models, increasingly used as omics feature extractors.
- **Spatial transcriptomics** (10x Visium, MERFISH, Xenium) integrating gene expression with tissue spatial coordinates and histology images — analyzed with Squidpy, SpaGCN, or BayesSpace.
- **Federated learning:** trains models across multiple hospital EHR systems without centralizing patient-level data — critical for multi-site clinical AI where data-sharing agreements restrict raw data movement.
- **Double Machine Learning (DML) / Targeted Maximum Likelihood Estimation (TMLE):** modern causal-inference estimators that combine flexible ML nuisance-parameter models with valid statistical inference for treatment-effect estimation, increasingly used in place of simple propensity-score matching in real-world evidence (RWE) studies.

5.2.5 Example

- **MIMIC-III/IV** (Johnson et al., 2016) — de-identified ICU EHR (vitals, labs, notes, imaging) from Beth Israel Deaconess — canonical multimodal benchmark.

- **TCGA** (Weinstein et al., 2013) + **TCIA** (Clark et al., 2013) — matched genomics (TCGA) and radiology/pathology imaging (The Cancer Imaging Archive) for the same patients.
- **UK Biobank Imaging** (Bycroft et al., 2018) — brain/cardiac/abdominal MRI linked to genotype and EHR for ~100K participants.
- **ADNI** (Alzheimer’s Disease Neuroimaging Initiative) — MRI/PET + genetics + cognitive/clinical data.
- **GTEx** (GTEx Consortium, 2020) — tissue-specific gene expression across ~50 tissues, foundational for eQTL/multi-tissue reference work.

5.2.6 Code (Python — simple late-fusion skeleton)

```
import torch, torch.nn as nn

class LateFusionModel(nn.Module):
    def __init__(self, img_dim, omics_dim, ehr_dim, hidden=64):
        super().__init__()
        self.img_branch = nn.Sequential(nn.Linear(img_dim, hidden), nn.ReLU())
        self.omics_branch = nn.Sequential(nn.Linear(omics_dim, hidden), nn.ReLU())
        self.ehr_branch = nn.Sequential(nn.Linear(ehr_dim, hidden), nn.ReLU())
        self.head = nn.Linear(hidden*3, 1)

    def forward(self, img_feat, omics_feat, ehr_feat):
        fused = torch.cat([self.img_branch(img_feat),
                           self.omics_branch(omics_feat),
                           self.ehr_branch(ehr_feat)], dim=1)
        return torch.sigmoid(self.head(fused))
```

5.2.7 Pitfalls

- **Missing modalities:** not every patient has imaging + omics + EHR — design models robust to missing modalities (modality dropout during training, or late-fusion with per-modality imputation) rather than dropping incomplete cases outright.
- **Modality imbalance:** a high-dimensional imaging embedding can dominate a smaller omics/clinical vector unless dimensions are balanced or fusion weights are learned/regularized.
- **Data leakage across modalities:** e.g., splitting train/test by scan rather than by patient can leak information if a patient has multiple scans.

5.2.8 Summary Table — Fusion Strategy Selection

Strategy	When to Use	Tradeoff
Early fusion	Modalities are low-dimensional, on comparable scales	Simple but can be dominated by one modality's noise
Late fusion	Modalities are very heterogeneous / different sample availability	Loses cross-modal interactions
Intermediate/joint fusion	Want to learn cross-modal interactions, sufficient data	More complex, needs more data/regularization

5.3 References

Van Griethuysen, J. J. M., Fedorov, A., Parmar, C., Hosny, A., Aucoin, N., Narayan, V., Beets-Tan, R. G. H., Fillion-Robin, J. C., Pieper, S., & Aerts, H. J. W. L. (2017). Computational radiomics system to decode the radiology phenotype. *Cancer Research*, 77(21), e104-e107.

Tustison, N. J., Avants, B. B., Cook, P. A., Zheng, Y., Egan, A., Yushkevich, P. A., & Gee, J. C. (2010). N4ITK: Improved N3 bias correction. *IEEE Transactions on Medical Imaging*, 29(6), 1310-1320.

Isensee, F., Jaeger, P. F., Kohl, S. A. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2), 203-211.

Johnson, W. E., Li, C., & Rabinovic, A. (2007). Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics*, 8(1), 118-127.

Johnson, A. E. W., Pollard, T. J., Shen, L., Lehman, L. H., Feng, M., Ghassemi, M., Moody, B., Szolovits, P., Celi, L. A., & Mark, R. G. (2016). MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3, 160035.

Clark, K., Vendt, B., Smith, K., Freymann, J., Kirby, J., Koppel, P., Moore, S., Phillips, S., Maffitt, D., Pringle, M., Tarbox, L., & Prior, F. (2013). The Cancer Imaging Archive (TCIA): Maintaining and operating a public information repository. *Journal of Digital Imaging*, 26(6), 1045-1057.

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3), 837-845.

Chen, R. J., Chen, C., Li, Y., Chen, T. Y., Trister, A. D., Krishnan, R. G., & Mahmood, F. (2022). Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 16144-16155). ## Sample Questions

Q: Walk me through how you would preprocess raw medical images before feeding them into a predictive model. *Model answer* (mirrors Module 6.1): “I’d start with format standardization (e.g., DICOM to NIfTI), then denoise and correct for scanner-specific intensity artifacts (bias-field correction for MRI), register images to a common template/space so anatomy aligns across subjects, normalize intensities so scanner/site differences don’t dominate signal, segment the region of interest either with a validated deep-learning segmentation model (U-Net/nnU-Net) or an existing clinical segmentation, extract either handcrafted radiomic features or learned CNN/foundation-model embeddings, and finally run quality control to catch failed segmentations or acquisition artifacts before any modeling.” I’d also flag that harmonization across sites (ComBat-style) is essential in any multi-site imaging study to avoid a “batch effect” confound with the outcome of interest.

Q: How would you build a model that integrates imaging, omics, and EHR data to predict a clinical outcome? *Model answer* (mirrors Module 6.2): “I’d first evaluate data availability/overlap across modalities per patient, then choose a fusion strategy based on sample size and modality dimensionality — late fusion if modalities are very heterogeneous or often missing, intermediate/joint fusion with modality-specific encoders if I have enough patients with all modalities to learn cross-modal interactions. I’d encode each modality separately (CNN/ViT for imaging, autoencoder or simple linear model for omics given $p \gg n$, transformer/RNN or even simple feature engineering for EHR time series), fuse in a shared representation, and train jointly with the clinical prediction task, while explicitly handling missing modalities (e.g., modality dropout during training) rather than discarding incomplete patients. I’d validate with SHAP/attention-based interpretability to confirm each modality contributes meaningfully, and check that no single modality (e.g., EHR) is trivially predictive in a way that makes the imaging/omics contribution spurious.”

6 Foundation Models for Computational Biology and Medicine

Foundation models are large neural networks pretrained on broad data at scale using self-supervised objectives, then adapted (via fine-tuning or zero/few-shot prompting) to a wide range of downstream tasks. This chapter surveys the foundation models most relevant to computational biology, genomics, proteomics, drug discovery, and clinical prediction, following a consistent structure: model description, mathematical notation, example data, example outputs, interpretation, assumptions, advantages, limitations, data requirements, and use-cases.

6.1 Shared Mathematical Foundations

Before covering individual models, it is worth stating the mathematical building blocks nearly all of them share.

Embedding function. Any foundation model first maps a raw input (a token, an image patch, a DNA k-mer) into a continuous vector representation:

$$h = f(x; \theta)$$

where x is the raw input, θ the model's learned parameters, and $h \in \mathbb{R}^d$ the resulting embedding. Everything downstream - attention, fine-tuning, similarity search - operates on these embeddings rather than the raw input.

Self-attention. The core computation inside a transformer block, allowing every position in a sequence to weigh every other position when constructing its own representation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$ are learned linear projections of the input X , and d_k is the key dimension (the $\sqrt{d_k}$ scaling keeps the softmax's gradients well behaved as d_k grows).

Transformer block. A full transformer layer combines self-attention with a position-wise feed-forward network, each wrapped in a residual connection

and layer normalization:

$$X' = \text{LayerNorm}(X + \text{Attention}(Q, K, V)), \quad X'' = \text{LayerNorm}(X' + \text{FFN}(X'))$$

Masked-token (self-supervised) pretraining loss. Most foundation models covered in this chapter are pretrained without labels, by masking part of the input and predicting it from context:

$$\mathcal{L} = - \sum_{i \in M} \log p(x_i \mid x_{\setminus M})$$

where M is the set of masked positions and $x_{\setminus M}$ is the unmasked context. This single objective – applied to words, DNA bases, amino acids, or image patches – is the mechanism that lets a model learn broadly useful representations from unlabeled data at massive scale, before any task-specific fine-tuning occurs.

Contrastive alignment loss (for multimodal models pairing two modalities, e.g., image and text):

$$\mathcal{L}_{\text{contrastive}} = - \log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_j \exp(\text{sim}(z_i, z_j)/\tau)}$$

which pulls matched pairs' embeddings together and pushes mismatched pairs apart, as introduced in Chapter 5's multimodal fusion discussion.

6.2 GPT-Style Language Models (LLMs)

Model Description: Autoregressive transformer decoders (Radford et al., 2018) trained to predict the next token given all previous tokens, scaled to billions of parameters and trained on broad text corpora; in biomedicine, adapted for clinical note summarization, literature synthesis, and increasingly as a general-purpose reasoning layer over structured and unstructured biomedical data.

Mathematical Notation: Autoregressive factorization of a sequence's probability:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1}; \theta)$$

trained by minimizing the negative log-likelihood of the next token at every position (a special case of the masked-token loss above, with $M = \{t\}$ for each position and context restricted to $x_{<t}$).

Example Data: A clinical note snippet: *“65-year-old male with severe persis-*

tent asthma, FEV1 48% predicted, blood eosinophils 420 cells/ μ L, on high-dose ICS/LABA, being evaluated for biologic therapy."

Example Output: Given the note above, a fine-tuned clinical LLM might generate a structured extraction: {age: 65, sex: "M", asthma_severity: "severe persistent", FEV1_pct: 48, eos_count: 420, current_therapy: "ICS/LABA", candidate_for: "biologic therapy"} – converting unstructured text into a structured feature vector usable by downstream statistical or ML models.

Interpretation: The extracted structured fields are only as reliable as the underlying model's clinical language understanding; each field should be spot-checked against the source note before being trusted in a downstream risk model, since LLMs can produce plausible-looking but incorrect extractions (hallucination), particularly for values not explicitly stated.

Assumptions: Enough broad pretraining data for the autoregressive objective to learn generalizable language structure; downstream clinical use assumes the model's training distribution reasonably overlaps with the target clinical text style (a model trained mostly on general web text may perform poorly on dense clinical shorthand without domain adaptation).

Advantages: Extremely broad transfer – a single pretrained model supports summarization, extraction, question-answering, and drafting without task-specific architectures; increasingly strong zero/few-shot performance reduces the labeled-data burden for new tasks.

Limitations: Hallucination risk in biomedical fact generation (Chapter 4); substantial compute cost to train from scratch (fine-tuning or prompting a pretrained model is the practical route for nearly all biomedical applications); explainability is limited – attention weights and post-hoc tools (Chapter 4) provide only partial insight into why a specific output was generated.

Data Requirements: Pretraining requires massive unlabeled text corpora (not typically undertaken in-house); downstream adaptation typically needs a modest labeled or instruction-formatted dataset (hundreds to thousands of examples) for fine-tuning, or well-crafted prompts for zero/few-shot use.

Use-Cases: Clinical note structuring and phenotyping from EHR free text; literature-scale evidence synthesis; drafting regulatory or scientific document sections (with mandatory human review); patient-facing chatbots (with strict safety guardrails, outside the scope of this book).

6.3 Vision Transformers (ViT)

Model Description: Adapts the transformer architecture to images (Dosovitskiy et al., 2021) by splitting an image into fixed-size patches, linearly embedding each patch, and processing the resulting sequence of patch embeddings with standard self-attention – removing the convolutional locality assumption entirely (Chapter 4).

Mathematical Notation: An image $I \in \mathbb{R}^{H \times W \times C}$ is split into N patches of size $P \times P$, each flattened and linearly projected:

$$z_0 = [x_{class}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{pos}$$

where E is the learned patch-embedding matrix, E_{pos} a learned positional embedding, and x_{class} a prepended classification token whose final-layer representation is used for downstream prediction. The sequence z_0 is then processed by standard transformer blocks (as above).

Example Data: A 224×224 pathology image patch from a hematoxylin-and-eosin (H&E) stained tumor biopsy slide, split into 16×16 -pixel patches (196 patches total).

Example Output: A 768-dimensional embedding vector per patch (or a single pooled embedding for the whole image via the class token), which can be used directly as a feature vector for a downstream tumor-grade classifier, or visualized via the model’s attention weights, showing which regions of the tissue the model focused on.

Interpretation: An attention map overlaid on the original image (bright regions = high attention) can reveal whether the model is focusing on tumor cells, stroma, or an irrelevant artifact (e.g., a pen mark or tissue fold) – a critical sanity check before trusting the model’s prediction, since a high-accuracy model that is actually attending to scanning artifacts will not generalize to a new pathology lab’s slides.

Assumptions: Sufficient pretraining data for the transformer to learn useful visual structure without the convolutional locality prior (ViTs generally need larger pretraining datasets than CNNs to reach comparable performance, as discussed in Chapter 4); patch size and image resolution are fixed at both pretraining and inference time (or require positional-embedding interpolation to change).

Advantages: Captures long-range spatial dependencies across an entire image in a single attention operation (e.g., relating a tumor region to distant immune infiltrate); scales well with more data and larger models; strong

transfer-learning performance when pretrained on large image corpora (e.g., ImageNet-21k) or, increasingly, domain-specific pathology foundation models (UNI, CONCH, Virchow, introduced in Chapter 5).

Limitations: Data-hungry relative to CNNs when trained from scratch; quadratic attention cost in the number of patches limits resolution/context length without architectural modifications; interpretability again requires post-hoc tools (attention maps, Chapter 4), which can be visually compelling but should not be over-interpreted as a rigorous causal explanation.

Data Requirements: Large labeled or self-supervised pretraining image corpora; for pathology-specific applications, whole-slide images (WSIs) tiled into patches as described in Chapter 5’s image preprocessing pipeline.

Use-Cases: Digital pathology tumor classification and grading; radiology image classification; as an image encoder within a multimodal fusion pipeline (Chapter 5).

6.4 CLIP (Contrastive Language-Image Pretraining)

Model Description: A multimodal foundation model (Radford et al., 2021) that jointly trains an image encoder and a text encoder to produce aligned embeddings for matched image-text pairs, using the contrastive loss introduced above – enabling zero-shot image classification by comparing an image’s embedding to the embeddings of candidate text labels.

Mathematical Notation: For a batch of N matched (image, text) pairs, CLIP maximizes the cosine similarity of matched pairs’ embeddings while minimizing similarity of the $N^2 - N$ mismatched pairs, using the contrastive loss $\mathcal{L}_{contrastive}$ above, applied symmetrically for image-to-text and text-to-image retrieval.

Example Data: A dataset of paired chest X-ray images and their corresponding radiology report captions (e.g., “Bilateral ground-glass opacities consistent with atypical pneumonia”).

Example Output: Given a new, unlabeled chest X-ray, CLIP’s zero-shot classification computes the image embedding’s cosine similarity to a set of candidate text-label embeddings (e.g., “pneumonia,” “pneumothorax,” “normal chest X-ray”) and predicts whichever text label the image embedding is most similar to – without ever being explicitly trained on those specific labels.

Interpretation: A high similarity score to “bilateral ground-glass opacities” and a low score to “normal chest X-ray” indicates the model’s joint embedding space has captured a meaningful correspondence between this visual pattern

and its textual description; because the label set is specified at inference time (as text), a new candidate diagnosis can be added without retraining, but performance on any specific label is only as good as how well that clinical concept was represented in the original image-text pretraining data.

Assumptions: Sufficient paired image-text data covering the domain of interest at pretraining time; the assumption that natural-language captions provide a meaningful, sufficiently specific supervisory signal for the visual concepts of interest (a general caption like “chest X-ray” provides much weaker supervision than a detailed radiological description).

Advantages: Zero-shot transfer to new label sets without retraining; a single joint embedding space supports both image-to-text and text-to-image retrieval; naturally extensible to new classes by simply writing new text prompts.

Limitations: Performance on rare or highly specific clinical concepts is limited by how well-represented they were in pretraining captions; contrastive pretraining requires large paired datasets, which are far scarcer in medical imaging than in general web image-caption pairs; like other foundation models, subject to bias inherited from its training data’s demographic and institutional composition.

Data Requirements: Large paired image-text datasets; in medicine, typically radiology or pathology images paired with report text, which raises data-sharing and de-identification requirements beyond a simple image-only dataset.

Use-Cases: Zero-shot and few-shot medical image classification; joint image-text retrieval (e.g., finding similar historical cases from a report); as the vision-language backbone within larger multimodal clinical AI systems.

6.5 Segment Anything Model (SAM)

Model Description: A promptable image segmentation foundation model (Kirillov et al., 2023) that, given an image and a prompt (a point, a bounding box, or a rough mask), produces a precise segmentation mask – trained on a massive, diverse segmentation dataset to generalize to new objects and image types without task-specific retraining.

Mathematical Notation: SAM combines a ViT-based image encoder (producing a dense image embedding once per image), a lightweight prompt encoder (embedding the point/box/mask prompt), and a fast mask decoder that combines the two embeddings to predict a segmentation mask $\hat{M}$ and an asso-

ciated confidence score, trained with a combined focal and Dice loss:

$$\mathcal{L}_{seg} = \mathcal{L}_{focal}(\hat{M}, M) + \mathcal{L}_{Dice}(\hat{M}, M), \quad \mathcal{L}_{Dice} = 1 - \frac{2|\hat{M} \cap M|}{|\hat{M}| + |M|}$$

Example Data: A CT scan slice with a single point prompt placed inside a visible liver lesion.

Example Output: A precise binary segmentation mask outlining the lesion’s boundary, along with a confidence score for the predicted mask.

Interpretation: A high-confidence mask that tightly follows the lesion’s visible boundary suggests the prompt was sufficient and unambiguous; a low-confidence or poorly bounded mask (e.g., including surrounding healthy tissue) signals that the lesion’s boundary is ambiguous from a single point and may need an additional prompt (a second point or bounding box) or manual correction before the mask is used for downstream radiomic feature extraction (Chapter 5).

Assumptions: The prompt (point/box) is placed on or near the actual object of interest; SAM’s pretraining data, while broad, is predominantly natural (non-medical) imagery, so medical-image segmentation quality varies substantially by modality and often benefits from medical-domain fine-tuning (e.g., MedSAM).

Advantages: Enables rapid, interactive segmentation without training a bespoke segmentation network per organ/lesion type, dramatically reducing the annotation burden described in Chapter 5’s image-preprocessing pipeline; generalizes reasonably well to novel object shapes given an informative prompt.

Limitations: Out-of-the-box performance on medical images (CT, MRI, pathology) is typically lower than on natural images, since the base model was not pretrained predominantly on medical imagery; still requires a human-provided prompt (point/box) rather than being fully automatic; like any segmentation model, errors propagate directly into any downstream radiomic or volumetric measurement (Chapter 5).

Data Requirements: No task-specific training data required for zero-shot use with prompts; medical-domain fine-tuning (recommended for clinical deployment) requires labeled segmentation masks in the target modality.

Use-Cases: Rapid, interactive tumor/organ segmentation to accelerate radiology/pathology annotation pipelines; a drop-in replacement or complement to nnU-Net (Chapter 5) for exploratory or low-resource segmentation tasks; generating training labels semi-automatically for a subsequent supervised segmentation model.

6.6 Protein Language Models (ESM-2, ESMFold, UniRep)

Model Description: Transformer models pretrained on hundreds of millions of protein sequences using the masked-token objective, treating each amino acid as a token – learning representations that capture structural, functional, and evolutionary information directly from sequence alone, without any explicit structural training signal. **ESM-2** (Lin et al., 2023) is the pretrained sequence encoder; **ESMFold** extends it with a structure-prediction module that predicts 3D atomic coordinates directly from the ESM-2 embedding, without a multiple-sequence alignment (unlike AlphaFold2’s original pipeline); **UniRep** (Alley, Khimulya, Biswas, AlQuraishi, & Church, 2019) is an earlier recurrent (LSTM-based, rather than transformer-based) protein representation model with a similar self-supervised sequence-modeling objective.

Mathematical Notation: Masked-token pretraining loss applied to amino acid sequences:

$$\mathcal{L} = - \sum_{i \in M} \log p(a_i \mid a_{\setminus M})$$

where $a_i \in \{20 \text{ amino acids}\}$. ESMFold additionally predicts a 3D structure $\hat{X} \in \mathbb{R}^{L \times 3}$ (backbone coordinates for a protein of length L) from the ESM-2 embedding via a structure module trained with a frame-aligned point error (FAPE) loss, conceptually similar to AlphaFold2’s structure module.

Example Data: A protein sequence: MKTLLVAILAVAAVTAF... (a signal-peptide-containing precursor protein, for illustration).

Example Output: (i) A per-residue embedding matrix ($L \times 1280$ for ESM-2’s largest public variant), capturing contextual information about each amino acid’s structural/functional role; (ii) an attention map showing which residue pairs the model attends to strongly (often corresponding to residues that are close in 3D space despite being distant in the linear sequence – a signal of tertiary contacts); (iii) for ESMFold, a full predicted 3D structure with a per-residue confidence score (pLDDT).

Interpretation: The strong diagonal band reflects the unsurprising fact that nearby residues in the linear sequence tend to interact chemically and structurally; the off-diagonal bright spot is more informative – it suggests the model has learned, purely from patterns across millions of training sequences, that these two sequence-distant positions are likely to be spatially close in the folded protein, which is exactly the kind of implicit structural knowledge that makes ESMFold’s sequence-only structure prediction possible.

Interpretation: Proteins with the same functional annotation (kinases, proteases, transcription factors, etc.) cluster together in the embedding space

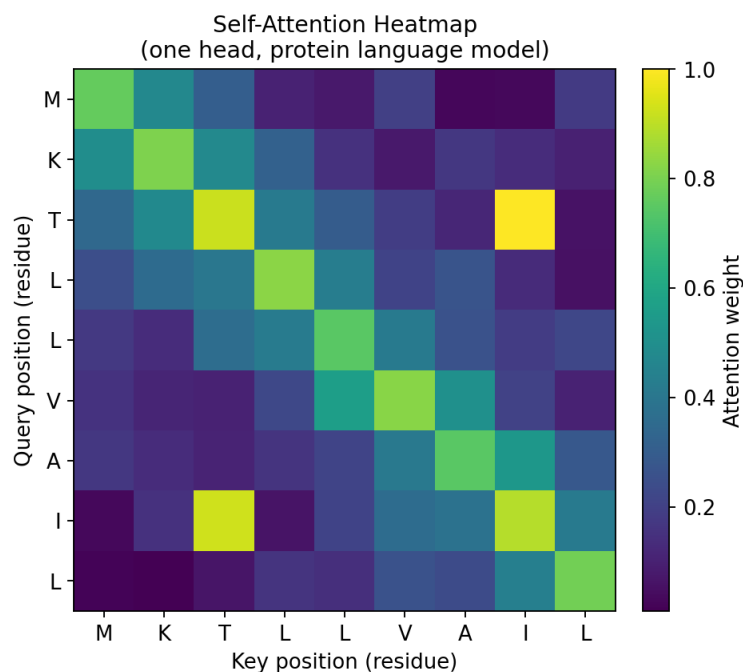


Figure 6.1: **Self-attention heatmap for a short protein sequence.** Brighter cells indicate stronger attention weight between the query residue (row) and key residue (column). The bright off-diagonal cluster (residues 3 and 8) illustrates how attention can capture a long-range structural contact that is not visible from sequence adjacency alone.

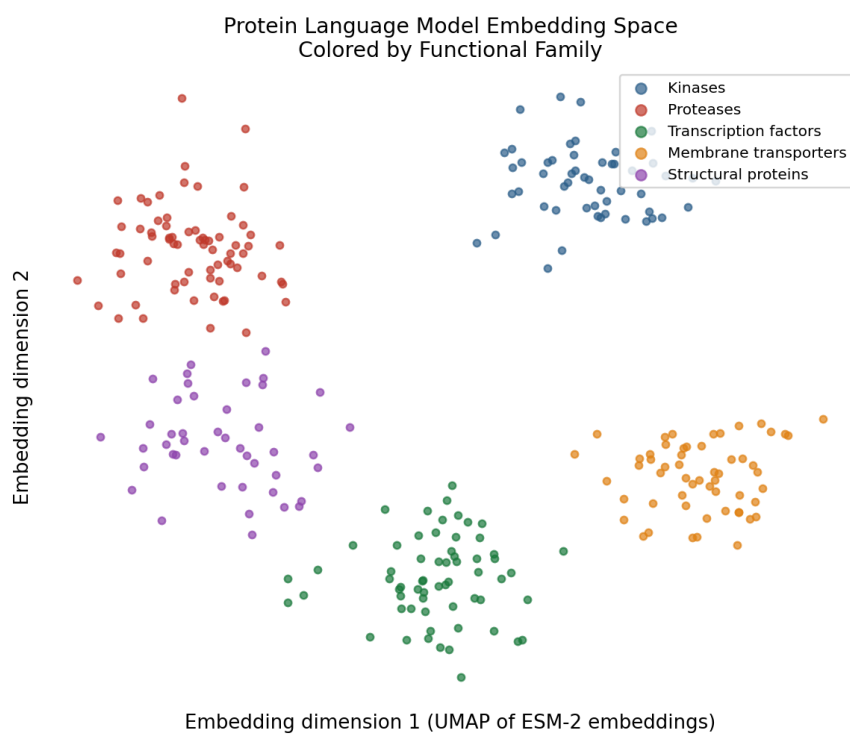


Figure 6.2: **Protein embedding space colored by functional family.** A two-dimensional projection (e.g., UMAP) of ESM-2 embeddings for a diverse set of proteins, colored by known functional annotation.

even though the model was never explicitly trained to predict protein function – function emerged as a byproduct of the masked-token pretraining objective learning evolutionary and structural patterns. This is the practical basis for using ESM-2 embeddings as off-the-shelf features for downstream tasks like function prediction, variant-effect scoring, or protein engineering, often outperforming hand-crafted sequence features.

Assumptions: Evolutionary and structural information is sufficiently encoded in sequence alone for the pretraining objective to recover it (well-supported empirically for well-studied protein families, less so for orphan proteins with few evolutionary relatives); ESMFold assumes the query protein’s fold is reasonably well represented, directly or by analogy, in the pretraining sequence corpus.

Advantages: No multiple-sequence alignment (MSA) required at inference time (unlike AlphaFold2’s original pipeline), making ESMFold dramatically faster for single-sequence structure prediction; embeddings transfer well to a wide range of downstream tasks (function prediction, variant-effect prediction, protein engineering) with minimal task-specific data; UniRep and ESM-2 embeddings can serve directly as omics-style feature vectors in the multimodal fusion pipelines of Chapter 5.

Limitations: Structure predictions are typically less accurate than MSA-based methods (e.g., AlphaFold2) for proteins with few close evolutionary relatives, since single-sequence models cannot exploit co-evolutionary signal from an alignment; large model sizes require substantial GPU memory for the largest ESM-2 variants; like all foundation models, embeddings can encode biases present in the reference sequence databases used for pretraining (e.g., under-representation of certain organisms).

Data Requirements: Pretraining requires large public sequence databases (e.g., UniRef); downstream fine-tuning for a specific task (e.g., stability prediction) typically needs hundreds to thousands of labeled sequence-property pairs.

Use-Cases: Variant-effect prediction for rare disease and pharmacogenomics (scoring how disruptive a missense mutation is likely to be); protein engineering and design; rapid structure prediction for proteins without close homologs; as a feature-extraction step for downstream drug-target interaction models.

6.7 Genomics Foundation Models (DNABERT, Geneformer)

Model Description: DNABERT (Ji, Zhou, Liu, & Davuluri, 2021) applies the masked-token transformer objective directly to DNA sequence, tokenized as overlapping k-mers, learning representations of regulatory motifs, splice sites, and other sequence-level genomic features. **Geneformer** (Theodoris et al., 2023) instead operates on single-cell transcriptomes, treating each cell’s ranked gene-expression profile as an ordered “sentence” of gene tokens (ranked by expression, with commonly expressed housekeeping genes down-weighted), pretraining on millions of single-cell transcriptomes to learn a contextual representation of gene network biology.

Mathematical Notation: DNABERT’s masked-token loss operates on overlapping k-mer tokens of a DNA sequence (e.g., 6-mers) rather than individual bases:

$$\mathcal{L} = - \sum_{i \in M} \log p(k_i | k_{\setminus M}), \quad k_i \in \{4^6 \text{ possible 6-mers}\}$$

Geneformer’s tokenization ranks genes within each cell by a normalized expression value, so that a cell’s “sentence” is $(\text{gene}_{(1)}, \text{gene}_{(2)}, \dots, \text{gene}_{(G)})$ ordered from highest to lowest rank-normalized expression, with the same masked-token objective applied over gene tokens rather than DNA k-mers or words.

Example Data: DNA sequence input to DNABERT: ATCGGATCCGTAGCTAGCATG... (a promoter or enhancer region); Geneformer input: a single cell’s expression vector across ~20,000 genes, converted to a ranked gene-token sequence.

Example Output: DNABERT produces per-k-mer embeddings usable for downstream tasks like promoter/enhancer classification or splice-site prediction; a fine-tuned DNABERT classifier might output a probability that a given sequence window is an active promoter. Geneformer produces a per-cell embedding (summarizing that cell’s overall transcriptional state in the model’s learned representation space) and per-gene contextual embeddings, usable for cell-type classification, gene network inference, or – via in silico perturbation – predicting the downstream transcriptional effect of hypothetically deleting a specific gene from a specific cell’s context.

Interpretation: A high predicted promoter-activity probability from DNABERT for a given sequence window is a computational hypothesis about regulatory function, most useful for prioritizing candidate regulatory elements for experimental validation (e.g., a reporter assay) rather than as a standalone claim of function. Geneformer’s in silico gene-deletion output – a predicted

shift in the cell's embedding when a gene's expression is computationally zeroed out – is interpreted as a model-based estimate of that gene's regulatory importance in that specific cellular context, analogous in spirit to the SHAP-based feature-importance reasoning in Chapter 4, but operating directly on a learned gene-network representation rather than a downstream prediction.

Assumptions: DNABERT assumes k-mer tokenization captures sufficient local sequence context for the regulatory signal of interest (longer-range regulatory interactions, e.g., enhancer-promoter looping, are only partially captured by a fixed local context window); Geneformer assumes a cell's rank-ordered gene expression profile, rather than absolute expression values, is sufficient to capture its relevant biological state – a modeling choice that specifically reduces sensitivity to technical scaling differences between single-cell datasets (Chapter 4).

Advantages: DNABERT provides a reusable, pretrained sequence representation for a wide range of regulatory-genomics tasks without training a bespoke model per task; Geneformer's cell embeddings transfer well to new single-cell datasets and support in silico perturbation experiments that would be expensive or slow to run in a wet lab, useful for hypothesis prioritization.

Limitations: DNABERT's fixed k-mer tokenization and limited context window constrain its ability to model very long-range regulatory elements; Geneformer's in silico perturbation predictions are model-based hypotheses, not experimental measurements, and require wet-lab validation before being treated as established biology; both models inherit whatever biases and gaps exist in their pretraining corpora (e.g., under-representation of non-model organisms or rare cell types).

Data Requirements: DNABERT pretraining requires large reference genome sequence; Geneformer pretraining requires very large single-cell atlases (tens of millions of cells across many tissues/conditions); downstream fine-tuning for either typically requires a much smaller labeled dataset specific to the task (e.g., a few thousand labeled promoter/non-promoter sequences, or a labeled cell-type reference for a classification task).

Use-Cases: DNABERT: promoter/enhancer/splice-site prediction, variant-effect prediction for non-coding variants; Geneformer: cell-type annotation and transfer learning across single-cell datasets, in silico gene-perturbation screening to prioritize candidate therapeutic targets, disease-state classification from single-cell transcriptomes.

6.8 Single-Cell Foundation Models (scGPT)

Model Description: A transformer-based foundation model (Cui et al., 2024) pretrained on tens of millions of single cells across many tissues, cell types, and experimental conditions, treating each cell’s gene expression profile (and, in multimodal extensions, additional omics layers) as a token sequence in a manner conceptually related to Geneformer, but designed explicitly to support both representation learning (embeddings for downstream tasks) and generative tasks (e.g., predicting a cell’s expression profile under a perturbation it was never directly measured under).

Mathematical Notation: scGPT combines a masked gene-expression-value prediction objective (predicting the expression level of masked genes from the rest of the profile, similar in spirit to the general masked-token loss but regressing a continuous expression value rather than classifying a discrete token) with, in some training regimes, a generative objective for simulating a cell’s expression profile under a specified condition or perturbation.

Example Data: A gene-by-cell expression matrix from a 10x Genomics experiment (e.g., 20,000 genes $\times$ 5,000 cells) from asthmatic versus healthy airway epithelial brushings, as introduced in Chapter 4’s scRNA-seq discussion.

Example Output: A per-cell embedding usable for the same clustering/annotation workflow as Chapter 4’s Seurat/Scanpy pipeline, but benefiting from scGPT’s broad pretraining across many prior datasets (improving performance when the local dataset is small); additionally, a predicted expression profile for each cell under a simulated perturbation (e.g., “what would this goblet cell’s transcriptome look like if IL-13 signaling were computationally blocked”).

Interpretation: When scGPT’s embeddings are used in place of a standard PCA-based representation in the clustering pipeline from Chapter 4, cell-type separation is often cleaner in datasets with limited local sample size, since the model brings prior knowledge from its much larger pretraining corpus – conceptually the single-cell analogue of using an ImageNet-pretrained CNN instead of training from scratch on a small local imaging dataset (Chapter 4). A simulated perturbation output should be treated as a testable hypothesis about cellular response, not a validated prediction, and would typically motivate a targeted, smaller-scale wet-lab experiment to confirm.

Assumptions: The pretraining corpus’s tissue and condition coverage is broad enough to transfer usefully to the target dataset’s biological context (transfer is generally stronger for well-represented tissue types than for rare or highly specialized cell states); simulated perturbation outputs assume the

model has implicitly learned a reasonable approximation of the relevant gene-regulatory dynamics from correlational training data, not causal experimental data.

Advantages: Substantially reduces the amount of local training data needed for robust cell-type annotation and downstream classification, particularly valuable for smaller single-cell studies (e.g., a single clinical cohort) that cannot support training a large model from scratch; supports both representation learning and hypothesis-generating generative/perturbation tasks within one framework.

Limitations: Generative/perturbation outputs require experimental validation before being treated as biological findings (echoing this book’s repeated caution around SHAP, mediation, and other model-based signals that require independent confirmation); computationally intensive to pretrain (though fine-tuning or embedding extraction from a released pretrained model is much lighter-weight); performance depends on how well the target tissue/condition is represented in the pretraining corpus.

Data Requirements: Pretraining requires very large, diverse single-cell atlases; downstream use on a new dataset requires only a standard single-cell experiment (as in Chapter 4’s scRNA-seq workflow) processed through the same QC/normalization steps before embedding extraction.

Use-Cases: Cell-type annotation transfer to new, smaller single-cell datasets; batch-robust integration across studies (extending Harmony/scVI, Chapter 4); in silico perturbation screening to prioritize candidate drug targets or gene knockouts before wet-lab validation; disease-state classification from single-cell profiles in translational research.

6.9 Multimodal Foundation Models (Perceiver IO, Flamingo, GPT-4o)

Model Description: General-purpose architectures designed to accept and reason over multiple, heterogeneous input modalities within a single model. **Perceiver IO** (Jaegle et al., 2021) uses a fixed-size latent array that cross-attends into arbitrarily large and varied inputs (images, audio, text, or structured omics data), decoupling compute cost from input size. **Flamingo** (Alayrac et al., 2022) interleaves frozen vision and language backbones via learned cross-attention layers, enabling few-shot, in-context multimodal learning from just a handful of examples. **GPT-4o** (OpenAI, 2024) (“omni”) is trained end-to-end across text, image, and audio modalities within a single unified model, rather than bolting separate encoders together after the fact.

Mathematical Notation: Perceiver IO’s cross-attention from a fixed-size latent array $Z \in \mathbb{R}^{N \times d}$ (with N much smaller than the input size) into a large input X :

$$Z' = \text{LayerNorm}(Z + \text{Attention}(Q = Z, K = X, V = X))$$

followed by standard self-attention among the latents, decoupling the expensive attention computation from the (potentially very large) raw input size – directly addressing the quadratic-attention-cost limitation of standard transformers noted in Chapter 4. Flamingo’s cross-attention layers similarly let a frozen language model attend into visual features extracted by a frozen vision encoder, adding only a small number of new trainable cross-attention parameters rather than retraining either backbone from scratch.

Example Data: A multimodal clinical case combining a chest CT scan, a structured EHR summary (labs, vitals, comorbidities), and a free-text radiology report – exactly the three-modality setup introduced in Chapter 5’s multimodal fusion architecture.

Example Output: A unified multimodal embedding or a direct natural-language answer to a clinical query spanning modalities (e.g., “Given this patient’s CT findings and lab trends, what is the most likely diagnosis, and which specific findings support it?”), with the model’s response drawing on and referencing both the imaging and structured data.

Interpretation: A well-functioning multimodal foundation model’s response should explicitly ground its reasoning in checkable evidence from each modality (e.g., “the CT shows a 2cm right-upper-lobe nodule, and the patient’s rising CEA trend in the EHR is consistent with malignancy”) rather than a generic answer that could have been produced from only one modality – a useful practical check for whether the model is genuinely fusing information or merely defaulting to its strongest single modality, echoing the modality-benchmarking discipline from Chapter 5.

Assumptions: Sufficient paired/aligned multimodal pretraining data covering the domains and modality combinations of interest; cross-attention-based fusion (Perceiver IO, Flamingo) assumes a frozen or lightly adapted backbone still transfers well to the target domain without full retraining, which holds better for well-represented domains (general radiology) than for narrow, data-scarce specialties.

Advantages: A single model handles multiple modalities and tasks without separate bespoke architectures per modality combination; Perceiver IO’s fixed-latent design scales to very high-dimensional or long-sequence inputs (e.g., a full whole-slide pathology image plus a lengthy EHR history) without the

quadratic cost blowup of standard attention; Flamingo-style few-shot adaptation reduces the labeled-data burden for new clinical tasks.

Limitations: Substantial computational and data infrastructure requirements, generally exceeding what most academic or single-institution biomedical teams can pretrain from scratch (most practical use involves adapting an existing pretrained model); explainability compounds the challenges already present in unimodal deep models (Chapter 4), since an error could originate in any modality’s encoder, the fusion mechanism, or the final reasoning step; data governance and privacy requirements multiply when combining imaging, omics, and EHR text under a single model, particularly for models accessed via external APIs.

Data Requirements: Large-scale, ideally institutionally diverse, multimodal paired datasets for pretraining; realistic clinical adaptation typically relies on fine-tuning or prompting a large pretrained model with a comparatively small, well-curated local multimodal dataset (hundreds to low thousands of cases) rather than pretraining from scratch.

Use-Cases: Unified multimodal clinical decision support drawing on imaging, labs, and notes simultaneously; automated multimodal case summarization for tumor boards or multidisciplinary review; few-shot adaptation to a new imaging modality or rare disease area where labeled data is scarce.

6.10 Graph Foundation Models (GNN-FM)

Model Description: Foundation models built around graph neural network (GNN) architectures, pretrained on large collections of graphs (molecular graphs, protein-protein interaction networks, patient-similarity networks, knowledge graphs) using self-supervised graph objectives, then adapted to downstream graph-structured prediction tasks.

Mathematical Notation: A GNN layer updates each node’s representation by aggregating information from its neighbors, extending the message-passing formulation (Kipf & Welling, 2017) introduced in Chapter 4:

$$h_v^{(k)} = \text{UPDATE}\left(h_v^{(k-1)}, \text{AGGREGATE}(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\})\right)$$

Common self-supervised pretraining objectives for graph foundation models include **masked node/edge prediction** (analogous to the masked-token loss, but masking a node’s features or an edge’s existence and predicting it from graph context) and **graph contrastive learning** (maximizing agreement between two augmented views of the same graph, using the contrastive loss introduced above).

Example Data: A protein-protein interaction (PPI) network where nodes are proteins (with initial features from an ESM-2 embedding, connecting this section back to the protein language models above) and edges represent experimentally observed or predicted interactions; alternatively, a molecular graph for a drug candidate, where nodes are atoms and edges are chemical bonds.

Example Output: A learned node embedding for each protein in the PPI network, positioning functionally or pathway-related proteins near each other in embedding space (an analogous structure to the protein-family clustering shown in the embedding-space figure above, but incorporating network topology rather than sequence alone); for a molecular graph, a predicted property (e.g., binding affinity to a target, or predicted toxicity) derived from a graph-level pooled embedding.

Interpretation: Two proteins positioned close together in the GNN-derived embedding space, despite having dissimilar sequences, suggests the model has inferred a shared functional role or pathway membership from the interaction network's topology rather than from sequence homology alone – a complementary signal to the sequence-based clustering from a protein language model, and a useful additional line of evidence when prioritizing a candidate gene or protein for follow-up (echoing the multi-omics convergence principle from Chapter 2: a hypothesis supported by both sequence-based and network-based evidence is more credible than one supported by either alone).

Assumptions: The graph structure provided (PPI network, molecular graph, knowledge graph) is a reasonably accurate and complete representation of the true underlying relationships (PPI networks in particular are known to be incomplete and biased toward well-studied proteins); message-passing GNNs assume that a node's relevant context is well captured by its (multi-hop) local neighborhood, which can under-represent very long-range graph dependencies without deeper architectures or graph-transformer variants.

Advantages: Naturally represents relational biomedical data (interaction networks, molecular structure, knowledge graphs) that does not fit neatly into the sequence or grid-image formats other foundation models assume; pretraining on large public graph corpora (e.g., STRING PPI database, large molecular libraries) transfers to smaller, task-specific downstream graphs; directly supports drug-discovery tasks like molecular property prediction and drug-target interaction prediction, which are naturally graph-structured problems.

Limitations: Incomplete or biased input graphs (e.g., a PPI network skewed toward heavily studied disease genes) propagate that bias into every downstream embedding and prediction; over-smoothing (node representations becoming indistinguishable after many message-passing layers) limits how deep a GNN can practically go; explainability tools for GNNs (e.g., GNNExplainer,

identifying which edges/nodes most influenced a prediction) are less mature and less widely adopted than SHAP/LIME for tabular models or Grad-CAM for images.

Data Requirements: A graph-structured dataset (network edges plus node/edge features) at the appropriate scale for the task; pretraining benefits from large public graph databases (STRING, Reactome pathway graphs, ChEMBL molecular libraries); downstream fine-tuning can often work with a substantially smaller, task-specific graph.

Use-Cases: Drug-target interaction prediction and virtual screening; molecular property prediction (solubility, toxicity, binding affinity) in early-stage drug discovery; gene prioritization by propagating known disease associations across a PPI or pathway network; patient-similarity network analysis for cohort stratification.

6.11 Chapter Summary: Foundation Model Selection Guide

Data Type	Foundation Model Family	Typical Use-Case
Free text (clinical notes, literature)	GPT-style LLMs	Note structuring, summarization, extraction
Natural or medical images	Vision Transformers (ViT)	Classification, feature extraction
Paired image + text	CLIP	Zero-shot classification, retrieval
Images needing segmentation	Segment Anything Model (SAM)	Interactive/rapid segmentation
Protein sequences	ESM-2 / ESMFold / UniRep	Structure prediction, variant-effect scoring
DNA sequence	DNABERT	Regulatory element / splice-site prediction
Single-cell transcriptomes	Geneformer, scGPT	Cell-type annotation, in silico perturbation
Multiple heterogeneous modalities	Perceiver IO, Flamingo, GPT-4o	Unified multimodal reasoning
Networks/graphs (PPI, molecules)	Graph foundation models (GNN-FM)	Drug-target prediction, gene prioritization

6.12 Cross-Cutting Assumptions, Advantages, and Limitations

Assumptions common to all foundation models: self-supervised pretraining objectives (masked-token prediction, contrastive alignment) provide a sufficiently rich learning signal to recover broadly useful representations without labels; tokenization (of text, DNA k-mers, image patches, or graph nodes) preserves the information relevant to downstream tasks; for multimodal models, that paired/aligned data across modalities meaningfully reflects the same underlying entity (the same patient, the same protein) rather than introducing spurious correspondences.

Advantages common to all foundation models: dramatic reduction in labeled-data requirements for new downstream tasks via transfer learning; a single pretrained model amortizes an enormous pretraining investment across many subsequent applications; increasingly strong multimodal capability, integrating data types that previously required entirely separate modeling pipelines (Chapter 5).

Limitations common to all foundation models: substantial computational cost to pretrain from scratch (nearly always impractical outside a small number of well-resourced organizations, making adaptation of existing released models the practical norm); explainability challenges compound with model scale and modality count, requiring the full toolkit of Chapter 4’s interpretability methods (SHAP, attention maps, Grad-CAM, integrated gradients, TCAV) and still leaving open questions a domain expert must adjudicate; bias inherited from pretraining data composition (demographic, institutional, geographic) can silently propagate into every downstream application built on a given foundation model, making bias auditing on the specific downstream population a necessary step before clinical deployment, not an optional extra.

6.13 References

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving language understanding by generative pre-training*. OpenAI.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning* (pp. 8748-8763).

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W. Y., Dollár, P., & Girshick, R. (2023). Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 4015-4026).

Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., & Rives, A. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130.

Alley, E. C., Khimulya, G., Biswas, S., AlQuraishi, M., & Church, G. M. (2019). Unified rational protein engineering with sequence-based deep representation learning. *Nature Methods*, 16(12), 1315-1322.

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., ... Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589.

Ji, Y., Zhou, Z., Liu, H., & Davuluri, R. V. (2021). DNABERT: Pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 37(15), 2112-2120.

Theodoris, C. V., Xiao, L., Chopra, A., Chaffin, M. D., Al Sayed, Z. R., Hill, M. C., Mantineo, H., Brydon, E. M., Zeng, Z., Liu, X. S., & Ellinor, P. T. (2023). Transfer learning enables predictions in network biology. *Nature*, 618(7965), 616-624.

Cui, H., Wang, C., Maan, H., Pang, K., Luo, F., Duan, N., & Wang, B. (2024). scGPT: Toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21, 1470-1480.

Jaegle, A., Borgeaud, S., Alayrac, J. B., Doersch, C., Ionescu, C., Ding, D., Koppula, S., Zoran, D., Brock, A., Shelhamer, E., Hénaff, O., Botvinick, M. M., Zisserman, A., Vinyals, O., & Carreira, J. (2021). Perceiver IO: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795*.

Alayrac, J. B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., Ring, R., Rutherford, E., Cabi, S., Han, T., Gong, Z., Samangooei, S., Monteiro, M., Menick, J., Borgeaud, S., ... Simonyan, K. (2022). Flamingo: A visual language model for few-shot learning.

In *Advances in Neural Information Processing Systems 35* (pp. 23716-23736).

OpenAI. (2024). *Hello GPT-4o* [System card]. OpenAI.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., & Leskovec, J. (2020). Open Graph Benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems 33* (pp. 22118-22133).

Szklarczyk, D., Kirsch, R., Koutrouli, M., Nastou, K., Mehryary, F., Hachilif, R., Gable, A. L., Fang, T., Doncheva, N. T., Pyysalo, S., Bork, P., Jensen, L. J., & von Mering, C. (2023). The STRING database in 2023: Protein-protein association networks and functional enrichment analyses for any sequence of organisms of interest. *Nucleic Acids Research*, 51(D1), D638-D646.

7 Comprehensive Model Reference Guide

*Every method below follows the same eight-part template: **(1)** Model Description, **(2)** Mathematical Notation, **(3)** Assumptions, **(4)** Data Requirements, **(5)** Advantages, **(6)** Limitations, **(7)** When to Use, **(8)** When NOT to Use — followed by an example and R/Python code. This chapter is designed to stand alone as a methods-selection and professional reference.*

7.1 Simple Linear/Logistic Regression (Baseline for Comparison)

(1) Model Description: Models a single outcome as a linear (or log-odds-linear) function of predictors, assuming each observation is an independent draw.

(2) Mathematical Notation:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$$

Logistic form: $\text{logit}(P(y_i = 1)) = \beta_0 + \sum_j \beta_j x_{ij}$

(3) Assumptions: Independence of observations; linearity in the predictors (or log-odds); homoscedasticity (linear case); no perfect multicollinearity; correctly specified link function (logistic case).

(4) Data Requirements: One row per independent unit (one measurement per subject); n at least $\sim 10\text{-}20\times$ the number of predictors as a rule of thumb; continuous or categorical predictors, properly coded.

(5) Advantages: Highly interpretable coefficients (effect size, odds ratio); computationally trivial; well-understood inferential theory (CIs, p-values); regulatory-familiar.

(6) Limitations: Cannot handle repeated/correlated measurements without violating independence; sensitive to outliers and multicollinearity; assumes linear relationship unless explicitly transformed/interacted.

(7) When to Use: Single time-point/cross-sectional analyses; one outcome per subject; as an interpretable baseline before more complex models.

(8) When NOT to Use: Repeated measures on the same subject (violates independence — see 8.2); highly nonlinear relationships without transformation; $p \gg n$ settings (use 8.7/regularized regression instead).

Example: Logistic regression of asthma exacerbation (yes/no) on baseline IgE, age, and sex in a single-visit cross-sectional cohort (e.g., a CAAPA sub-analysis).

```
model <- glm(exacerbation ~ age + sex + log(IgE), family = binomial, data = df)
summary(model); exp(cbind(OR = coef(model), confint(model)))
```

7.2 Mixed-Effects Models for Longitudinal Data

(1) Model Description: Extends regression to repeated-measures/clustered data by adding subject-specific (or cluster-specific) random effects that capture correlation among observations from the same unit, while fixed effects estimate population-average associations.

(2) Mathematical Notation:

$$y_{ij} = X_{ij}\beta + Z_{ij}b_i + \epsilon_{ij}$$

$$b_i \sim N(0, \Sigma), \quad \epsilon_{ij} \sim N(0, \sigma^2 R_i), \quad b_i \perp \epsilon_{ij}$$

where i indexes subject, j indexes repeated measurement, $X_{ij}\beta$ is the fixed-effects design, $Z_{ij}b_i$ is the subject-specific random-effects design (e.g., random intercept and/or slope), and R_i allows a flexible within-subject residual covariance structure (compound symmetry, AR(1), unstructured).

Marginal (population-averaged) covariance implied: $\text{Var}(y_i) = Z_i \Sigma Z_i^T + \sigma^2 R_i$.

(3) Assumptions: Random effects normally distributed, $b_i \sim N(0, \Sigma)$; correctly specified random-effects structure (random intercept only vs. intercept+slope); linearity in fixed effects; missing data mechanism is Missing At Random (MAR) for valid maximum-likelihood inference; residuals within-subject follow the specified covariance structure.

(4) Data Requirements: Long-format data (one row per subject-visit); works with unbalanced/missing visits (a key advantage over repeated-measures ANOVA); needs enough subjects (typically 30+ for stable variance-component estimation) and enough repeated measures per subject (≥ 3 recommended) to estimate a random slope.

(5) Advantages: Valid standard errors under within-subject correlation; handles unbalanced/missing visit schedules gracefully under MAR; separates

within- vs. between-subject variance components; widely accepted by FDA/EMA for repeated-measures endpoints; flexible covariance structures.

(6) Limitations: Computationally more demanding than OLS (requires REML/ML iterative fitting); can have convergence issues with complex random-effects structures or small samples; misspecifying the random-effects structure can bias inference; not naturally suited to MNAR dropout (informative censoring) without extension (joint models, pattern-mixture models).

(7) When to Use: Clinical trials/cohort studies with repeated measurements (e.g., FEV1 at multiple visits); any nested/clustered data (patients within clinics, cells within patients); when the scientific question concerns individual trajectories or requires proper handling of within-subject correlation.

(8) When NOT to Use: Single time-point data (unnecessary complexity — use 8.1); very few repeated measures per subject with very small n (variance components unstable — consider GEE for population-average inference instead); when data are severely MNAR (informative dropout) without additional modeling.

Example: UK Biobank-style repeated spirometry (FEV1/FVC) modeled with time, treatment, time:treatment fixed effects and (1+time|subject_id) random effects to test whether a drug slows lung-function decline.

```
library(lme4); library(lmerTest)
model <- lmer(FEV1 ~ time * treatment + age + sex + (1 + time | subject_id), data
  ↪ = longdata)
summary(model)           # fixed effects with Satterthwaite/KR p-values (lmerTest)
VarCorr(model)          # random-effects variance components

import statsmodels.formula.api as smf
model = smf.mixedlm("FEV1 ~ time * treatment + age + sex", data=df,
  groups=df["subject_id"], re_formula="~time")
result = model.fit()
print(result.summary())
```

7.3 Machine Learning / Deep Learning as Alternatives to Mixed-Effects Models

(1) Model Description: Flexible, often nonparametric models (LSTMs, Gaussian Processes, mixed-effects random forests/MERF, GLMM-boosting, transformer time-series models) that can represent longitudinal trajectories without assuming a specific parametric fixed/random-effects form.

(2) Mathematical Notation (representative — LSTM cell, repeated

from Module 2.3 for completeness):

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

Mixed-Effects Random Forest (MERF) conceptually iterates:

$$y_{ij} = f_{RF}(X_{ij}) + Z_{ij}b_i + \epsilon_{ij}$$

alternating between fitting a random forest f_{RF} on the fixed-effects part and updating the random-effects estimate b_i via a linear mixed-model step (EM-style), until convergence.

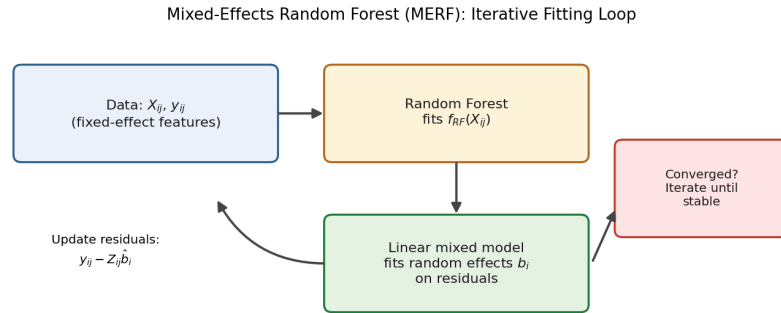


Figure 7.1: **Mixed-Effects Random Forest (MERF): the iterative fitting loop.** A random forest captures nonlinear fixed effects, a linear mixed model captures subject-level random effects on the residuals, and the two steps alternate until the fit stabilizes.

Interpretation: Unlike a standard random forest, which would treat every visit from every patient as an independent row and ignore the repeated-measures structure entirely, MERF explicitly separates two jobs: the random forest handles whatever nonlinear, high-order-interaction relationship exists between the observed covariates and the outcome, while a conventional linear mixed model handles the part a standard mixed model is good at — capturing how much each subject’s trajectory deviates from that fitted curve. The loop repeats because the two components are entangled: a better random-effects estimate changes the residuals the random forest sees next, and vice versa. This hybrid is a practical middle ground for teams that want deep-learning-style flexibility in the fixed-effects relationship without giving up an explicit, subject-level random-effects structure entirely.

(3) Assumptions: Far fewer distributional assumptions than parametric mixed models; LSTMs/GPs assume sufficient training data to learn the functional form; MERF assumes the random-effects part is still reasonably linear/Gaussian even though the fixed part is flexible; deep models generally assume i.i.d. train/test splits at the *subject* level to avoid leakage.

(4) Data Requirements: Typically need substantially larger sample sizes than mixed models to avoid overfitting (hundreds to thousands of subjects with dense repeated measures for DL; MERF can work with moderate n given a random-forest fixed-effects part regularizes reasonably well); DL time-series methods benefit from regularly sampled or explicitly time-encoded irregular sampling.

(5) Advantages: Can capture nonlinear, non-additive trajectory shapes and complex interactions automatically; scales well to large EHR-style longitudinal datasets; can incorporate high-dimensional time-varying covariates naturally (e.g., full lab panels at each visit).

(6) Limitations: Typically lack calibrated inferential uncertainty (p-values/CIs) for a specific fixed effect like “treatment effect,” which most regulatory submissions require; harder to interpret; higher risk of overfitting with small clinical-trial-sized samples; less established acceptance in regulatory (FDA/EMA) contexts for confirmatory endpoints.

(7) When to Use: Large-scale prediction tasks (e.g., predicting next-visit risk score from EHR trajectories) where accuracy, not effect-size inference, is the goal; highly nonlinear trajectory shapes; very large sample sizes (biobank/EHR scale).

(8) When NOT to Use: Small clinical trials ($n < 200$ -300) where a specific treatment-effect estimate with valid inference is the primary endpoint; regulatory confirmatory analyses requiring pre-specified, interpretable models; when interpretability of “why” is as important as “what.”

Example: Predicting 1-year exacerbation risk from a longitudinal EHR trajectory of lung function, inhaler refills, and inflammatory labs using an LSTM on MIMIC-style structured time series, vs. a mixed model used for the confirmatory trial endpoint of the same drug.

```
import torch.nn as nn
class TrajectoryLSTM(nn.Module):
    def __init__(self, n_features, hidden=32):
        super().__init__()
        self.lstm = nn.LSTM(input_size=n_features, hidden_size=hidden,
                             ↪ batch_first=True)
        self.head = nn.Linear(hidden, 1)
```

```
def forward(self, x):                                # x: (batch, time_steps, n_features)
    out, (h_n, c_n) = self.lstm(x)
    return torch.sigmoid(self.head(h_n[-1]))
```

7.3.1 Comparison Table — Mixed-Effects vs. ML/DL for Longitudinal Data

Criterion	Mixed-Effects Model	ML/DL (LSTM, MERF, GP)
Sample size needed	Moderate (works with n~30-300)	Large (hundreds-thousands)
Primary goal	Inference on a specific effect	Prediction accuracy
Interpretability	High (coefficients, variance components)	Low-moderate (needs SHAP/attention)
Handles nonlinearity	Limited (needs manual splines/polynomials)	Native
Regulatory acceptance	Well-established	Emerging, case-by-case
Handles irregular/missing visits	Yes (under MAR)	Varies by architecture

7.4 Frequentist vs. Bayesian Inference

(1) Model Description: Two overarching philosophies for statistical inference. Frequentist inference treats parameters as fixed unknowns and evaluates procedures by their long-run behavior over hypothetical repeated sampling. Bayesian inference treats parameters as random variables with a prior distribution, updated to a posterior via observed data.

(2) Mathematical Notation: Frequentist MLE: $\hat{\theta}_{MLE} = \arg \max_{\theta} L(\theta | D) = \arg \max_{\theta} \prod_i f(x_i; \theta)$ Bayesian posterior: $P(\theta | D) = \frac{P(D | \theta) P(\theta)}{\int P(D | \theta') P(\theta') d\theta'} \propto L(\theta | D) P(\theta)$ Credible interval: region C such that $P(\theta \in C | D) = 0.95$ (direct probability statement, unlike a frequentist CI).

(3) Assumptions: Frequentist: correctly specified likelihood/model family, asymptotic normality for Wald-type inference (or exact small-sample methods), well-defined sampling distribution. Bayesian: correctly specified likelihood **and** a prior $P(\theta)$ that the analyst must choose (informative, weakly informative, or non-informative); computational convergence (MCMC chains mixing well, or valid variational approximation).

(4) Data Requirements: Frequentist large-sample methods perform best with adequate sample size for asymptotics to hold (rule of thumb ≥ 10 events per parameter for logistic/Cox models); Bayesian methods can be used validly at any sample size, including very small n , since the posterior is always well-defined (though wide/uncertain with little data).

(5) Advantages: *Frequentist* — no subjective prior needed, well-established Type-I-error control, simple software/ubiquity, strong regulatory precedent. *Bayesian* — coherent uncertainty quantification via full posterior, naturally handles small samples/rare events, supports sequential/adaptive designs (interim looks without alpha-spending penalties in the same way), allows incorporation of external/historical data via informative priors.

(6) Limitations: *Frequentist* — p-values/CIs are frequently misinterpreted; asymptotic methods can fail in small samples or rare events; no formal mechanism to incorporate prior trial data. *Bayesian* — computationally intensive (MCMC/HMC via Stan, or variational inference); results can be sensitive to prior specification if data are sparse; less uniform regulatory acceptance (though increasing, e.g., FDA's guidance on Bayesian adaptive trial designs).

(7) When to Use: *Frequentist* — standard confirmatory clinical trials, regulatory submissions, well-powered GWAS/EWAS with large n . *Bayesian* — rare disease/small-sample trials, adaptive/sequential trial designs, hierarchical/multi-level borrowing of information (e.g., across sub-studies or biomarker sub-groups), rare-variant genetic association, integrating historical control data.

(8) When NOT to Use: *Frequentist* — extremely small samples/rare events where asymptotic approximations break down (use exact tests or Bayesian methods instead). *Bayesian* — when regulatory precedent strongly requires a standard frequentist confirmatory analysis and there's no time/expertise to justify and pre-specify priors, or when stakeholders will not accept subjective prior specification regardless of sensitivity analysis.

Example: A Phase I rare-variant pharmacogenomic sub-study with 12 patients uses a Bayesian hierarchical model (borrowing strength across similar variants) to estimate a dose-response relationship, while the pivotal Phase III trial primary endpoint uses a standard frequentist Cox model per FDA convention.

Figure 3.2 in Chapter 1 (Biostatistics) shows the frequentist point-estimate-plus-CI representation side by side with the Bayesian full-posterior representation of the same underlying effect — the two panels are worth reviewing together with the notation above.

7.4.1 Example Result — Same Data, Two Framings

Quantity	Frequentist Output	Bayesian Output (weakly informative prior)
Point summary	$\hat{\beta} = 2.01$	Posterior mean = 1.97
Uncertainty interval	95% CI: (0.62, 3.40)	95% credible interval: (0.68, 3.31)
Interpretation	“95% of such intervals, over repeated sampling, contain the true β ”	“There is a 95% probability β lies in (0.68, 3.31), given the data and prior”
Direct probability statement about a threshold?	Not directly available	$P(\beta > 1.5 \mid D) = 0.87$ (computable directly from posterior draws)

Interpretation: With a reasonably large, well-powered dataset, the two numeric intervals end up nearly identical (as this table shows) — the real difference is in what question each can directly answer. Only the Bayesian posterior can be used to answer “what is the probability the effect exceeds a pre-specified clinically important threshold?” directly; the frequentist interval requires a separate (and often misinterpreted) hypothesis test to approximate the same question.

```
library(brms)
bayes_model <- brm(FEV1_change ~ dose + age + (1|site), data = df,
  family = gaussian(), prior = set_prior("normal(0,10)",
    ↪ class="b"),
  chains = 4, iter = 2000)
summary(bayes_model)
```

7.5 Is Bayesian Inference Valid Without Strong Prior Evidence?

(1) Model Description: Bayesian inference using **weakly informative or non-informative priors** — priors chosen to be minimally influential relative to the likelihood, used precisely when little external/prior evidence exists.

(2) Mathematical Notation: With a flat/vague prior $P(\theta) \propto 1$ (or a diffuse $N(0, \tau^2)$ with large τ), the posterior reduces to being (approximately) proportional to the likelihood alone:

$$P(\theta \mid D) \propto L(\theta \mid D) \times 1 \Rightarrow \text{posterior mode} \approx \hat{\theta}_{MLE} \text{ as } n \rightarrow \infty$$

This is a direct consequence of the **Bernstein-von Mises theorem**: under regularity conditions, the posterior distribution converges to a normal distribution centered at the MLE as $n \rightarrow \infty$, regardless of the (non-degenerate) prior chosen.

(3) Assumptions: The likelihood is correctly specified; the prior, while “weak,” must still be **proper** (integrates to 1) in most practical implementations to guarantee a proper posterior and valid MCMC sampling; enough data exist for the likelihood to meaningfully dominate (for genuinely tiny samples, the prior *will* still matter — this should be disclosed via sensitivity analysis).

(4) Data Requirements: No specific minimum — this is precisely the setting for small/rare-event data — but the analyst should report a **prior sensitivity analysis** (re-fit with 2-3 alternative reasonable priors) to demonstrate the conclusion isn’t an artifact of prior choice.

(5) Advantages: Still yields a full posterior (uncertainty quantification) even in a 5-10-sample rare-variant or rare-adverse-event setting where frequentist asymptotics are invalid; provides a principled way to say “we have great uncertainty” (wide credible interval) rather than a single fragile p-value; supports hierarchical partial-pooling across related strata even with weak priors at the top level.

(6) Limitations: “Weakly informative” is itself a modeling choice with a spectrum (flat vs. weakly regularizing vs. moderately informative) — reviewers may (reasonably) ask for justification; with very small n , results remain highly uncertain no matter the inferential paradigm — Bayesian methods make that uncertainty explicit rather than resolving it.

(7) When to Use: Rare disease trials, rare-variant genetic association, early-phase/small- n biomarker studies, whenever you want a full uncertainty distribution rather than a binary significance call under data scarcity.

(8) When NOT to Use: As a way to manufacture false confidence by unintentionally choosing an overly informative prior that isn’t disclosed/justified; when a genuinely informative prior (e.g., from a prior identical trial) exists but is not incorporated — that would waste real information the Bayesian framework is well-suited to use.

Example: Estimating the effect of a rare pharmacogenomic variant (8 carriers) on drug clearance using a weakly informative $N(0, 5^2)$ prior on the log-clearance-ratio, reporting the full posterior and a sensitivity check against a $N(0, 1^2)$ and flat prior.

Figure 3.3 in Chapter 1 shows this convergence phenomenon directly: as sample size grows from $n=2$ to $n=100$, a fixed weakly informative prior is

progressively overwhelmed by the likelihood, and the posterior concentrates tightly around the true effect.

7.5.1 Example Result — Posterior Sensitivity Analysis

Prior on log-clearance-ratio	Posterior mean	95% credible interval	Conclusion changes?
Flat / non-informative	0.41	(-0.05, 0.87)	—
Weakly informative $N(0, 5^2)$	0.39	(-0.04, 0.82)	No
Weakly informative $N(0, 1^2)$	0.35	(-0.02, 0.72)	No
Moderately informative $N(0.2, 0.3^2)$	0.28	(0.03, 0.53)	Narrows CI, same direction

How this is obtained: The same model (`stan_glm`) is re-fit four times, varying only the prior on the coefficient of interest, holding the likelihood (data model) fixed.

Interpretation: The posterior mean and interval barely move across the flat, weakly informative, and second weakly informative priors — exactly the robustness a reviewer is checking for when they ask “how do you know your prior isn’t driving the result?” The fourth row, using a deliberately more informative prior, narrows the interval and shifts the mean slightly, but does not reverse the direction or overall conclusion. Presenting a table like this alongside any small-sample Bayesian analysis is the standard, concrete way to demonstrate — rather than merely assert — that a conclusion is not an artifact of prior choice.

```
library(rstanarm)
fit <- stan_glm(log_clearance ~ variant_carrier + age + weight, data = df,
               prior = normal(0, 5), chains = 4, iter = 2000)
posterior_interval(fit, prob = 0.95)
```

7.6 Image Preprocessing Pipeline (Raw → QC → Normalization → Segmentation → Feature Extraction → Modeling)

(1) Model Description: A sequential pipeline transforming raw acquired images (MRI/CT/WSI/microscopy) into a QC'd, harmonized, feature-ready input for statistical or ML modeling.

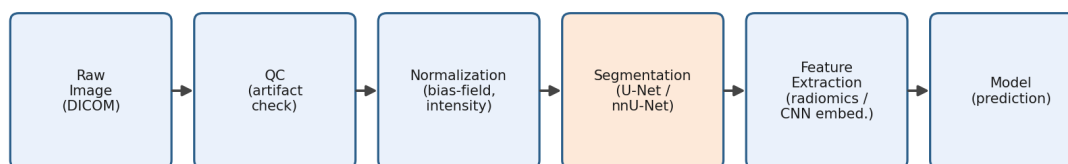


Figure 7.2: **The medical image preprocessing pipeline, raw to model-ready.** Each stage addresses a specific source of unwanted variation before the image can serve as a reliable model input; the segmentation stage (highlighted) is the step most likely to silently fail and should always be visually spot-checked.

Interpretation: Reading the pipeline left to right, each box removes one specific kind of nuisance variation: QC removes corrupted acquisitions before any time is spent processing them; normalization removes scanner- and site-specific intensity shifts that have nothing to do with biology; segmentation isolates the anatomical region actually relevant to the question; and feature extraction converts a segmented region into a fixed-length numeric vector a statistical or ML model can consume. A common mistake is treating this as a single monolithic “preprocessing” step — in practice, each arrow in the diagram is a place where an error introduced upstream (e.g., a failed segmentation) silently propagates into every feature computed downstream, which is why the segmentation stage is highlighted and always deserves a visual QC pass on a random sample of cases, not just an automated pass/fail flag.

(2) Mathematical Notation (representative steps):

- Intensity normalization (z-score per image/site): $x' = \frac{x - \mu}{\sigma}$
- Bias-field correction model (MRI, N4): observed intensity $I(v) = B(v) \cdot J(v) + n(v)$, where $B(v)$ is a smooth multiplicative bias field to be estimated/removed, $J(v)$ the true signal, $n(v)$ noise.
- Dice coefficient for segmentation QC: $\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$ (A = predicted mask, B = ground truth).

(3) Assumptions: Registration assumes a shared anatomical/spatial reference frame is meaningful across subjects (may be violated with severe pathology/deformation); intensity normalization assumes site/scanner differences are largely multiplicative/additive and correctable rather than reflecting genuine biological signal; segmentation models assume the training distribution (imaging protocol, contrast, resolution) matches the deployment distribution.

(4) Data Requirements: Consistent metadata (DICOM headers with scanner/protocol info) for harmonization; sufficient labeled masks (typically hundreds to thousands of annotated images) if training a custom segmentation network, or a validated pretrained model (nnU-Net) for common organs/structures; adequate resolution/field-of-view consistency across the cohort.

(5) Advantages: Standardizes heterogeneous acquisition into analysis-ready features; enables pooling multi-site data; radiomic features are interpretable (shape/texture), CNN embeddings capture richer patterns.

(6) Limitations: Each step (denoising, registration, segmentation) can introduce or propagate errors; deep-learning segmentation models can silently fail (fail on out-of-distribution scans) without obvious visual cues; radiomic features are sensitive to acquisition parameters even after normalization (a well-documented reproducibility issue in the radiomics literature).

(7) When to Use: Any project using radiology (CT/MRI/PET), digital pathology (WSI), or microscopy images as a modeling input, especially multi-site/multi-scanner studies.

(8) When NOT to Use step skipping: Never skip harmonization/QC in multi-site imaging studies — doing so risks a scanner/site batch effect masquerading as a biological signal; don't apply population-template registration when the population has highly heterogeneous anatomy without validating registration quality per subject.

Example: Harmonizing T1-weighted brain MRI across 20 UK Biobank Imaging acquisition sites using N4 bias correction + ComBat-GAM before extracting hippocampal volume as a feature for a dementia-risk model, cross-referenced with ADNI for external validation.

```
import SimpleITK as sitk
img = sitk.ReadImage("raw.nii.gz")
img = sitk.CurvatureFlow(img, timeStep=0.125, numberOfIterations=5) #
↪ denoise
corrector = sitk.N4BiasFieldCorrectionImageFilter()
img = corrector.Execute(img) #
↪ bias-field correction
```

```

img = sitk.Resample(img, reference_template) #
↪ registration
arr = sitk.GetArrayFromImage(img)
arr = (arr - arr.mean()) / arr.std() #
↪ normalization

```

7.6.1 Summary Table — Imaging Pipeline Stages

Stage	Purpose	Common Tools	Failure Mode if Skipped
QC	Detect corrupt/artifact-laden scans	MRIQC, FastQC-equivalents	Garbage-in-garbage-out downstream
Bias/denoise correction	Remove scanner-induced intensity artifact	N4ITK, Non-Local Means	Spurious intensity “signal” that’s actually scanner noise
Registration	Align to common anatomical space	ANTs, FSL FLIRT/FNIRT	Features not spatially comparable across subjects
Normalization	Remove scale/site differences	Z-score, ComBat-GAM	Site acts as a confound
Segmentation	Isolate region of interest	nnU-Net, Cellpose	Feature extraction on wrong/noisy region
Feature extraction	Convert pixels to model inputs	PyRadiomics, CNN embeddings	No usable input for modeling

7.6.2 Example Result — Extracted Radiomic Features (Simulated)

Patient ID	Site	Hippocampal		
		Hippocampal Volume (mm ³ , raw)	Hippocampal Volume (mm ³ , ComBat-GAM harmonized)	GLCM Contrast (texture)
P001	Site A	3412	3390	0.184
P002	Site A	3105	3086	0.201
P003	Site B	3560	3401	0.176

Patient ID	Site	Hippocampal	Hippocampal	GLCM
		Volume (mm ³ , raw)	Volume (mm ³ , ComBat-GAM harmonized)	Contrast (texture)
P004	Site B	2890	2755	0.223

How this is obtained: Raw hippocampal volume is computed after segmentation (nnU-Net); the harmonized column applies ComBat-GAM to remove the systematic Site A/Site B scanner offset while preserving each patient’s relative standing within their site.

Interpretation: Before harmonization, Site B volumes appear systematically different from Site A’s, which could easily be mistaken for a genuine biological difference (e.g., disease-related atrophy) if the two sites happened to have different case mixes. After ComBat-GAM harmonization, the site-driven shift is largely removed while each patient’s relative volume (compared to same-site peers) is preserved — this is exactly the harmonization step warned about in the pitfalls above, and skipping it in a multi-site study risks attributing a scanner artifact to a disease effect.

7.7 Multimodal Modeling (Imaging + Omics + EHR)

(1) Model Description: Joint modeling framework that combines modality-specific representations (imaging embeddings, omics features, EHR time series) to predict a shared clinical outcome, via early, late, or intermediate/joint fusion (see Module 6.2 for the full pipeline diagram).

(2) Mathematical Notation: Joint objective with optional cross-modal alignment term:

$$\mathcal{L} = \mathcal{L}_{task}(\hat{y}, y) + \lambda \mathcal{L}_{contrastive}(z_{img}, z_{omics}, z_{ehr})$$

where $\mathcal{L}_{task}$ is e.g. cross-entropy or Cox partial likelihood, and $\mathcal{L}_{contrastive}$ (Module 6.2 formula) encourages aligned embeddings for the same patient across modalities.

(3) Assumptions: Modalities collected on (at least partially) the same patients with a defined temporal relationship (e.g., imaging and omics collected within a defined window of the EHR outcome); modality-specific encoders assume enough within-modality signal to learn a useful embedding; fusion assumes the outcome is genuinely influenced by more than one modality (otherwise a unimodal model may perform as well with less complexity).

(4) Data Requirements: Sufficient patients with **all** modalities present for joint/intermediate fusion (late fusion is more forgiving of missingness); consistent patient-level linkage keys across data sources; harmonized preprocessing per modality (Module 6.1 for imaging, standard omics QC for -omics, structured/coded EHR fields, e.g., ICD/CPT/LOINC).

(5) Advantages: Can outperform any single modality by capturing complementary information (e.g., imaging shows structural change, omics shows the molecular driver, EHR shows clinical trajectory); more clinically holistic; foundation-model embeddings increasingly reduce the feature-engineering burden per modality.

(6) Limitations: Requires larger, more complex datasets with multi-modal linkage (rarer and more expensive to assemble); missing-modality handling adds modeling complexity; harder to interpret than a unimodal model; risk that one high-dimensional modality dominates learning unless carefully balanced/regularized.

(7) When to Use: Precision medicine questions where no single data type is expected to be sufficient (e.g., cancer prognosis combining radiology + genomics + labs); biobank-scale studies with rich multi-modal linkage (UK Biobank, TCGA+TCIA, MIMIC).

(8) When NOT to Use: Small cohorts where multi-modal linkage sample size collapses to a fraction usable for joint fusion (better to model modalities separately or use late fusion); when one modality alone already achieves near-ceiling performance (added complexity isn't justified) — always benchmark against the best unimodal model first.

Example: Combining chest CT radiomic features, blood-based inflammatory biomarkers, and EHR comorbidity history to predict COPD exacerbation risk within 90 days, benchmarked against an EHR-only baseline to confirm imaging/omics add incremental value (e.g., via DeLong's test (DeLong, DeLong, & Clarke-Pearson, 1988) on AUCs).

Interpretation: The key design choice visible in the diagram is that each modality gets its own encoder *before* fusion — an imaging CNN, an omics autoencoder, and an EHR transformer are each trained (or fine-tuned) to compress their respective modality into a compact embedding, rather than trying to learn a single model directly on concatenated raw features from all three sources at once. This matters because imaging, omics, and EHR data have completely different native dimensionalities, noise structures, and missingness patterns; forcing them all through the same first layer (true early fusion) tends to let whichever modality has the most raw features dominate the representation, regardless of how informative it actually is. The fusion layer is where the

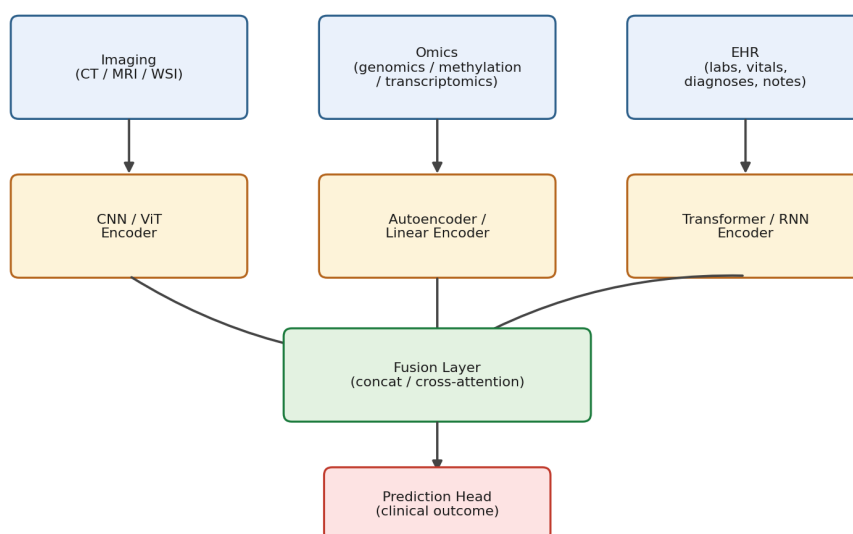


Figure 7.3: **Intermediate/joint multimodal fusion architecture.** Each modality (imaging, omics, EHR) is first passed through its own encoder to produce a fixed-length embedding; the three embeddings are then combined in a fusion layer (concatenation or cross-attention) before a shared prediction head produces the final clinical output.

model actually learns how to weigh and combine the three modality-specific summaries.

```
# Late-fusion benchmark: compare AUC of unimodal vs fused model
from sklearn.metrics import roc_auc_score
auc_ehr_only = roc_auc_score(y_test, model_ehr.predict_proba(X_ehr_test)[: ,1])
auc_fused     = roc_auc_score(y_test,
    ↪ model_fused.predict_proba(X_fused_test)[: ,1])
```

7.7.1 Example Result — Unimodal vs. Fused Model Benchmark (Simulated)

Model	AUC	95% CI	Δ AUC vs. EHR-only
EHR only	0.71	(0.66, 0.76)	—
Imaging (CT radiomics) only	0.68	(0.63, 0.73)	-0.03
Omics (inflammatory panel) only	0.65	(0.59, 0.71)	-0.06

Model	AUC	95% CI	Δ AUC vs. EHR-only
Late fusion (EHR + imaging + omics)	0.77	(0.72, 0.82)	+0.06 (p=0.01, DeLong)

How this is obtained: Each unimodal model is trained and validated identically (same folds, same outcome definition); the fused model combines their predicted probabilities via a simple logistic stacking layer, and DeLong’s test compares the fused AUC against the strongest unimodal baseline (EHR-only).

Interpretation: No single modality alone reaches the fused model’s performance — this is the empirical justification for the added complexity of multimodal fusion described in the Advantages/Limitations above. The DeLong test’s p-value confirms the +0.06 AUC improvement is unlikely to be due to chance alone, which is exactly the kind of benchmark a technical reviewer will ask for before accepting that multimodal fusion was worth the added data-collection and modeling complexity, per point (8) above.

7.8 Ensemble Models (XGBoost, LightGBM, Random Forest)

(1) Model Description: Tree-based ensembles that combine many weak learners (decision trees) via bagging (Random Forest) or boosting (XGBoost/LightGBM) to produce a strong predictive model.

(2) Mathematical Notation: Random Forest: $\hat{f}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$, each T_b trained on a bootstrap sample with random feature subsampling at each split. Gradient boosting (additive model): $F_m(x) = F_{m-1}(x) + \eta h_m(x)$, $h_m = \arg \min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i))$ XGBoost regularized objective: $\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$, $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ (T =number of leaves, w =leaf weights).

Interpretation: The structural difference in the diagram explains the two families’ different strengths. Because bagging’s trees are fit independently, they can be trained in parallel and tend to be robust to overfitting (averaging cancels out individual trees’ noise) but can’t correct each other’s systematic mistakes. Because boosting’s trees are fit sequentially, each one explicitly targets whatever the ensemble so far got wrong — which is why boosting often achieves higher accuracy, but also why it requires more careful regularization (learning rate η , tree depth, number of rounds) to avoid eventually overfitting

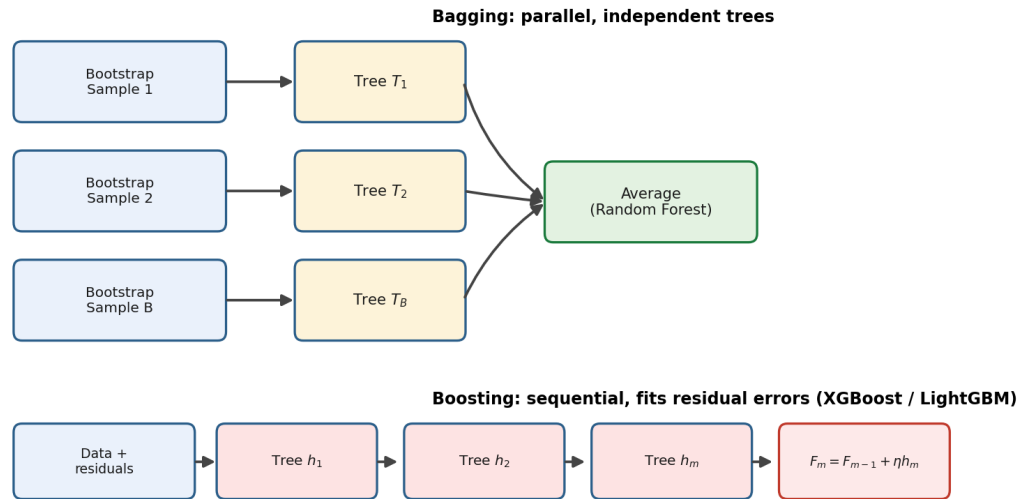


Figure 7.4: **Bagging versus boosting.** *Top:* Random Forest (bagging) fits B trees independently and in parallel to different bootstrap samples, then averages their predictions. *Bottom:* gradient boosting (XGBoost/LightGBM) fits trees sequentially, with each new tree specifically targeting the residual error left by the previous ensemble.

to noise in the residuals.

(3) Assumptions: No linearity assumption (trees handle nonlinearity/interactions natively); features can be of mixed type; Random Forest assumes bootstrap resampling approximates the underlying population; boosting assumes sequential residual-fitting converges without excessive overfitting (controlled via learning rate/early stopping); trees assume enough samples per split to estimate reliable splits (deep trees on tiny n overfit).

(4) Data Requirements: Tabular data (rows = samples, columns = features); handles missing values natively (XGBoost/LightGBM) or via imputation (RF); no strict minimum n , but reliable performance typically needs n in the hundreds+ with a $p \gg n$ regime requiring strong regularization/cross-validation; categorical features handled natively by LightGBM, need encoding for RF/XGBoost (or use `enable_categorical` in newer XGBoost).

(5) Advantages: High predictive accuracy on tabular/omics data with minimal preprocessing; built-in feature importance and (for tree-based models) fast, exact SHAP computation; robust to outliers and mixed feature scales; handle nonlinear interactions automatically.

(6) Limitations: Prone to overfitting with too many boosting rounds/too-deep trees without regularization/early stopping, especially in $p \gg n$ omics settings; less interpretable than linear models without post-hoc tools (SHAP); per-

formance on very high-dimensional sparse data (e.g., raw genomic sequence) is typically worse than deep learning designed for that structure; boosting is sequential (harder to parallelize than RF's independent trees, though LightGBM/XGBoost have engineering workarounds).

(7) When to Use: Tabular clinical/omics/EHR feature data with a moderate-to-large number of engineered features; when nonlinear interactions are expected and pure interpretability isn't the sole requirement; competition-grade prediction tasks.

(8) When NOT to Use: Very high-dimensional, spatially/sequentially structured raw data better suited to CNNs/transformers (raw images, raw sequence); extremely small n (<50 -100) where any complex model overfits — prefer regularized linear models; when full model transparency (a single readable equation) is a hard regulatory/clinical requirement rather than post-hoc explainability.

Example: XGBoost model predicting drug response from a panel of 200 engineered pharmacogenomic + clinical features in a Phase II trial ($n \approx 500$), validated with 5×5 nested cross-validation and interpreted via SHAP.

```
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold, cross_val_score
model = xgb.XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.05,
                          subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")
```

7.8.1 Example Result — Ensemble Comparison on Simulated Omics Data ($n=500$, $p=200$)

Model	Mean CV AUC	SD across folds	Training time (relative)
Elastic Net (baseline)	0.74	0.03	1×
Random Forest	0.79	0.04	3×
XGBoost	0.82	0.05	4×
LightGBM	0.81	0.05	2×

How this is obtained: All four models are evaluated with the same outer 5-fold stratified cross-validation split, with hyperparameters tuned only within an inner CV loop (nested CV, as described in the Pitfalls section) to avoid optimistic bias.

Interpretation: Both boosting methods outperform Random Forest by

roughly 2-3 AUC points, consistent with the sequential error-correction property shown in the figure above, at the cost of somewhat higher fold-to-fold variability (SD 0.05 vs. 0.04) and longer training time. The Elastic Net baseline, while lower in raw AUC, remains a useful reference point: the ~ 0.08 AUC gap between it and XGBoost quantifies how much nonlinear/interaction signal the tree ensembles are capturing that a linear model cannot — information worth reporting alongside the headline AUC, since it tells a reviewer whether the added model complexity is actually earning its keep.

7.9 Deep Learning (CNN, RNN, LSTM, Transformers)

(1) Model Description: Hierarchical, differentiable models that learn representations directly from raw or minimally processed data (images, sequences, text) via layers of learned transformations, trained end-to-end via backpropagation.

(2) Mathematical Notation: - **CNN** convolution: $(f * g)(t) = \sum_{\tau} f(\tau) g(t - \tau)$; a conv layer output: $z^{(l)} = \sigma(W^{(l)} * a^{(l-1)} + b^{(l)})$ - **RNN** hidden-state update: $h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ - **LSTM** gating (repeated for completeness): $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$, $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, $h_t = o_t \odot \tanh(C_t)$ - **Transformer** (Vaswani et al., 2017) self-attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$, with $Q = XW_Q$, $K = XW_K$, $V = XW_V$.

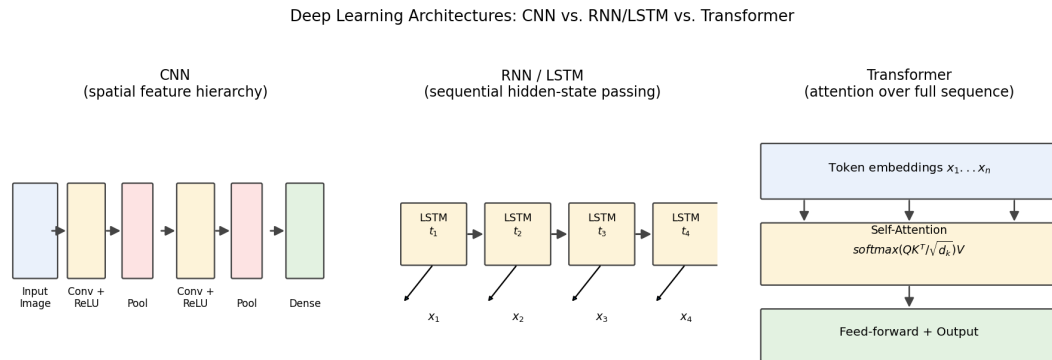


Figure 7.5: **Three deep learning architectures compared.** *Left:* a CNN builds up spatial features through alternating convolution and pooling layers. *Middle:* an RNN/LSTM processes a sequence one time step at a time, passing a hidden state forward. *Right:* a Transformer computes self-attention over the entire input sequence at once, letting every token directly attend to every other token.

Interpretation: The structural difference in how each architecture handles its input directly predicts what kind of data it suits. A CNN’s convolution-and-pooling stack builds increasingly large receptive fields layer by layer, which is why it’s the natural choice for spatial data like a radiology image, where nearby pixels are far more related than distant ones. An RNN/LSTM processes its input strictly in order, one step at a time, carrying a hidden-state summary forward — a natural fit for a patient’s chronologically ordered lab and vital-sign history, but one that must pass information through many sequential steps to connect a very early event to a very late outcome. A Transformer removes this sequential bottleneck entirely: every position attends directly to every other position in a single attention operation, which is why transformers excel when long-range dependencies matter (e.g., relating a genetic variant to a distant regulatory element), at the cost of needing more data and compute to learn effectively, since it has fewer built-in assumptions about the data’s structure than a CNN or RNN.

(3) Assumptions: Sufficient labeled training data relative to model capacity (deep nets have millions of parameters); i.i.d. or well-understood dependency structure between train/validation/test splits at the correct unit of independence (patient-level, not scan-level); stationarity assumptions for RNN/LSTM (the same dynamics apply across the sequence, though LSTMs relax this somewhat via gating); transformers assume enough data/compute for effective pre-training or transfer learning, since they lack strong built-in inductive biases (no convolutional locality assumption).

(4) Data Requirements: Large datasets (thousands to millions of samples) for training from scratch; transfer learning/pretrained foundation models (e.g., ImageNet-pretrained CNNs, ESM2, scGPT) substantially reduce the labeled-data requirement; GPU/TPU compute; consistent, well-QC’d raw input format (fixed image size, tokenized sequence, etc.).

(5) Advantages: State-of-the-art performance on images, sequences, and unstructured text; automatic feature learning (no manual feature engineering); transformers/attention capture long-range dependencies and support transfer learning at scale via foundation models.

(6) Limitations: Data-hungry — poor performance on small clinical-trial-sized datasets without transfer learning; computationally expensive to train; “black box” — requires post-hoc interpretability tools (Grad-CAM for CNNs, attention weights or SHAP for transformers) and even then interpretability is imperfect; prone to overfitting and to learning spurious shortcuts (e.g., scanner artifacts) without careful validation.

(7) When to Use: Raw image data (radiology, pathology, microscopy); raw sequence data (DNA/protein sequence, clinical note text); large EHR/biobank-

scale longitudinal data; whenever a strong pretrained foundation model exists for transfer learning.

(8) When NOT to Use: Small tabular clinical/omics datasets (hundreds of samples, tens-hundreds of features) where a regularized linear model or tree ensemble will perform as well or better with far more interpretability and less overfitting risk; settings requiring fully transparent, auditable models for regulatory submission without a validated interpretability layer.

Example: A CNN (transfer-learned from ImageNet, fine-tuned) classifying diabetic retinopathy severity from retinal fundus images; an LSTM predicting ICU deterioration from MIMIC-IV vital-sign time series; a transformer-based protein language model (ESM2) generating embeddings used as omics features for variant-effect prediction.

```
import torch.nn as nn
class SimpleCNNClassifier(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Linear(64*56*56, num_classes)  # for 224x224 input
    def forward(self, x):
        x = self.features(x).flatten(1)
        return self.classifier(x)
```

7.9.1 Comparison Table — DL Architecture Selection

Architecture	Best For	Key Limitation
CNN	Images, spatial grid data (radiology, pathology, genomic sequence motifs via 1D-CNN)	Fixed receptive field unless deep/dilated
RNN/LSTM	Sequential/time-series (EHR trajectories, longitudinal labs)	Sequential computation, harder to parallelize, can struggle with very long-range dependencies

Architecture	Best For	Key Limitation
Transformer	Long-range dependencies, large-scale pretraining (protein/DNA/text)	Data/compute hungry, quadratic attention cost in sequence length

7.9.2 Example Result — Architecture Choice on a Chest X-ray Classification Task (Simulated)

Architecture	Training data used	Test AUC	Notes
CNN, trained from scratch	2,000 labeled images	0.71	Limited by small labeled set
CNN, ImageNet-pretrained + fine-tuned	2,000 labeled images	0.86	Transfer learning recovers most of the gap
Vision Transformer, trained from scratch	2,000 labeled images	0.64	Underperforms without large-scale pretraining
Vision Transformer, pretrained + fine-tuned	2,000 labeled images	0.87	Matches/exceeds pretrained CNN once pretrained

Interpretation: With only 2,000 labeled images, a transformer trained entirely from scratch clearly underperforms a similarly-sized CNN — exactly as predicted by the architectural comparison above, since transformers lack the convolutional locality assumption that lets a CNN learn useful spatial filters from less data. Once both architectures are initialized from a large-scale pre-trained model and only fine-tuned on the 2,000 local images, the gap disappears entirely. The practical lesson for a small clinical imaging dataset: architecture choice matters far less than whether a suitable pretrained foundation model is available and used.

7.10 Explainable AI (SHAP, LIME)

(1) Model Description: Post-hoc, model-agnostic (or model-specific, for SHAP’s tree/deep variants) techniques that attribute a model’s prediction to

its input features, making “black-box” models interpretable.

(2) Mathematical Notation: SHAP (Shapley value):

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

satisfying **local accuracy** ($\sum_i \phi_i = f(x) - E[f(x)]$), **missingness**, and **consistency**. LIME: fits a local surrogate $g \in G$ (e.g., sparse linear model) minimizing $\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$, where π_x weights perturbed samples by proximity to x and $\Omega(g)$ penalizes surrogate complexity.

(3) Assumptions: SHAP’s exact computation (TreeSHAP) assumes access to the tree structure; Kernel SHAP (model-agnostic) assumes feature independence for its sampling approximation unless using more advanced conditional variants — a commonly violated assumption with correlated omics features; LIME assumes local linearity is a reasonable approximation of the model’s behavior in the neighborhood of x , and that the chosen perturbation/kernel width is appropriate.

(4) Data Requirements: Needs the trained model plus a representative background/reference dataset (for SHAP’s baseline expectation); no additional labels required beyond what trained the original model; computation scales with number of features and (for Kernel SHAP) number of background samples.

(5) Advantages: SHAP provides theoretically grounded, consistent, locally accurate attributions; works across model classes (tree ensembles, deep nets, linear models) with specialized fast implementations for trees; LIME is fast, intuitive, and easy to explain to non-technical stakeholders.

(6) Limitations: Both reflect model behavior, not ground-truth causality; correlated features split/dilute attribution; LIME can be unstable across repeated runs (different random perturbations can give different local explanations); Kernel SHAP’s feature-independence approximation can mislead when features are highly correlated (common in omics data); computationally expensive for very high-dimensional models without an efficient exact method.

(7) When to Use: Any deployed ML model in a clinical/pharma setting requiring transparency for clinicians, regulators, or scientific collaborators; biomarker prioritization as one line of evidence (paired with independent validation); model debugging (detecting reliance on a spurious/leaky feature).

(8) When NOT to Use as sole evidence: Claiming causal/mechanistic importance of a feature based on SHAP/LIME alone without independent biological or causal-inference validation (Mendelian Randomization, mediation, replication); using LIME’s local linear approximation to characterize genuinely

highly nonlinear/non-local model behavior.

Example: Using TreeSHAP on an XGBoost sepsis-risk model trained on MIMIC-IV vitals/labs to show clinicians which features (e.g., rising lactate, falling MAP) are driving an individual patient’s risk score in real time.

```
import shap
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
shap.plots.waterfall(explainer(X_test)[0]) # single-patient explanation
```

7.11 scRNA-seq Workflows (Seurat, Scanpy)

(1) Model Description: A multi-step analytical pipeline that converts raw single-cell RNA-seq count matrices into normalized, dimensionality-reduced, clustered, and biologically annotated cell populations.

(2) Mathematical Notation: Normalization (log-normalize):

$$x'_{ij} = \log \left(1 + \frac{x_{ij}}{\sum_j x_{ij}} \times 10,000 \right)$$

PCA objective (dimensionality reduction): find orthogonal directions w_k maximizing $\text{Var}(Xw_k)$ subject to $\|w_k\| = 1$ and orthogonality to prior components.

Graph-based clustering (Leiden) optimizes modularity: $Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$, where A_{ij} is the shared-nearest-neighbor graph adjacency, k_i node degree, c_i cluster assignment.

(3) Assumptions: UMI counts approximately follow a negative-binomial/Poisson-like distribution (motivating log-normalization and variance-stabilizing transforms like SCTransform); highly variable gene selection assumes biological variation of interest is captured in the most variable genes after accounting for mean-variance trend; clustering assumes discrete or continuum cell states are separable in reduced PCA space; batch-integration methods (Harmony, scVI) assume batch effects are separable from biological variation of interest.

(4) Data Requirements: Raw UMI count matrix (genes $\times$ cells) typically from 10x Genomics (or Smart-seq2 for full-length reads); recommended minimum a few hundred to a few thousand cells per condition/sample for robust cluster detection; sufficient sequencing depth per cell (typically $>20,000$ reads/cell for standard 10x protocols); multiple samples/batches for integration if compar-

ing across conditions.

(5) Advantages: Reveals cellular heterogeneity invisible to bulk RNA-seq (rare cell populations, cell-state transitions); well-supported, mature open-source ecosystems (Seurat in R, Scanpy in Python) with extensive tutorials; increasingly supported by foundation models (scGPT, Geneformer) for transfer learning and batch-robust embeddings.

(6) Limitations: High technical noise/dropout (zero-inflation) per cell; ambient RNA contamination and doublets can create spurious “hybrid” cell types; batch effects across runs/samples require careful integration (naive pooling creates false clusters); clustering resolution is a user-chosen parameter without a single “correct” answer, requiring biological validation.

(7) When to Use: Characterizing cellular composition/heterogeneity of a tissue in health vs. disease; identifying rare or novel cell populations; trajectory/differentiation studies; spatial transcriptomics integration.

(8) When NOT to Use: When the biological question concerns average tissue-level expression and bulk RNA-seq would answer it more cheaply and with less technical noise; extremely limited cell numbers (a few dozen cells) where cluster-level statistical power is inadequate; when budget/sequencing depth cannot support adequate UMI/cell counts, risking uninterpretable technical noise dominating the signal.

Example: Characterizing airway epithelial cell-type shifts (ciliated, goblet, basal cells) between asthmatic and healthy pediatric bronchial brushings using 10x Genomics scRNA-seq, integrated across subjects with Harmony, annotated via canonical marker genes (e.g., *FOXJ1* for ciliated cells, *MUC5AC* for goblet cells).

```
library(Seurat)
obj <- CreateSeuratObject(counts = raw_counts, min.cells = 3, min.features = 200)
obj[["percent.mt"]] <- PercentageFeatureSet(obj, pattern = "^MT-")
obj <- subset(obj, subset = nFeature_RNA > 200 & percent.mt < 10)
obj <- NormalizeData(obj) |> FindVariableFeatures(nfeatures = 2000) |>
  ↪ ScaleData()
obj <- RunPCA(obj) |> RunHarmony("sample_id") |> RunUMAP(reduction="harmony",
  ↪ dims=1:20)
obj <- FindNeighbors(obj, reduction="harmony", dims=1:20) |>
  ↪ FindClusters(resolution=0.5)
FeaturePlot(obj, features = c("FOXJ1", "MUC5AC")) # marker validation

import scanpy as sc
adata = sc.read_10x_mtx("filtered_feature_bc_matrix/")
sc.pp.filter_cells(adata, min_genes=200)
```

```

sc.pp.calculate_qc_metrics(adata, qc_vars=["mt"], inplace=True)
adata = adata[adata.obs.pct_counts_mt < 10]
sc.pp.normalize_total(adata, target_sum=1e4); sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=2000)
sc.tl.pca(adata); sc.external.pp.harmony_integrate(adata, "sample_id")
sc.pp.neighbors(adata, use_rep="X_pca_harmony"); sc.tl.leiden(adata);
↪ sc.tl.umap(adata)

```

7.12 Variant Calling Pipelines (GATK, bcftools, SAMtools)

(1) Model Description: Statistical/algorithmic pipelines that identify genomic variants (SNVs, indels, and with extensions, CNVs/SVs) from aligned sequencing reads by modeling the probability of each possible genotype given observed base calls.

(2) Mathematical Notation: GATK's HaplotypeCaller genotype likelihood (simplified diploid model): for genotype $G \in \{AA, AB, BB\}$ and observed reads D ,

$$P(D \mid G) = \prod_{r \in D} P(r \mid G)$$

with per-read probability incorporating base quality (error probability ϵ from Phred score, $Q = -10 \log_{10} \epsilon$). Posterior genotype probability (Bayesian genotyping): $P(G \mid D) \propto P(D \mid G) P(G)$, with $P(G)$ often from a population-genetics prior (e.g., HWE-based).

(3) Assumptions: Reads are correctly aligned to the reference (alignment errors directly bias variant calls); base-quality scores accurately reflect true sequencing error rates (motivating BQSR — base quality score recalibration); diploid genotype model assumes exactly two alleles per locus per individual (violated in copy-number-variable regions, mosaicism, or pooled/tumor samples, which need specialized somatic callers like Mutect2); joint genotyping assumes samples are appropriately batched with comparable sequencing characteristics.

(4) Data Requirements: Aligned, sorted, duplicate-marked BAM/CRAM files; adequate sequencing depth (typically $\geq 30\times$ for germline whole-genome, $\geq 100\text{--}500\times$ for targeted panels/liquid biopsy to detect low-frequency somatic variants); a reference genome with consistent build (GRCh37/GRCh38) across the entire cohort; ideally a validated truth set (e.g., Genome in a Bottle) for pipeline benchmarking.

(5) Advantages: GATK Best Practices represents a widely validated, community-standard workflow with strong documentation and broad clini-

cal/research adoption; bcftools/SAMtools are fast and lightweight for large-cohort processing; joint genotyping across a cohort improves sensitivity for rare variants by sharing information across samples.

(6) Limitations: Computationally intensive at scale (BQSR, HaplotypeCaller are resource-heavy); sensitive to reference build mismatches and alignment artifacts; standard diploid callers underperform in complex regions (segmental duplications, low-complexity repeats) and in somatic/mosaic contexts without specialized tools; VQSR requires a sufficiently large cohort to train reliable variant-quality models (poorly calibrated in small cohorts, where hard-filtering is often preferred).

(7) When to Use: Germline variant discovery in cohort or trio studies (GATK Best Practices); rapid variant calling/filtering in large-cohort or resource-constrained settings (bcftools); any project requiring BAM/CRAM manipulation, indexing, or basic alignment QC (SAMtools); genomic interval operations for annotation overlap (BEDTools alongside these).

(8) When NOT to Use: Somatic/tumor variant calling with a standard germline diploid caller (use Mutect2 or a dedicated somatic caller that models tumor purity/subclonality instead); structural variant/CNV detection with SNV-focused callers (use dedicated tools: Manta, DELLY, CNVkit); very low-coverage sequencing (<10x) with standard hard filters, without accounting for the resulting loss of genotyping confidence (favor imputation-aware or genotype-likelihood-based joint calling instead).

Example: GATK Best Practices germline joint-genotyping of a 50-trio pediatric asthma cohort to identify de novo and inherited rare variants, followed by bcftools-based hard filtering and VEP annotation for downstream burden testing.

The full pipeline diagram (Figure 1.2) and an example trio VCF summary table (Ts/Tv ratio, Mendelian error rate) appear in Chapter 2, Section “Genomic Data Processing” — the same QC logic applies whether variant calling is done with GATK’s HaplotypeCaller or with bcftools.

GATK Best Practices (germline, per-sample then joint)

```
gatk BaseRecalibrator -R ref.fa -I sample.bam --known-sites known_sites.vcf.gz -O
↪ recal.table
gatk ApplyBQSR -R ref.fa -I sample.bam --bqsr-recal-file recal.table -O
↪ sample.recal.bam
gatk HaplotypeCaller -R ref.fa -I sample.recal.bam -O sample.g.vcf.gz -ERC GVCF
gatk GenomicsDBImport --genomicsdb-workspace-path db -V sample1.g.vcf.gz -V
↪ sample2.g.vcf.gz -L intervals.list
gatk GenotypeGVCFs -R ref.fa -V gendb://db -O joint.vcf.gz
```

```
gatk VariantFiltration -R ref.fa -V joint.vcf.gz --filter-expression "QD<2.0"
↪ --filter-name QD2 -O joint.filtered.vcf.gz
```

```
# bcftools alternative (fast, large cohorts)
```

```
bcftools mpileup -f ref.fa -Ou sample.bam | bcftools call -mv -Oz -o sample.vcf.gz
bcftools norm -f ref.fa -m -both sample.vcf.gz -Oz -o sample.norm.vcf.gz
```

7.13 A Quick Guide to This Book’s Mathematical Notation

Before comparing models by sample size, it is worth collecting the recurring symbols used throughout this book in one place, so that any formula can be read at a glance without hunting back through earlier chapters.

Symbol	Meaning
n, p	Sample size (number of subjects/observations); number of features/covariates
β, θ	A model’s fixed/unknown parameter(s) to be estimated (a coefficient, or a general parameter)
$\hat{\beta}, \hat{\theta}$	An estimate of that parameter, computed from data
σ^2, Σ	Variance of a single quantity; a covariance matrix for multiple correlated quantities
b_i	A subject-specific random effect (Chapter 1), distinct from the population-level fixed effect β
$P(\cdot), P(\cdot \mid \cdot)$	Probability; conditional probability (“probability of the left side, given the right side”)
$E[\cdot]$	Expected value (long-run average) of a quantity
$L(\theta \mid D)$	Likelihood: how probable the observed data D is, as a function of candidate parameter values θ
$h(t), S(t)$	Hazard function and survival function at time t (Chapter 3) — instantaneous event rate, and probability of remaining event-free, respectively

Symbol	Meaning
$\arg \min, \arg \max$	“The value of the input that minimizes/maximizes the following expression” — used to define what a model’s fitting procedure is actually solving for
$\propto$	“Proportional to” — used in Bayes’ theorem to mean the posterior has the same shape as the right-hand side, up to a normalizing constant
$\sum, \prod$	Summation and product across an index (e.g., across all subjects, all features, or all trees in an ensemble)
$f(x; \theta)$	A model, read as “a function of input x , governed by parameters θ ” — the same shorthand used for everything from a linear predictor to a full neural network

With this legend in hand, the sample-size guidance below can freely reuse n , p_{eff} , β , and $L(\theta \mid D)$ without re-defining them each time.

7.14 Model Selection by Sample Size: Small-n vs. Large-n Regimes

A recurring, practical question this book has touched on repeatedly in individual chapters – “is my sample big enough for this model?” – deserves its own concise treatment, since the underlying reasoning is the same across very different model families.

7.14.1 The Core Idea: Effective Parameters vs. Available Data

Every model has some number of quantities it must estimate from data – call it its **effective number of parameters**, p_{eff} (for a linear model this is simply the number of coefficients; for a tree ensemble it is related to the number of leaves across all trees; for a neural network it is the literal parameter count, often in the millions or billions). The single most important sample-size heuristic in this book can be stated as a ratio:

$$\text{Data sufficiency ratio} = \frac{n}{p_{eff}}$$

A useful rule of thumb across classical statistics is that this ratio should be comfortably above 10-20 for stable, generalizable estimates (e.g., “10 events per parameter” for a Cox model, Chapter 3); deep learning models routinely operate with $p_{eff} \gg n$ and only avoid overfitting because of implicit and explicit regularization (dropout, weight decay, early stopping, data augmentation) and, critically, because n itself is typically enormous in absolute terms even when p_{eff} is larger still.

7.14.2 Why Decision Trees, Random Forests, and XGBoost Work Well for Small Samples

A single decision tree, and the ensembles built from it (Random Forest, XGBoost, LightGBM; Chapter 4), have three properties that make them comparatively data-efficient:

1. **Greedy, local parameter estimation.** A tree only estimates a small number of quantities *at each split* (a single threshold on a single feature), not a large joint parameter vector fit simultaneously across all features the way a linear or neural model does – so a modest n can still support many reasonable splits, one at a time.
2. **Built-in variance reduction via bagging.** Random Forest’s aggregation over B bootstrap-trained trees directly reduces variance:

$$\text{Var}(\hat{f}_{RF}) \approx \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

where σ^2 is a single tree’s variance and ρ is the correlation between trees. Averaging many decorrelated, individually noisy (high-variance, low-bias) trees trained on a small dataset yields a much lower-variance ensemble than any single tree could – without needing more data, only more trees.

3. **Regularized sequential fitting.** XGBoost/LightGBM’s boosting objective explicitly penalizes tree complexity (Chapter 4’s $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$), and the learning rate η further limits how much any single tree can move the fitted function – both act as a brake on overfitting that does not require additional data to be effective.

Example: A pharmacogenomic biomarker panel with $n = 180$ patients and $p = 40$ engineered features is a very reasonable setting for Random Forest or XGBoost (with cross-validation for hyperparameter tuning, Chapter 4) but a poor setting for a neural network trained from scratch.

7.14.3 Why Deep Learning Requires Large Samples

A modern CNN or transformer (Chapter 4) has p_{eff} in the millions to billions – vastly exceeding any biomedical cohort’s sample size on its own terms. Two related facts explain why this becomes a genuine overfitting risk rather than just a large-but-harmless number:

- **Capacity/expressiveness.** A model family’s capacity to fit arbitrary patterns (formalized classically via VC dimension or, more practically, effective parameter count) grows with p_{eff} ; a sufficiently high-capacity model can drive **training loss to zero while learning noise** rather than signal, if n is too small relative to that capacity.
- **The bias-variance decomposition** (Chapter 4) makes the mechanism explicit:

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

A high-capacity model can achieve very low bias, but with small n its variance term dominates – the fitted function changes drastically depending on exactly which samples happened to be in the training set, which is the definition of overfitting.

This is precisely why Chapter 4 and Chapter 6 recommend deep learning only when n is large (thousands to millions) *or* when transfer learning from a model pretrained on a much larger external corpus is available (Chapter 6’s foundation models) – borrowing p_{eff} ’s effective “prior knowledge” from that larger pretraining sample size rather than needing an equivalently large *local* dataset.

Example: A CNN trained from scratch on 300 local pathology slides will typically overfit badly; the same architecture, initialized from a pathology foundation model (Chapter 6) pretrained on hundreds of thousands of slides and only fine-tuned on the local 300, is a fundamentally different (and much more viable) proposition.

7.14.4 When Bayesian Models Are Preferred for Small Samples

As established in Chapter 1 and Chapter 3, a Bayesian posterior $P(\theta \mid D) \propto L(\theta \mid D)P(\theta)$ is **always well-defined**, regardless of how small n is – there is no asymptotic requirement for the machinery itself to produce an answer. This makes Bayesian methods a natural fit whenever n is small by necessity rather than by design: rare-disease trials, rare-variant genetic association, or early-phase studies with a handful of patients. A weakly informative prior (Chapter 1) still lets the posterior reflect appropriately wide uncertainty rather than forcing

a fragile point estimate and p-value.

7.14.5 When Frequentist Models Need Large Samples

Most standard frequentist inference (Wald tests, MLE-based confidence intervals) leans on **asymptotic** guarantees – properties proven to hold as $n \rightarrow \infty$:

$$\sqrt{n}(\hat{\theta}_{MLE} - \theta) \xrightarrow{d} N(0, I(\theta)^{-1})$$

This asymptotic normality result is what justifies the familiar “estimate $\pm 1.96 \times \text{SE}$ ” confidence interval – but it is only a good approximation once n is large enough for the estimator’s true sampling distribution to have converged close to this normal limit. With small n or rare events, this approximation can be poor, motivating exact small-sample methods or, per the section above, a Bayesian approach that does not rely on this large-sample limit at all.

7.14.6 When Mixed-Effects Models Outperform Simple Regression for Repeated Measures

This is a sample-size question of a different kind: it is not about *how many subjects* n you have, but about **how many independent pieces of information** your data actually contain once repeated measurements are accounted for (Chapter 1). Simple regression implicitly (and incorrectly) treats $n_{\text{subjects}} \times n_{\text{visits}}$ repeated measures as if they were that many *independent* observations, artificially inflating the apparent sample size and understating standard errors. A mixed-effects model

$$y_{ij} = X_{ij}\beta + Z_{ij}b_i + \epsilon_{ij}$$

correctly partitions variance into within- and between-subject components, so its effective degrees of freedom reflect the true, smaller amount of independent information – mixed models outperform simple regression whenever data are clustered or repeated **regardless of whether total n is small or large**, because the problem being corrected (pseudoreplication) does not go away with more repeated measurements on the same subjects; it only goes away with more independent subjects.

7.14.7 Summary Table — Model Selection by Sample Size

Sample Size Regime	Recommended Models	Why
Very small ($n < 50$ -100, or rare events)	Bayesian models with weakly informative priors; exact (non-asymptotic) frequentist tests	Posterior always well-defined; asymptotic frequentist guarantees do not yet hold
Small-to-moderate (n in the hundreds, p moderate-large)	Elastic Net / LASSO; Random Forest; XGBoost/LightGBM with nested CV	Data-efficient: local/greedy split estimation, bagging/boosting variance control, explicit regularization
Moderate, repeated/clustered measures	Linear mixed-effects models; GEE	Corrects pseudoreplication; effective sample size reflects true independent information, not raw row count
Large (n in the thousands+)	Standard frequentist regression/Cox models; ML ensembles without heavy regularization	Asymptotic approximations (Wald CIs, MLE normality) become reliable
Very large (n in the tens of thousands to millions), or transfer learning available	Deep learning (CNN/RNN/Transformer); foundation models fine-tuned on a smaller local set	Sufficient data (or borrowed pretraining data) to support high effective parameter counts without overfitting

7.15 Master Comparison Table — Choosing the Right Model

Scenario	Recommended Model(s)	Why
Single time-point outcome, moderate n , need interpretability	Logistic/linear regression, Elastic Net if p is large	Simple, interpretable, well-understood inference

Scenario	Recommended Model(s)	Why
Repeated measures, clinical trial, $n < 500$	Linear mixed-effects model	Valid inference on treatment effect with within-subject correlation
Very large longitudinal EHR data, prediction-focused	LSTM / MERF / GP / deep mixed models	Captures nonlinearity at scale; inference is secondary goal
Rare event / very small n	Bayesian model with weakly informative prior	Valid posterior uncertainty when frequentist asymptotics fail
Confirmatory Phase III endpoint	Frequentist Cox/mixed model	Regulatory precedent, well-defined Type I error control
Tabular omics/clinical features, moderate-large n	XGBoost/LightGBM/RF + SHAP	High accuracy, native nonlinearity, interpretable post-hoc
Raw imaging data	CNN (transfer learning from foundation model)	Learns spatial features automatically
Raw sequence/protein/long-range dependency data	Transformer (foundation model, e.g., ESM2, scGPT)	Captures long-range dependencies, transfer learning
Multiple heterogeneous modalities available	Late fusion (heterogeneous/missing data) or joint fusion (dense multimodal data)	Combines complementary information
Single-cell heterogeneity	Seurat/Scanpy pipeline + Harmony/scVI integration	Standard, validated, batch-robust
Germline variant discovery	GATK Best Practices	Community-standard, well-validated
Large-cohort rapid variant calling	bcftools	Fast, lightweight, scales well

7.16 Best-Practice Guidelines for Model Selection

1. **Match model complexity to sample size.** With $p \gg n$ (typical omics) or small clinical trials, favor regularized linear models or mixed models over deep learning; reserve DL for large- n , high-dimensional raw data (images,

sequence) or when strong pretrained foundation models exist.

2. **Match the model to the scientific question.** If the goal is a specific, inferentially valid effect estimate (e.g., a drug's effect on a trajectory), use mixed-effects/Cox/GEE with proper inference. If the goal is pure prediction, a wider range of ML/DL methods becomes appropriate.
3. **Respect the independence structure of the data.** Repeated measures, clustered data (patients within clinics, cells within patients), and multi-site data all violate the i.i.d. assumption of simple regression — always identify the correct unit of independence before choosing a model.
4. **Validate splits at the correct level.** For multimodal/imaging/longitudinal data, split train/test by *patient*, not by scan/visit/cell, to avoid data leakage.
5. **Pair interpretability tools with independent validation.** SHAP/LIME/attention weights identify what the model relies on, not necessarily true causal drivers — triangulate with replication, pathway evidence, or causal-inference methods (MR, mediation, DML/TMLE) before making causal claims. For any mediation or observational causal-effect claim specifically, report an **E-value** (Module 2.6) alongside the estimate: it quantifies how strong an unmeasured confounder would need to be to explain the finding away, and a low E-value should downgrade the causal language used, regardless of how the model itself performed.
6. **Always report a sensitivity/robustness check.** Prior sensitivity for Bayesian models; alternative random-effects structures for mixed models; nested cross-validation for high-dimensional ML; harmonization checks for multi-site imaging/omics data.
7. **Consider regulatory context early.** If results will support a regulatory submission, favor well-precedented methods (mixed models/GEE, Cox PH, standard frequentist tests) as the primary confirmatory analysis, with ML/DL or Bayesian methods as supportive/exploratory analyses unless a Bayesian design has been formally pre-specified and agreed with regulators.

These notes are structured as a living curriculum — each module can be expanded into a standalone 2-3 hour lecture with additional derivations, in-class exercises, and dataset-specific labs (e.g., a full TCGA breast cancer multi-omics lab session, or a UK Biobank GWAS practicum).

8 Programming Languages and Computational Tools

8.1 Summary Table

Category	Tools	Primary Use
Python	pandas, NumPy, scikit-learn, matplotlib/seaborn	Data wrangling, ML, visualization
R	tidyverse, Bioconductor, lme4, survival, ggplot2, Shiny	Statistical modeling, omics analysis, interactive apps
HPC	SLURM/PBS job scheduling	Large-scale parallel jobs
SAS	PROC steps, macros	Regulatory/clinical trial biostatistics
Bash/Linux	Shell scripting, pipelines	Automation, reproducibility
PLINK/PRSice/MatrixEQTL/GWAS, PRS, eQTL	mapping	Statistical genetics
STAR/HISAT2/Bowtie2	RNA-seq/DNA alignment	Read mapping
Docker/Git	Containerization, version control	Reproducibility, collaboration
SQL	Querying relational databases	EHR/clinical data extraction
Jupyter	Interactive notebooks	Exploratory analysis, teaching

8.1.1 Example: PRS Calculation (PRSice-2)

```
Rscript PRSice.R --prsice PRSice_linux \
  --base gwas_summary.txt --target target_data \
  --binary-target T --stat OR --or --pvalue P
```

8.1.2 Example: eQTL Mapping (MatrixEQTL, R)

```
library(MatrixEQTL)
me <- Matrix_eQTL_engine(snps = snps, gene = gene_expr, cvrt = covariates,
                        output_file_name = "eqtl_results.txt",
                        pvOutputThreshold = 1e-5, useModel = modelLINEAR)
```

8.1.3 Reproducibility Best Practices

- Version control every analysis script (Git/GitHub); tag releases matching manuscript submissions.
- Containerize environments (Docker/Singularity) to freeze software versions for reproducibility across HPC/cloud.
- Use renv (R) or conda/poetry (Python) for dependency locking.

8.2 References

R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>

Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the Tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Huber, W., Carey, V. J., Gentleman, R., Anders, S., Carlson, M., Carvalho, B. S., Bravo, H. C., Davis, S., Gatto, L., Girke, T., Gottardo, R., Hahne, F., Hansen, K. D., Irizarry, R. A., Lawrence, M., Love, M. I., MacDonald, J., Obenchain, V., Oleś, A. K., ... Morgan, M. (2015). Orchestrating high-throughput genomic analysis with Bioconductor. *Nature Methods*, 12(2), 115-121.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8026-8037).

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). TensorFlow: A system for large-scale machine learning.

In *12th USENIX Symposium on Operating Systems Design and Implementation* (pp. 265-283).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2. # Practice Problems and Worked Solutions

1. (Statistical Genetics) A SNP has minor allele frequency 0.25 in controls. Under HWE, what proportion of controls are heterozygous? *Solution:* $2pq = 2(0.25)(0.75) = 0.375$ (37.5%).

2. (Survival Analysis) In a Cox model, $\hat{\beta} = 0.693$ for a binary exposure. Interpret the hazard ratio. *Solution:* $HR = e^{0.693} = 2.0$ — exposed group has 2x the hazard of the event at any time point, holding other covariates constant.

3. (Multiple Testing) You test 20,000 CpGs; the smallest p-value is $p_{(1)} = 2 \times 10^{-6}$. Does it survive Bonferroni correction at $\alpha = 0.05$? *Solution:* Threshold = $0.05/20000 = 2.5 \times 10^{-6}$. Since $2 \times 10^{-6} < 2.5 \times 10^{-6}$, it survives (barely).

4. (Mediation) Genotype $\rightarrow$ Methylation $\rightarrow$ Expression. If $a = 0.4$ (genotype $\rightarrow$ methylation) and $b = -0.5$ (methylation $\rightarrow$ expression, adjusting for genotype), what is the mediated (indirect) effect? *Solution:* Indirect effect = $a \times b = 0.4 \times (-0.5) = -0.20$.

5. (Power Analysis) For a two-sample t-test with expected Cohen's $d = 0.5$, $\alpha = 0.05$, power=0.8, approximately how many subjects per group are needed? *Solution:* Using standard tables/pwr.t.test, $n \approx 64$ per group.

6. (Mixed Models — Interview Style) A clinical trial measures FEV1 at baseline, month 6, and month 12 for 200 patients across 2 treatment arms, with ~15% dropout by month 12. Why is a repeated-measures ANOVA a poor choice here, and what would you use instead? *Solution:* ANOVA requires complete, balanced data — the 15% dropout creates an unbalanced design that ANOVA handles by listwise-deleting incomplete subjects (losing information and risking bias if dropout is related to outcome). A linear mixed-effects model (random intercept $\pm$ slope per patient, fixed effects for time, treatment, and time $\times$ treatment) uses all available data under a maximum-likelihood MAR assumption and directly estimates the treatment effect on the trajectory.

7. (Bayesian Reasoning) A rare-variant association study has only 8 carri-

ers of a candidate variant. Why might a Bayesian approach be preferred here over a standard frequentist test? *Solution:* With only 8 carriers, asymptotic frequentist tests (e.g., Wald tests relying on large-sample normality) are unreliable, and exact frequentist tests can be very conservative/low-powered. A Bayesian approach with a weakly informative prior can still yield a full posterior distribution over the effect size, appropriately reflecting the wide uncertainty from the small sample rather than forcing a binary “significant/not significant” call, and can borrow strength via hierarchical priors across similar variants/genes.

8. (Explainable AI) In a SHAP summary plot for an XGBoost model predicting asthma exacerbation risk, two co-methylated CpGs each show moderate, similar SHAP importance. A collaborator asks, “which one is the real driver?” How do you respond? *Solution:* SHAP importance reflects model reliance, and correlated features split attribution between them — moderate importance on both doesn’t rule out that one (or neither) is the true causal driver. I’d recommend orthogonal validation: check if either CpG maps to a *cis*-eQTM gene with independent genetic support (e.g., via mediation/MR), check replication in an independent cohort, and consider conditional/permutation-based importance that accounts for the correlation structure before attributing causal significance to either.

9. (Ensemble Methods) You have an omics dataset with $n=300$ samples and $p=15,000$ features (CpGs). A colleague proposes running LightGBM directly. What would you advise? *Solution:* LightGBM’s speed advantage matters most at large n (hundreds of thousands to millions of rows); with $n=300$ and $p=15,000$ (severe $p \gg n$), the bigger risk is overfitting regardless of which gradient-boosting implementation is used. I’d recommend starting with a strong regularized linear baseline (Elastic Net) and univariate pre-filtering, use nested cross-validation for any tree-ensemble model, and reserve LightGBM’s speed benefits for later work at larger sample sizes (e.g., pooled multi-cohort analyses).

10. (scRNA-seq) After integrating three 10x Genomics samples with Harmony, a cluster appears that is composed almost entirely of cells from a single sample. Is this a batch effect or a real biological finding? *Solution:* This is ambiguous without further evidence and should not be assumed either way. Check: (a) does the cluster express canonical, biologically meaningful marker genes consistent with a real cell state/type, or does it show high mitochondrial %/low gene counts suggestive of a QC/technical artifact; (b) was that sample processed differently (different day, reagent lot, or clinical condition uniquely present in that sample); (c) does adjusting the Harmony integration parameters (theta) merge or preserve the cluster. A sample-exclusive cluster driven by a unique clinical condition (e.g., only that sample came from a diseased tissue) is

a real and important finding; one driven by QC metrics or protocol differences is a technical artifact.

9 Cross-Chapter Method Selection Guide

Research Question	Recommended Method(s)
Is a SNP associated with disease risk?	GWAS (logistic regression + PC adjustment)
Does methylation regulate gene expression?	eQTM (linear regression), mediation analysis
Is there a causal effect of an exposure?	Mendelian Randomization
Which genes drive disease heterogeneity?	WGCNA, MOFA+, DIABLO
How to predict individual risk?	XGBoost/RF/Elastic Net + SHAP for interpretability
Time-to-event outcome?	Kaplan-Meier + Cox PH
Repeated measures over time?	Linear mixed-effects models
Single-cell heterogeneity?	Seurat/Scanpy clustering + trajectory inference
Combine multiple published studies?	Random-effects meta-analysis

10 Glossary of Key Terms

Allele Frequency — The relative frequency of a given variant (allele) at a genetic locus in a population.

AUC (Area Under the ROC Curve) — A discrimination metric equal to the probability that a randomly chosen case receives a higher predicted risk score than a randomly chosen control; ranges from 0.5 (no discrimination) to 1.0 (perfect discrimination).

Bayesian Inference — A framework for statistical inference in which parameters are treated as random variables with a prior distribution, updated by observed data into a posterior distribution via Bayes' theorem.

Benjamini-Hochberg (BH) Procedure — A method for controlling the false discovery rate (FDR) across many simultaneous hypothesis tests, less conservative than Bonferroni correction.

Bonferroni Correction — A multiple-testing correction that divides the significance threshold by the number of tests performed, controlling the family-wise error rate.

Calibration — The agreement between a model's predicted probabilities and the observed frequency of the outcome; distinct from discrimination (AUC), which only measures ranking ability.

ComBat / ComBat-seq — Empirical Bayes methods for removing batch effects from continuous (ComBat) or count-based (ComBat-seq) omics data while preserving biological variation of interest.

Cox Proportional Hazards Model — A semi-parametric survival model relating the hazard of an event to covariates via $h(t|X) = h_0(t) \exp(\beta X)$, assuming the hazard ratio between groups is constant over time.

Credible Interval — The Bayesian analogue of a confidence interval; a range within which the true parameter value lies with a stated posterior probability (e.g., 95%).

Cross-Validation — A model-validation technique that partitions data into folds, training on some folds and testing on the held-out fold(s) to estimate out-of-sample performance.

DICOM — Digital Imaging and Communications in Medicine; the standard file format and protocol for medical imaging data.

Differential Expression (DE) — Statistical identification of genes whose expression levels differ systematically between conditions, typically via count-based models (DESeq2, edgeR).

Double Machine Learning (DML) — A causal-inference framework that combines flexible machine-learning models for nuisance parameters with a debiasing step, yielding valid statistical inference for treatment-effect estimation.

E-value — A sensitivity-analysis statistic quantifying the minimum strength of association an unmeasured confounder would need to have with both exposure and outcome to fully explain away an observed effect.

EHR (Electronic Health Record) — Digitized patient health information (diagnoses, labs, medications, notes) used for clinical research and predictive modeling.

Elastic Net — A regularized regression method combining LASSO (L1) and Ridge (L2) penalties, useful in high-dimensional ($p \gg n$) settings.

eQTL (expression Quantitative Trait Locus) — A genetic variant associated with variation in gene expression levels.

eQTM (expression Quantitative Trait Methylation) — A statistical association between a DNA methylation site (CpG) and the expression level of a nearby or distant gene.

FDR (False Discovery Rate) — The expected proportion of false positives among all findings called statistically significant.

Fine-Mapping (SuSiE, FINEMAP) — Statistical methods for narrowing a genetic association signal down to the most likely causal variant(s) within a locus.

GATK (Genome Analysis Toolkit) — A widely used software suite for variant discovery in high-throughput sequencing data, implementing the “Best Practices” germline and somatic calling pipelines.

GEE (Generalized Estimating Equations) — A regression approach for correlated/repeated-measures data that estimates population-average effects and remains robust to misspecification of the within-subject correlation structure.

GWAS (Genome-Wide Association Study) — A study design testing association between genetic variants across the genome and a phenotype of interest, typically using a genome-wide significance threshold of $p < 5 \times 10^{-8}$.

Harmony / scVI — Batch-integration methods for single-cell RNA-seq data that align cells from different samples/batches into a shared embedding while preserving true biological variation.

Heritability (h^2) — The proportion of phenotypic variance in a population attributable to additive genetic variance.

Hardy-Weinberg Equilibrium (HWE) — The expected genotype frequencies ($p^2, 2pq, q^2$) in a population under random mating, absent selection, mutation, or migration; deviations can indicate genotyping error or population stratification.

Linkage Disequilibrium (LD) — Non-random association between alleles at different loci, typically quantified by D' or r^2 .

LASSO (Least Absolute Shrinkage and Selection Operator) — A regularized regression method using an L1 penalty that shrinks some coefficients exactly to zero, performing implicit variable selection.

LIME (Local Interpretable Model-agnostic Explanations) — A post-hoc explainability method that fits a simple, interpretable surrogate model locally around a specific prediction.

LSTM (Long Short-Term Memory) — A recurrent neural network architecture with gating mechanisms designed to capture long-range dependencies in sequential data.

MAF (Minor Allele Frequency) — The frequency of the less common allele at a genetic locus in a population.

Mediation Analysis — A causal-inference method decomposing a total effect of an exposure on an outcome into a direct effect and an indirect effect operating through a specified mediator.

Mendelian Randomization (MR) — A causal-inference method using genetic variants as instrumental variables for an exposure, exploiting the random assortment of alleles at conception.

Mixed-Effects Model — A regression model that includes both fixed effects (population-average parameters) and random effects (subject- or cluster-specific deviations), used for repeated-measures/longitudinal or clustered data.

MOFA / MOFA+ (Multi-Omics Factor Analysis) — A Bayesian latent-factor model for integrating multiple omics data types into shared and modality-specific factors.

mQTL (methylation Quantitative Trait Locus) — A genetic variant associated with variation in DNA methylation level at a specific CpG site.

Multiple Testing Correction — Statistical adjustment applied when performing many simultaneous hypothesis tests, to control either the family-wise error rate (Bonferroni) or the false discovery rate (Benjamini-Hochberg).

Negative Control Outcome — An outcome that a proposed exposure/mediator should have no plausible biological effect on, used as a sensitivity check for residual confounding.

Posterior Distribution — In Bayesian inference, the updated probability distribution of a parameter after combining the prior distribution with the observed data's likelihood.

PRS (Polygenic Risk Score) — A single score summarizing an individual's genetic risk for a trait or disease, computed as a weighted sum of risk allele dosages across many loci.

Radiomics — The extraction of quantitative features (shape, texture, intensity) from medical images for use in statistical or machine-learning models.

Random Forest — An ensemble learning method that aggregates predictions from many decision trees trained on bootstrap samples with random feature subsampling.

scRNA-seq (single-cell RNA sequencing) — A sequencing technology measuring gene expression at individual-cell resolution, enabling characterization of cellular heterogeneity.

Sequential Ignorability — The core, untestable assumption underlying mediation analysis: no unmeasured confounding of the exposure-mediator, exposure-outcome, and mediator-outcome relationships.

SHAP (SHapley Additive exPlanations) — A model-interpretability method grounded in cooperative game theory (Shapley values) that attributes a prediction to each input feature in a locally accurate and consistent manner.

Variant Calling — The computational process of identifying genetic variants (SNVs, indels) from aligned sequencing reads relative to a reference genome.

WGCNA (Weighted Gene Co-expression Network Analysis) — A method for identifying clusters (modules) of highly co-expressed genes and relating them to external traits.

XGBoost / LightGBM — Gradient-boosted decision tree implementations that sequentially fit trees to residual errors, widely used for high-accuracy tabular/omics prediction tasks.

Index

- 850K/EPIC Array QC Pipeline, [26](#)
- A Quick Guide to This Book's Mathematical Notation, [138](#)
- Accelerated Failure Time (AFT) Models, [44](#)
- Applications, [70](#)
- Artificial Intelligence and Machine Learning, [64](#)
- Assay Performance Modeling, [10](#)
- Assumption Common to Nearly All Classical Survival Models, [37](#)
- Batch Effects in RNA-seq, [25](#)
- Bayesian Survival Models, [52](#)
- Bayesian vs.-Frequentist Methods, [5](#)
- Best-Practice Guidelines for Model Selection, [144](#)
- Bias-Variance Tradeoff, [64](#)
- Biomarker Discovery, [27](#)
- Biostatistics and Statistical Modeling, [5](#)
- Career Application, [11](#)
- Causal Frameworks (relevant to your CIMLA/TRACE work), [71](#)
- Causal Inference and Mediation Modeling, [71](#)
- Cell-Type Heterogeneity — Why It Matters, [26](#)
- Censoring Types, [36](#)
- Chapter Summary: Choosing a Survival Model, [60](#)
- Chapter Summary: Foundation Model Selection Guide, [106](#)
- Choosing a Correction: It Depends on Whether the Analysis Is Hypothesis-Driven or Discovery-Driven, [11](#)
- Choosing Between Ensembles and Deep Learning — A Practical Decision Rule, [65](#)
- cis vs.-trans eQTM/mQTL — Why Significance Thresholds Differ, [27](#)
- CLIP (Contrastive Language-Image Pretraining), [92](#)
- CNN (for imaging/sequence motifs), [66](#)
- Code, [18](#)
- Code (Python — basic MRI preprocessing with SimpleITK/nibabel), [82](#)
- Code (Python — conceptual multimodal DeepSurv), [60](#)
- Code (Python — pycox), [58](#)
- Code (Python — PyRadiomics (van Griethuysen et al., 2017) feature extraction), [83](#)
- Code (Python — PyTorch CNN skeleton), [66](#)
- Code (Python — Scanpy), [76](#)
- Code (Python — scikit-survival), [55](#)
- Code (Python — simple late-fusion skeleton), [85](#)
- Code (Python — XGBoost), [65](#)
- Code (Python), [69](#)
- Code (R — cell-type deconvolution), [27](#)
- Code (R — DESeq2 with batch covariate), [25](#)

- Code (R — glmnet), [10](#)
- Code (R — LightGBM style via lightgbm pkg), [65](#)
- Code (R — lme4), [9](#)
- Code (R — metafor), [33](#)
- Code (R — MOFA+), [22](#)
- Code (R — pwr), [10](#)
- Code (R — ROC/AUC), [28](#)
- Code (R — Seurat), [76](#)
- Code (R — survival), [7](#)
- Code (R — TwoSampleMR), [32](#)
- Code (R), [9](#)
- Common Technical Pitfalls in Cross-Platform Integration, [23](#)
- Comparison Table — DL Architecture Selection, [131](#)
- Comparison Table — Mixed-Effects vs.-ML/DL for Longitudinal Data, [115](#)
- Competing Risks (Fine-Gray Model), [46](#)
- Comprehensive Model Reference Guide, [110](#)
- Computational Biology and Bioinformatics, [17](#)
- Conceptual Foundation, [17](#)
- Core Assumptions, [32](#)
- Core Definitions, [36](#)
- Cox Proportional Hazards Model, [7](#)
- CpG / Gene / Variant Prioritization, [33](#)
- Cross-Chapter Method Selection Guide, [151](#)
- Cross-Cutting Assumptions, Advantages, and Limitations, [107](#)
- Cross-Validation, [73](#)
- Deep Learning, [66](#)
- Deep Learning (CNN, RNN, LSTM, Transformers), [129](#)
- Deep-Learning Survival Models, [56](#)
- DESeq2 vs.-edgeR — Practical Differences, [25](#)
- Diagnostic Test Evaluation, [10](#)
- DNA Methylation QC and Cell-Type Heterogeneity, [26](#)
- Ensemble Methods, [64](#)
- Ensemble Models (XGBoost, LightGBM, Random Forest), [126](#)
- Example, [84](#)
- Example Result — AFT (Weibull) Output, [45](#)
- Example Result — Architecture Choice on a Chest X-ray Classification Task (Simulated), [132](#)
- Example Result — Cluster Marker Gene Table, [75](#)
- Example Result — Concordance Between SHAP and Known Clinical Risk Factors, [68](#)
- Example Result — Cox Model Output Table, [40](#)
- Example Result — DeepSurv vs.-Cox Discrimination and Calibration, [58](#)
- Example Result — Ensemble Comparison on Simulated Omics Data (n=500, p=200), [128](#)
- Example Result — Extracted Radiomic Features (Simulated), [122](#)
- Example Result — Fine-Gray Model Output, [47](#)
- Example Result — Joint Model Association Parameter, [52](#)
- Example Result — Log-Rank Test Output, [38](#)
- Example Result — Mixed-Effects

- Model Output, [8](#)
- Example Result — Model Discrimination Comparison, [55](#)
- Example Result — Model Selection by AIC, [44](#)
- Example Result — Multimodal Survival Benchmark (Simulated Oncology Cohort), [59](#)
- Example Result — Posterior Sensitivity Analysis, [119](#)
- Example Result — Recurrent Exacerbations (Andersen-Gill Model), [49](#)
- Example Result — Same Data, Two Framings, [116](#)
- Example Result — Unimodal vs.-Fused Model Benchmark (Simulated), [125](#)
- Example Result — VCF Summary Statistics (bcftools stats, Simulated Trio), [20](#)
- Example: eQTL Mapping (MatrixEQTL, R), [147](#)
- Example: PRS Calculation (PRSice-2), [146](#)
- Experimental Design and Power Analysis, [10](#)
- Explainable AI (SHAP, LIME), [67](#)
- Fixed vs.-Random Effects, [33](#)
- Formula (Two-Sample MR, Inverse-Variance Weighted), [32](#)
- Formula — Contrastive Alignment Loss (used in multimodal foundation models, e.g., CLIP-style), [84](#)
- Foundation Models for Computational Biology and Medicine, [88](#)
- Foundations of Time-to-Event Data, [36](#)
- Frailty Models (Random-Effects Survival Models), [50](#)
- Frequentist vs.-Bayesian Inference, [115](#)
- From Statistically Significant to Clinically Actionable, [28](#)
- Genomic Data Processing, [19](#)
- Genomics Foundation Models (DNABERT, Geneformer), [99](#)
- Glossary of Key Terms, [152](#)
- GPT-Style Language Models (LLMs), [89](#)
- Gradient Boosting (XGBoost, Chen and Guestrin, 2016; LightGBM, Ke et al., 2017), [64](#)
- Graph Foundation Models (GNN-FM), [104](#)
- Graph Neural Networks, [74](#)
- GWAS Statistical Model, [17](#)
- Heterogeneity, [33](#)
- High-Dimensional Modeling (LASSO, Tibshirani, 1996; Elastic Net, Zou and Hastie, 2005), [9](#)
- Hypothesis Testing, [10](#)
- Image Preprocessing Pipeline (Raw → Analysis-Ready), [82](#)
- Image Preprocessing Pipeline (Raw → QC → Normalization → Segmentation → Feature Extraction → Modeling), [120](#)
- Infectious Disease Genomics, [29](#)
- Interpretation Guidelines, [19](#)
- Is Bayesian Inference Valid Without Strong Prior Evidence?, [117](#)
- Joint Models (Longitudinal + Survival), [51](#)

-
- Kaplan-Meier Estimator, [7](#)
 - Key Methods, [29](#)
 - Key Metrics, [27](#)
 - Large Language Models (LLMs) for Biological Data, [70](#)
 - Logistic / Linear Regression, [9](#)
 - Longitudinal / Mixed-Effects Models (Laird and Ware, 1982; Pinheiro and Bates, 2000), [7](#)
 - Loss Functions, [64](#)
 - Machine Learning / Deep Learning as Alternatives to Mixed-Effects Models, [112](#)
 - Machine-Learning Survival Models, [54](#)
 - Master Comparison Table — Choosing the Right Model, [143](#)
 - Mediation Model, [71](#)
 - Mendelian Randomization (Davey Smith and Ebrahim, 2003), [32](#)
 - Meta-Analysis, [33](#)
 - Methods, [31](#)
 - Missing Data Across Omics Platforms — A Decision Framework, [24](#)
 - Mixed-Effects Models for Longitudinal Data, [111](#)
 - Model Evaluation and Validation, [73](#)
 - Model Selection by Sample Size: Small-n vs.-Large-n Regimes, [139](#)
 - Multi-Block Integration Methods, [22](#)
 - Multi-Omics Integration, [21](#)
 - Multimodal Data Integration and Medical Imaging, [82](#)
 - Multimodal Foundation Models (Perceiver IO, Flamingo, GPT-4o), [102](#)
 - Multimodal Fusion: Imaging + Omics + EHR, [83](#)
 - Multimodal Modeling (Imaging + Omics + EHR), [123](#)
 - Multimodal Survival Models (Image + Omics + EHR Fusion), [58](#)
 - Multiple Testing Correction (Benjamini and Hochberg, 1995), [11](#)
 - Network and Pathway Analysis, [31](#)
 - Parametric Survival Models, [41](#)
 - Pipeline (Pseudocode), [83](#)
 - Pipeline (Raw Reads → DE Results), [25](#)
 - Pitfalls, [21](#)
 - Pitfalls and Best Practices, [18](#)
 - Practical Integration Strategy: Discovery vs.-Confirmation, [23](#)
 - Predictive Modeling — Foundations, [64](#)
 - Preventing Overfitting on High-Dimensional Genomic Data (p textgreater textgreater n) — A Layered Checklist, [65](#)
 - Prioritization Framework, [34](#)
 - Programming Languages and Computational Tools, [146](#)
 - Protein Language Models (ESM-2, ESMFold, UniRep), [95](#)
 - Random Forest (Breiman, 2001), [64](#)
 - Real Data Example, [18](#)
 - Recurrent Event Models (Andersen-Gill, PWP, WLW), [48](#)
 - References, [12](#)
 - Reproducibility Best Practices, [147](#)
 - RNA-seq Differential Expression and Batch Effects, [24](#)

- RNN/LSTM (Hochreiter and Schmidhuber, 1997) (for sequential/longitudinal data), [66](#)
- scRNA-seq Analysis, [74](#)
- scRNA-seq Workflows (Seurat, Scanpy), [134](#)
- Segment Anything Model (SAM), [93](#)
- Sensitivity Analysis for Causal Claims — The Key Assumption Reviewers Probe, [71](#)
- SHAP (Shapley values; Lundberg and Lee, 2017), [68](#)
- Shared Mathematical Foundations, [88](#)
- Simple Linear/Logistic Regression (Baseline for Comparison), [110](#)
- Single-Cell Foundation Models (scGPT), [101](#)
- State-of-the-Art / Emerging Methods, [84](#)
- Statistical Framework — eQTM (expression Quantitative Trait Methylation), [21](#)
- Statistical Genetics and Genomics, [17](#)
- Summary Table, [146](#)
- Summary Table — Fusion Strategy Selection, [85](#)
- Summary Table — Imaging Pipeline Stages, [122](#)
- Summary Table — Metrics by Task, [73](#)
- Summary Table — Model Selection by Sample Size, [142](#)
- Summary Table — Module 3, [11](#)
- Summary Table — Sensitivity Diagnostics for Causal Claims, [72](#)
- Survival Analysis, [7](#)
- Survival Analysis: Classical, Machine-Learning, and Multimodal Methods, [36](#)
- The Core Idea: Effective Parameters vs.-Available Data, [139](#)
- The Validation Ladder (Discovery → Clinical Deployment), [28](#)
- Tools, [31](#)
- Tools Summary Table, [30](#)
- Variant Calling and QC, [30](#)
- Variant Calling Pipelines (GATK, bcftools, SAMtools), [136](#)
- Vision Transformers (ViT), [91](#)
- When Bayesian Models Are Preferred for Small Samples, [141](#)
- When Frequentist Models Need Large Samples, [142](#)
- When Mixed-Effects Models Outperform Simple Regression for Repeated Measures, [142](#)
- Why Decision Trees, Random Forests, and XGBoost Work Well for Small Samples, [140](#)
- Why Deep Learning Requires Large Samples, [141](#)
- Workflow, [19](#)
- Workflow (GWAS Pipeline), [18](#)
- Workflow (Raw → Final), [82](#)
- Workflow (Seurat/Scanpy), [74](#)